

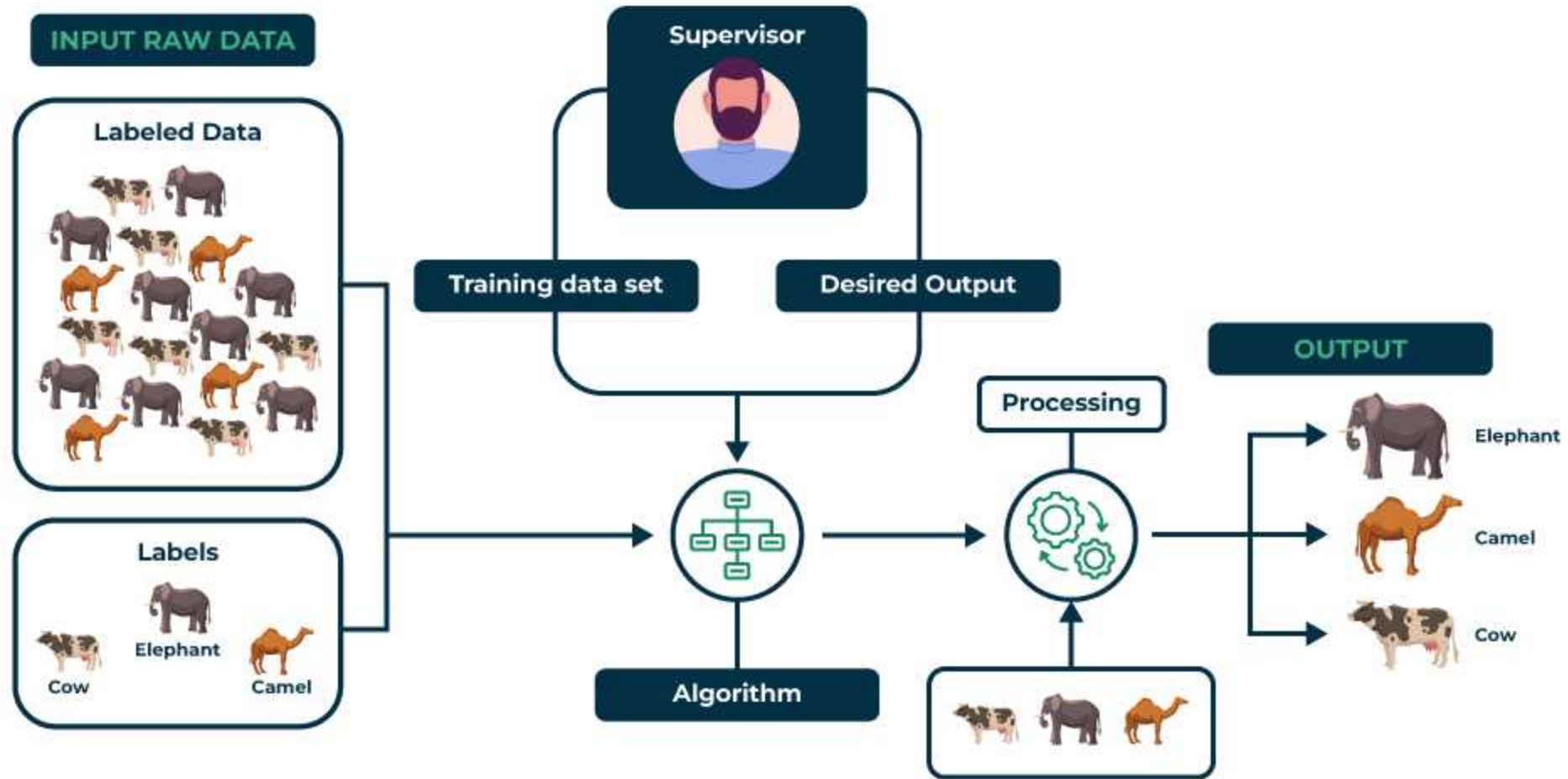
Machine Learning



- ◇ Chapter VIII:
- ◇ Supervised Learning: Part III

◇ Prof. Dr. Aderval S. Luna

Supervised Learning



Supervised Learning

```
graph TD; A[Supervised Learning] --> B[Regression]; A --> C[Classification];
```

The diagram illustrates the structure of Supervised Learning. At the top, a blue rectangular box contains the text 'Supervised Learning'. Two arrows originate from the bottom of this box, pointing downwards to two separate ovals. The left oval is green and contains the text 'Regression'. The right oval is light blue and contains the text 'Classification'.

Regression

Classification

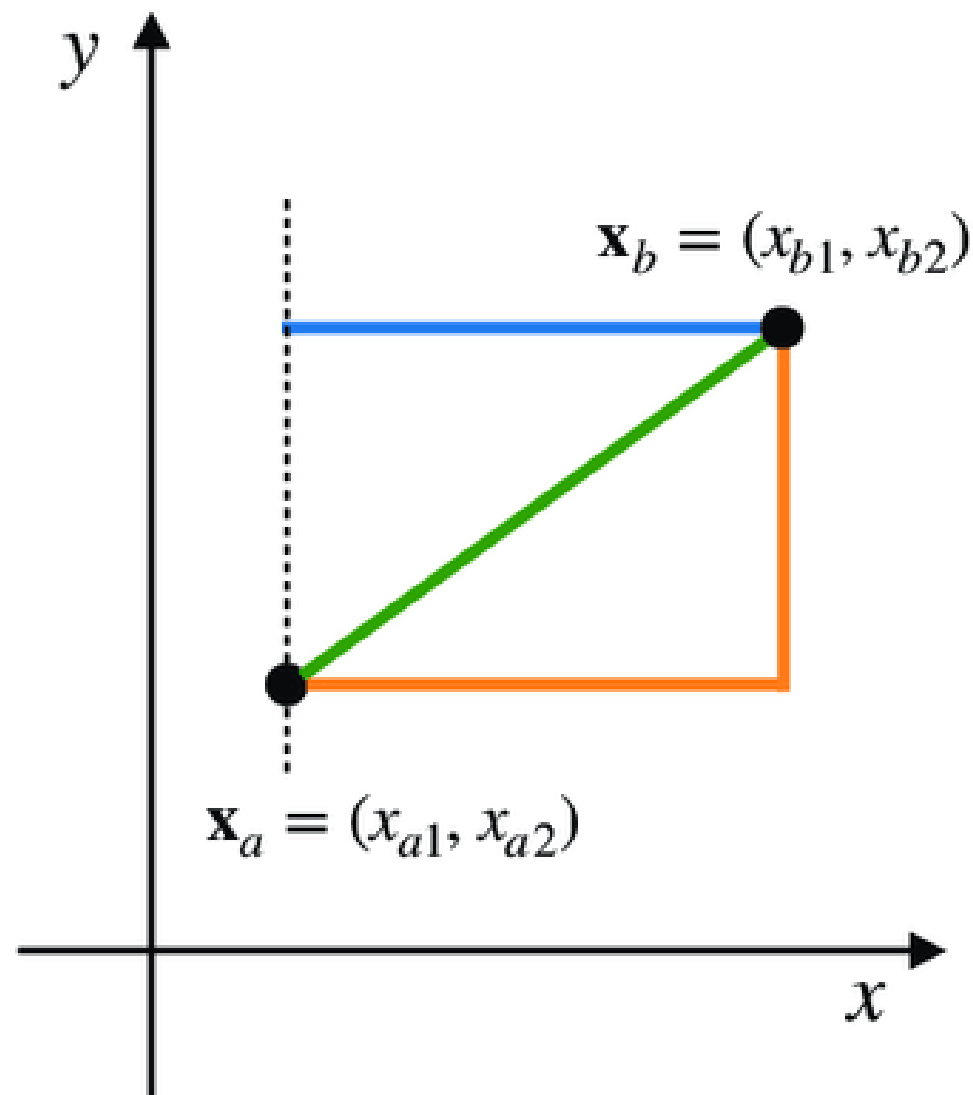
Supervised Learning Algorithms

- ◆ **Dual: Classification and Regression**

- ◆ k-Nearest Neighbors, kNN
- ◆ Ensemble Methods
 - Bagging
 - Boosting
 - Random Forest
- ◆ Support Vector Machines, SVM
- ◆ Artificial Neural Networks, ANN

k-Nearest Neighbors, kNN

- ◆ kNN is one the simplest methods of classification.
- ◆ Let X_c be the training set divided into class subsets, and $\mathbf{x}$ be a new unknown object, which should be classified.
- ◆ We calculated the distances (usually Euclidean) from $\mathbf{x}$ to all samples of the training set ($\mathbf{x}_1, \dots, \mathbf{x}_j$).
- ◆ Then, we select kNN, which are located at minimal distances.
- ◆ A new object $\mathbf{x}$ belongs to a class, encompassing most k neighbors.
- ◆ The parameter k is chosen empirically.
- ◆ An increase in k reduces the impact of errors, whereas its decrease worsens classification.



$p = 2$ Euclidean distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_2 = (|x_{a1} - x_{b1}|^2 + |x_{a2} - x_{b2}|^2)^{\frac{1}{2}}$$

$p = 1$ Manhattan distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_M = |x_{a1} - x_{b1}| + |x_{a2} - x_{b2}|$$

$p = \infty$ Chebyshev distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_\infty = \max\{|x_{a1} - x_{b1}|, |x_{a2} - x_{b2}|\}$$

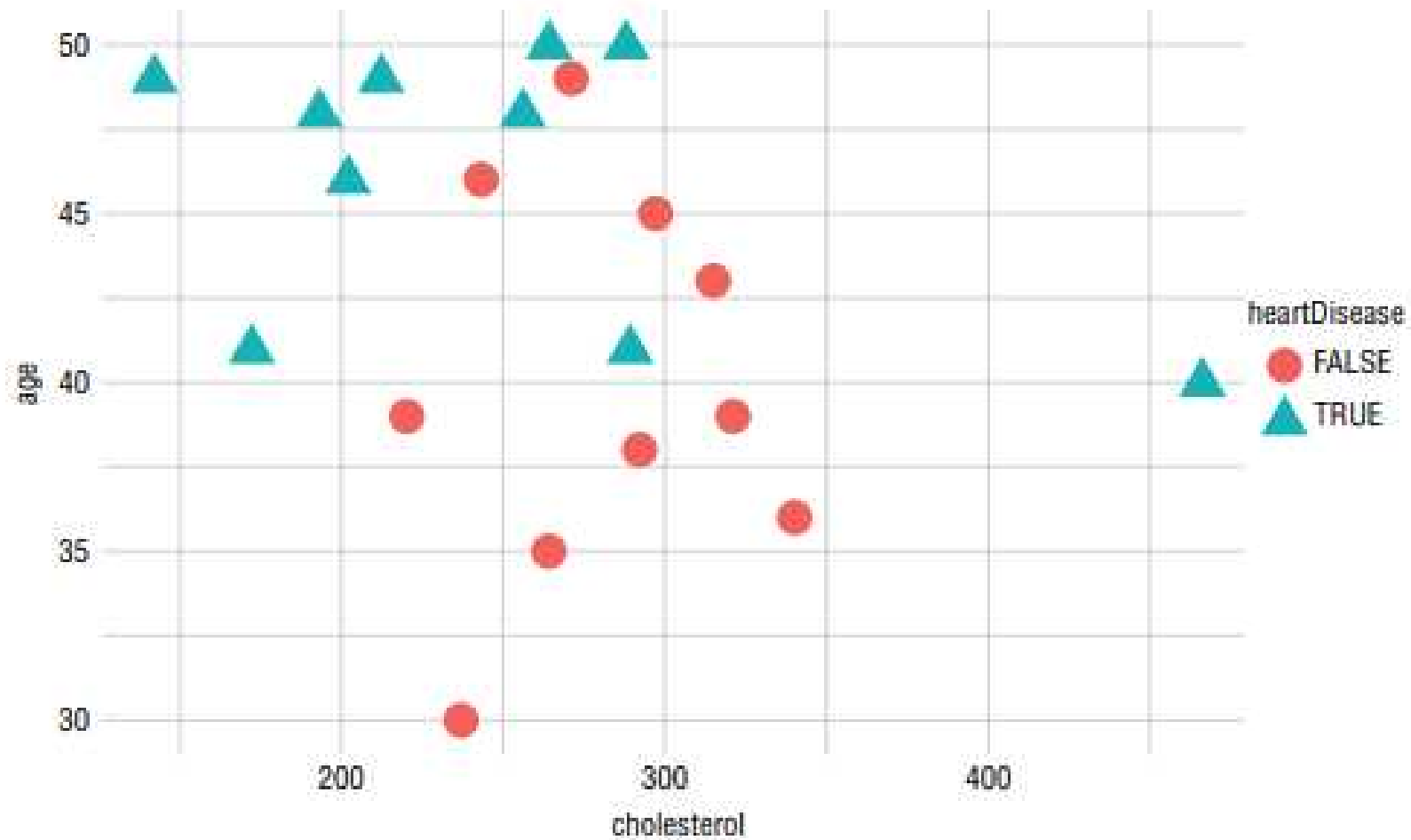


Figure 6.1 Scatterplot of age versus cholesterol levels for a sampling of 20 patients from our dataset. The shape and color of each point indicates whether the patient suffers from heart disease.

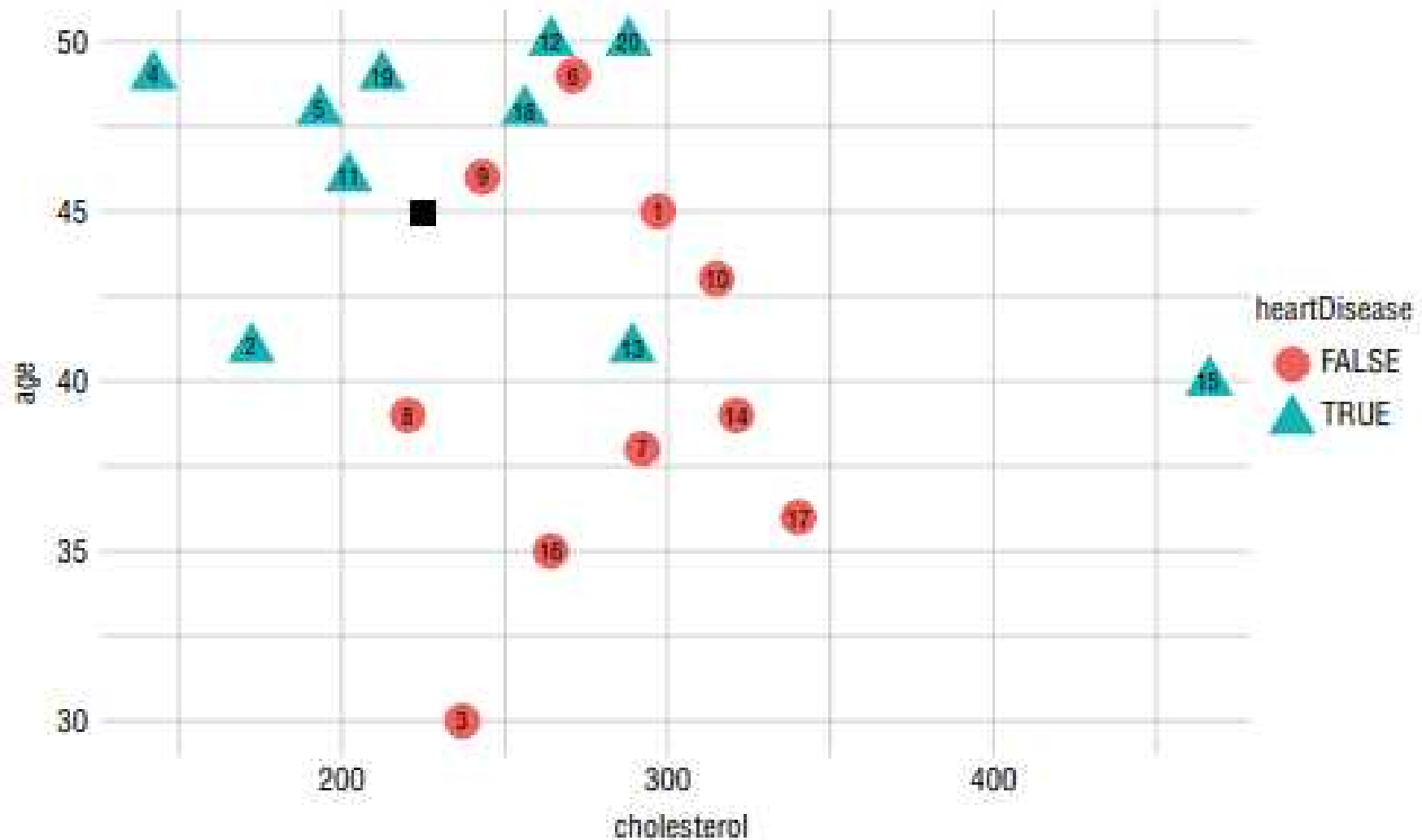


Figure 6.2 Scatterplot of age versus cholesterol levels for a sampling of 20 patients from our dataset and the new patient to be classified (black square)

Applying this approach to the data points representing all 20 patients in our sample gives us the following results, which show the distance between each existing patient and the new patient sorted in ascending order of distance:

Patient	Age	Cholesterol	Age (Normalized)	Cholesterol (Normalized)	Distance to P_{new}	heart- Disease
11	46	202	0.818	0.206	0.081	TRUE
9	46	243	0.818	0.306	0.088	FALSE
5	48	193	0.909	0.184	0.172	TRUE
18	48	256	0.909	0.337	0.182	TRUE
1	45	297	0.773	0.437	0.188	FALSE
2	41	172	0.591	0.133	0.197	TRUE
19	49	212	0.955	0.231	0.205	TRUE
13	41	289	0.591	0.417	0.231	TRUE
6	49	271	0.955	0.374	0.239	FALSE
10	43	315	0.682	0.481	0.240	FALSE
8	39	220	0.500	0.250	0.250	FALSE
12	50	264	1.000	0.357	0.272	TRUE
4	49	142	0.955	0.061	0.279	TRUE
20	50	288	1.000	0.415	0.300	TRUE
7	38	292	0.455	0.425	0.343	FALSE
14	39	321	0.500	0.495	0.350	FALSE
16	35	264	0.318	0.357	0.445	FALSE
17	36	340	0.364	0.541	0.484	FALSE
15	40	466	0.545	0.847	0.631	TRUE
3	30	237	0.091	0.291	0.660	FALSE

The results show that patient 11 is the closest in distance (nearest neighbor) to our hypothetical new patient, while patient 3 is the furthest away from our new patient.

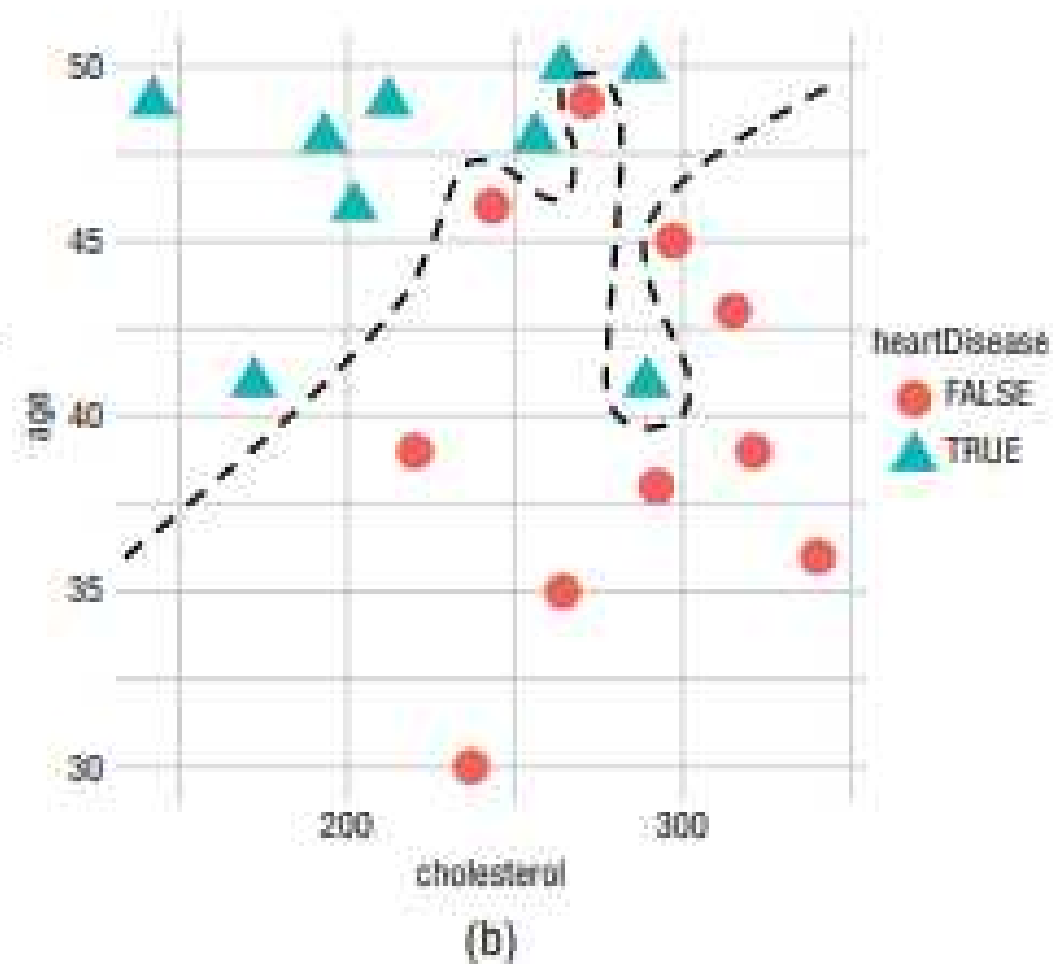
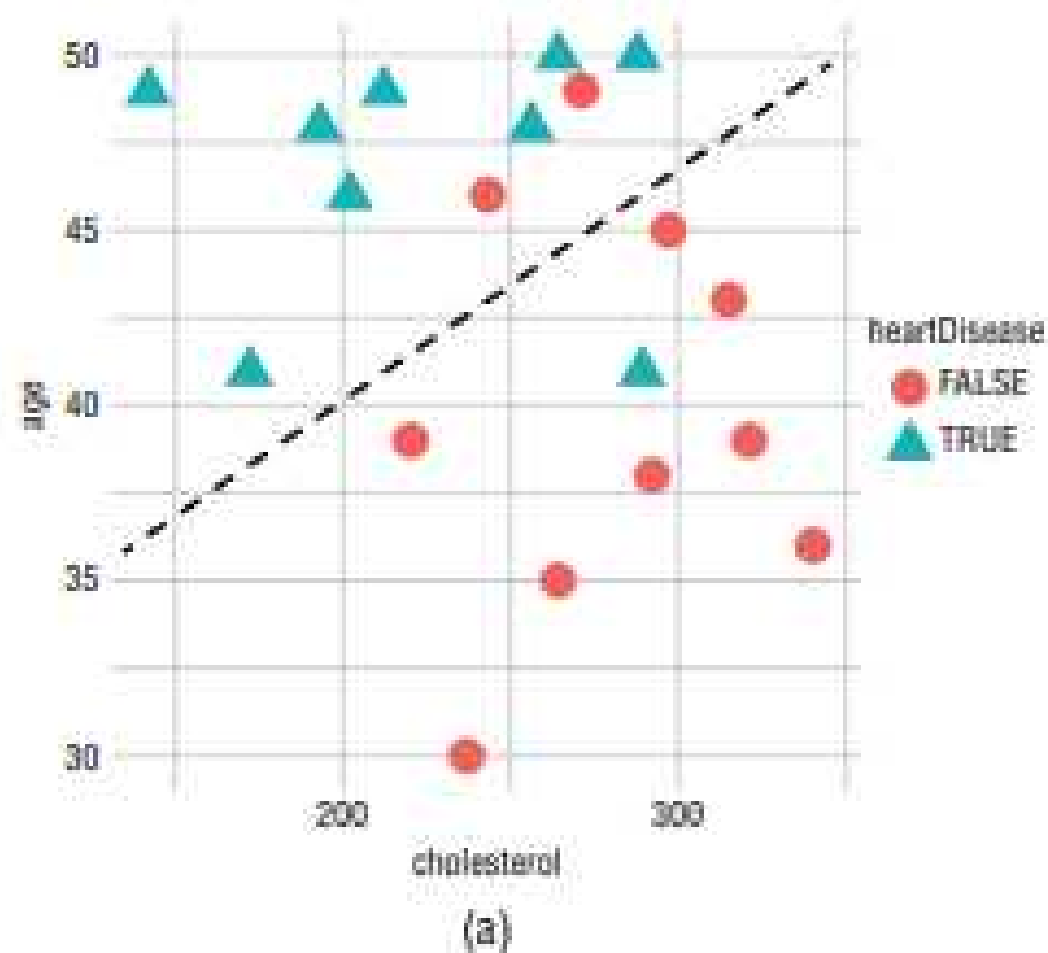
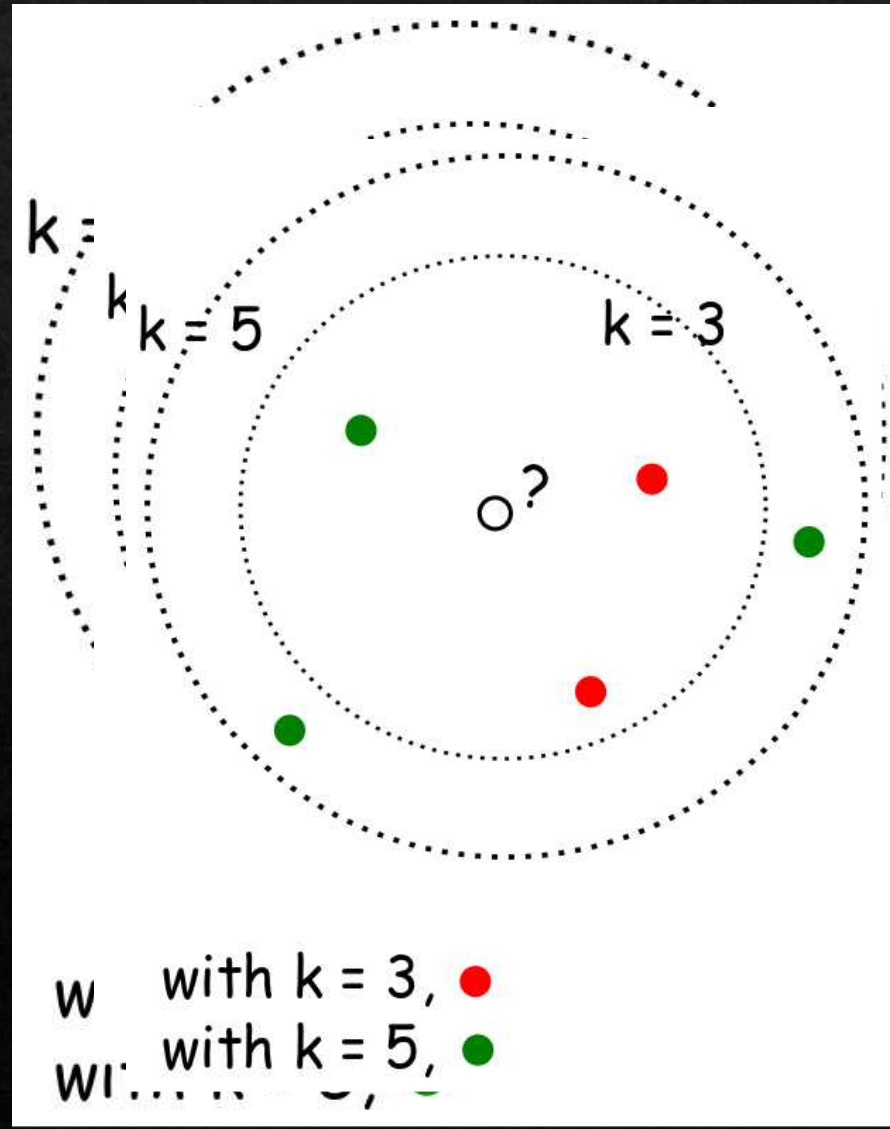
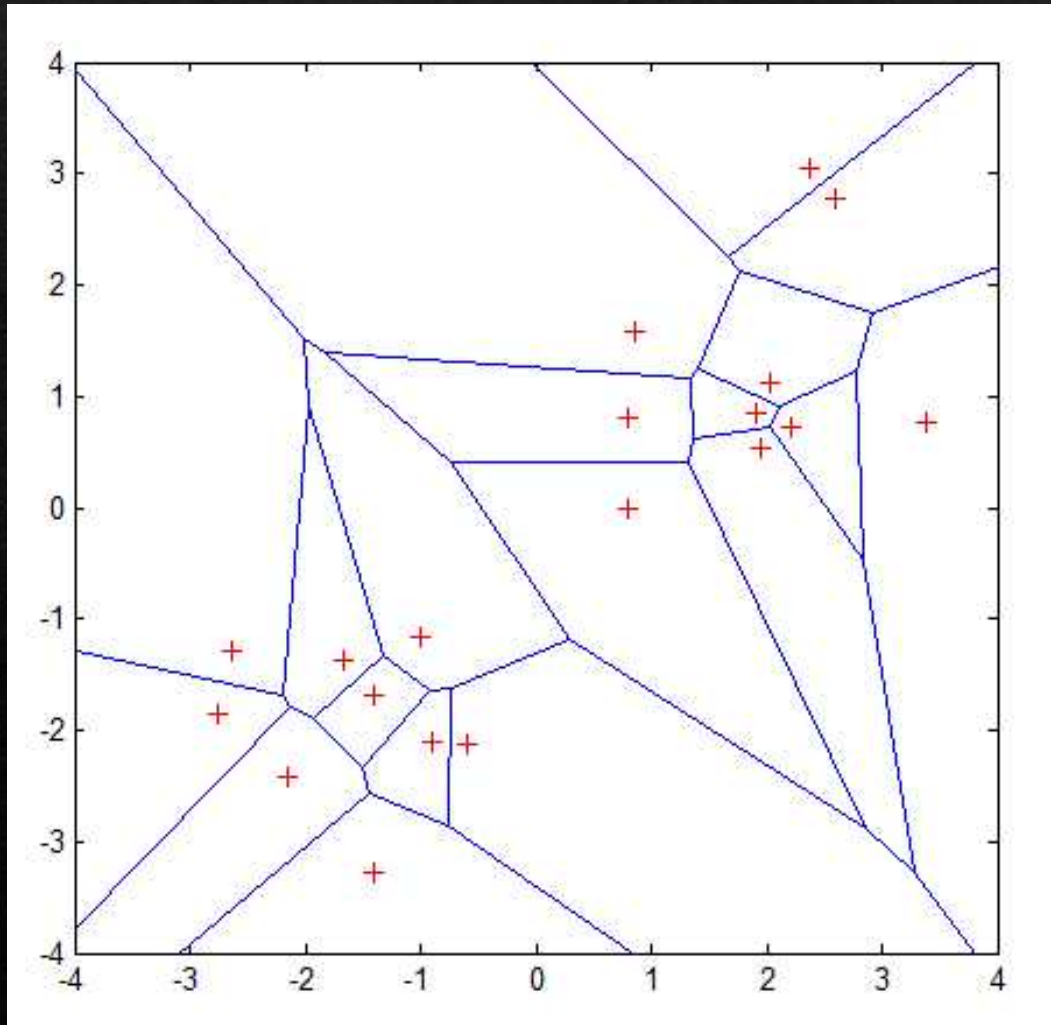


Figure 6.3 The impact of a large value for k (a) and a small value for k (b) on the decision boundary (dashed line) of a model

How to choose the k-NN?



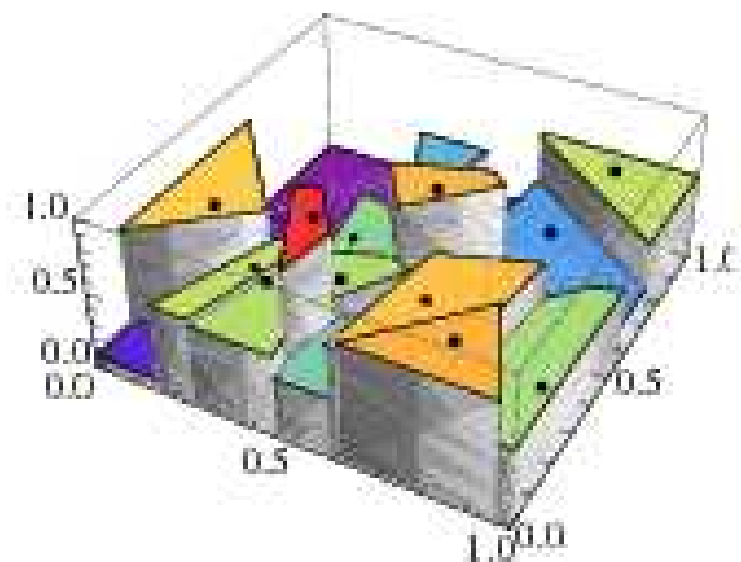
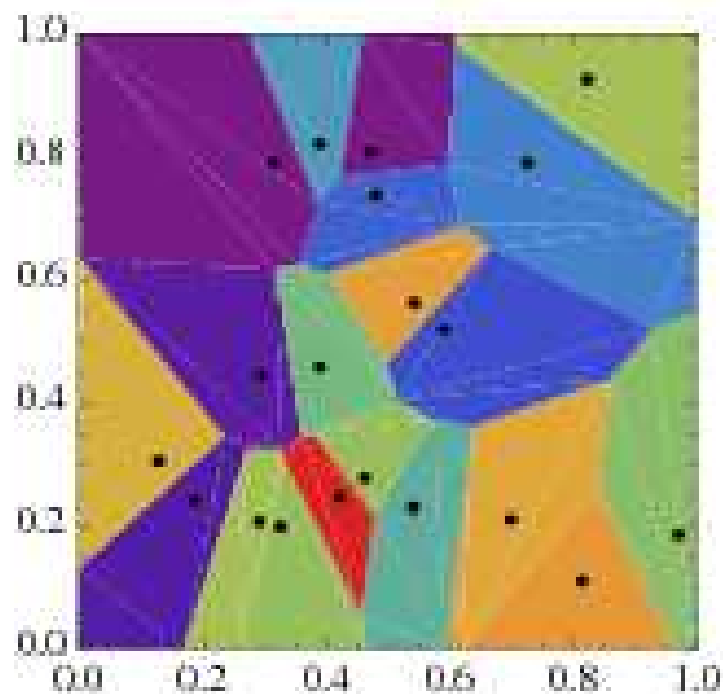
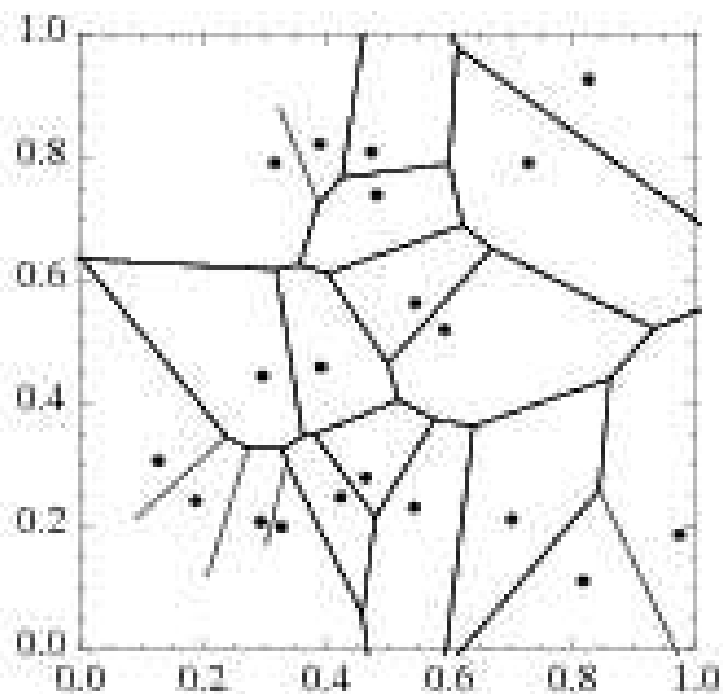
Graphic Representation



Properties:

- 1) All the possible points within the Voronoi cell of a sample are only the most visible points of the sample
- 2) 2) For any sample, the closest sample is determined on the edge of the closest Voronoi cell

Georgy Feodosevich Voronoy (Геóргий Феодóсьевич Воронóй)
24/04/1868 – 20/11/1908



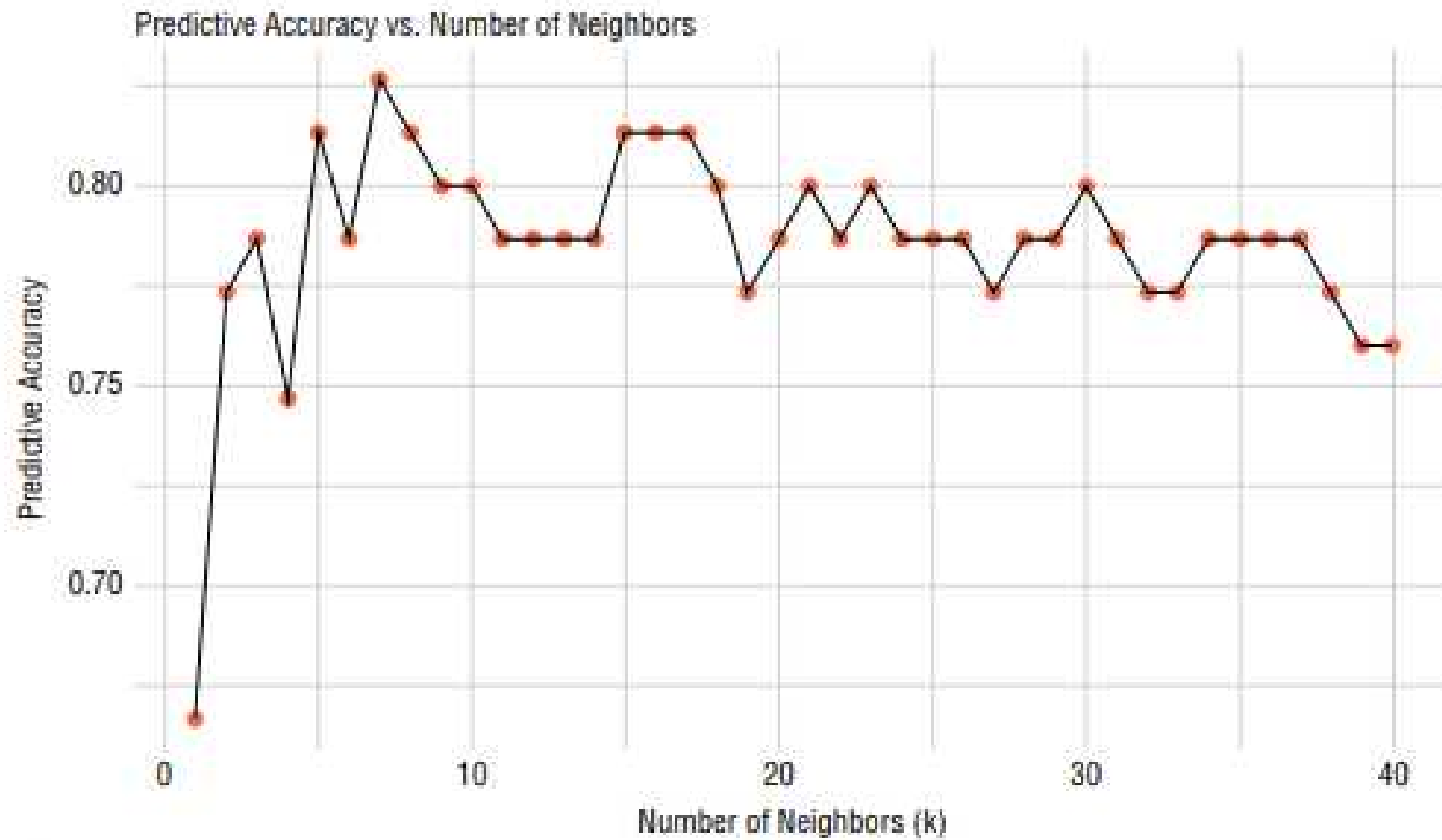


Figure 6.4 The predictive accuracy of our model for values of k -nearest neighbors between 1 and 40

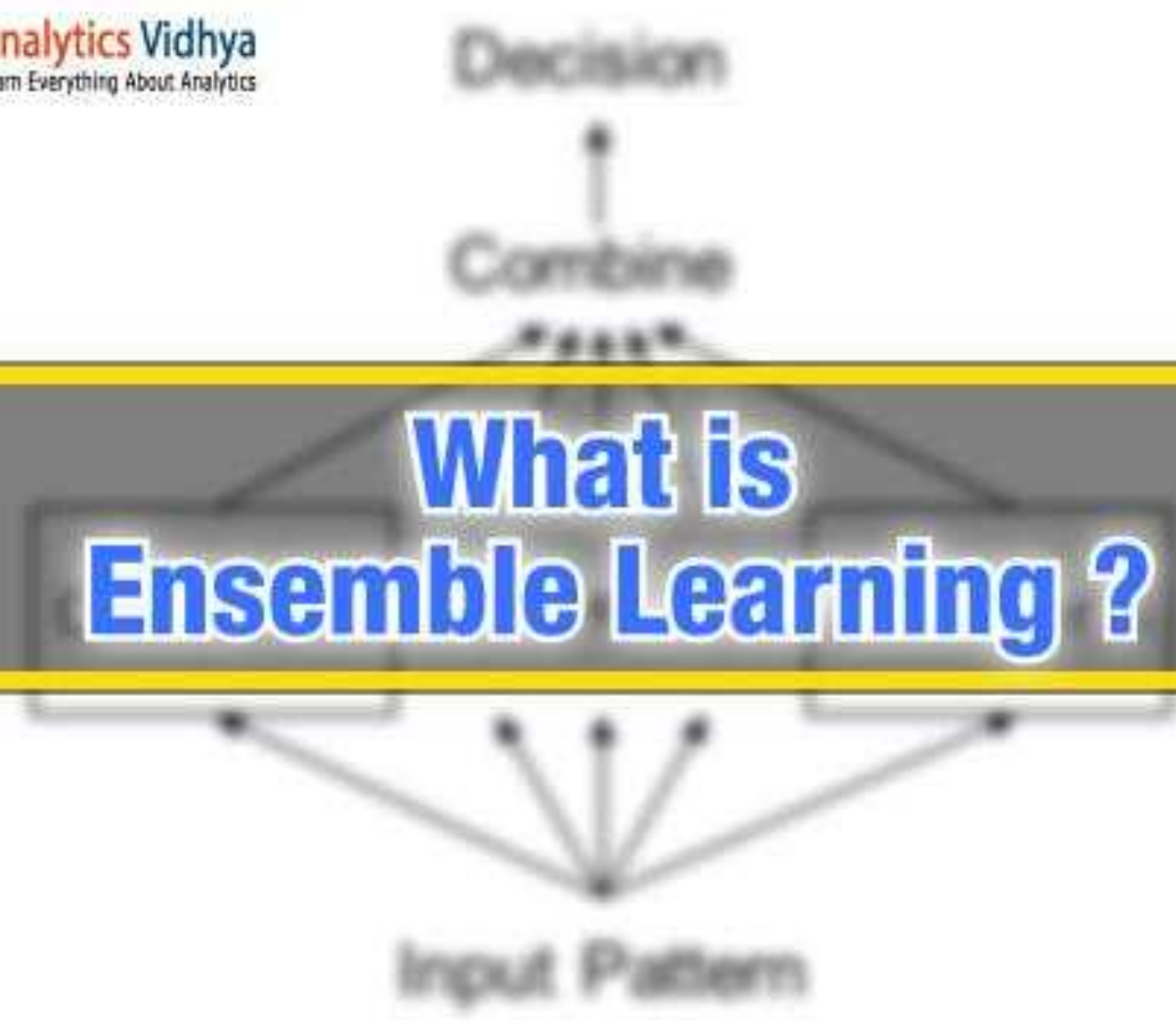
Advantages and Disadvantages of kNN

◆ Advantages of kNN:

- ◆ It does not use the distribution type assumptions.
- ◆ kNN can be used for a small number of samples in the training set.

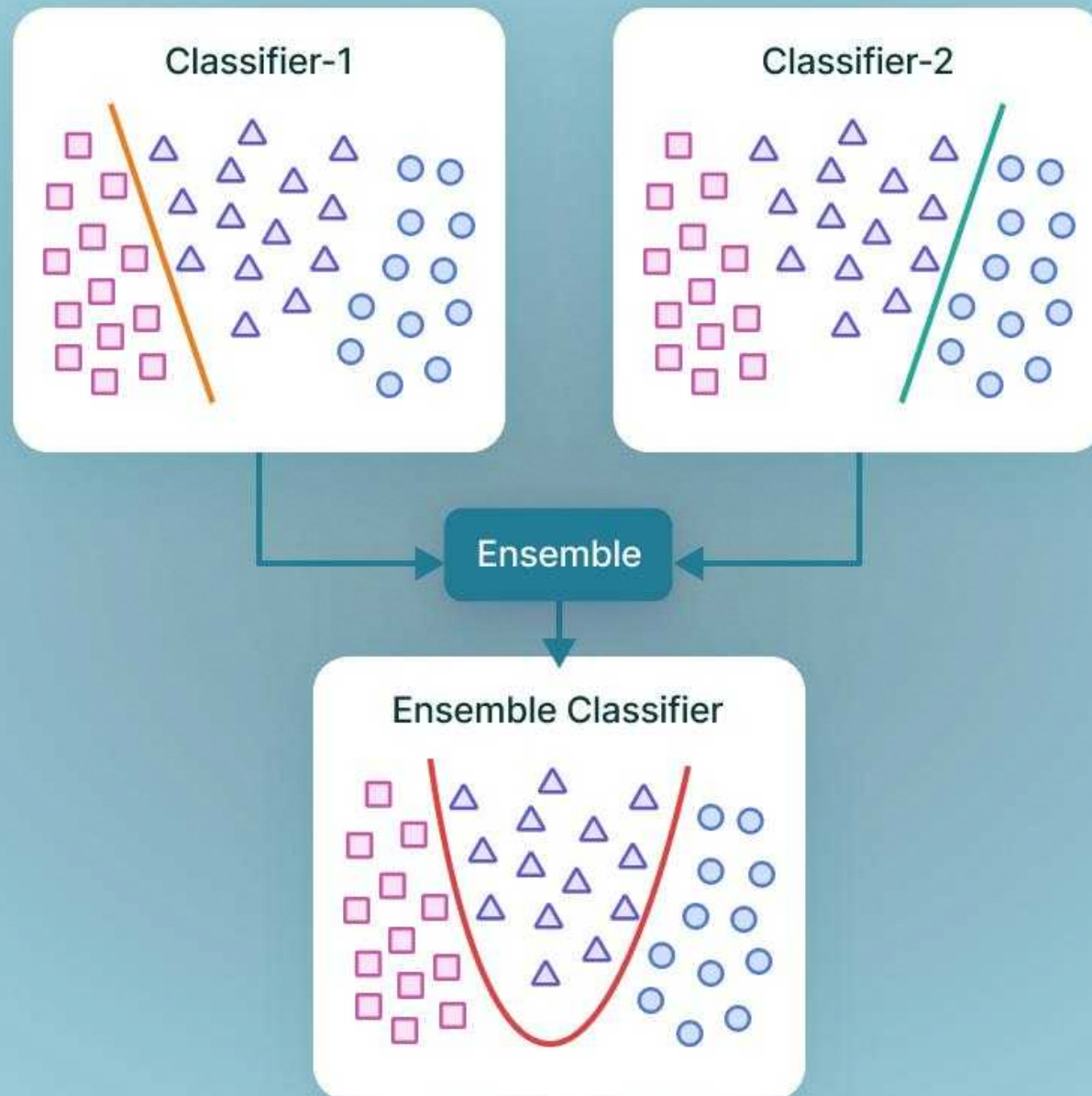
◆ “Drawbacks of kNN:”

- ◆ kNN does not work for a large number of features as the curse of dimensionality eliminates the differences between samples.
- ◆ The result depends on the choice of metric and the number of neighbors.



The diagram illustrates the ensemble learning process. At the bottom, an 'Input Pattern' is fed into multiple parallel models (represented by boxes). The outputs of these models are then 'Combine' into a single result, which leads to a final 'Decision' at the top.

What is Ensemble Learning ?



Ensemble Methods

- ◆ Construct a set of base classifiers learned from the training data
- ◆ Predict class label of test records by combining the predictions made by multiple classifiers (e.g., by taking a majority vote)

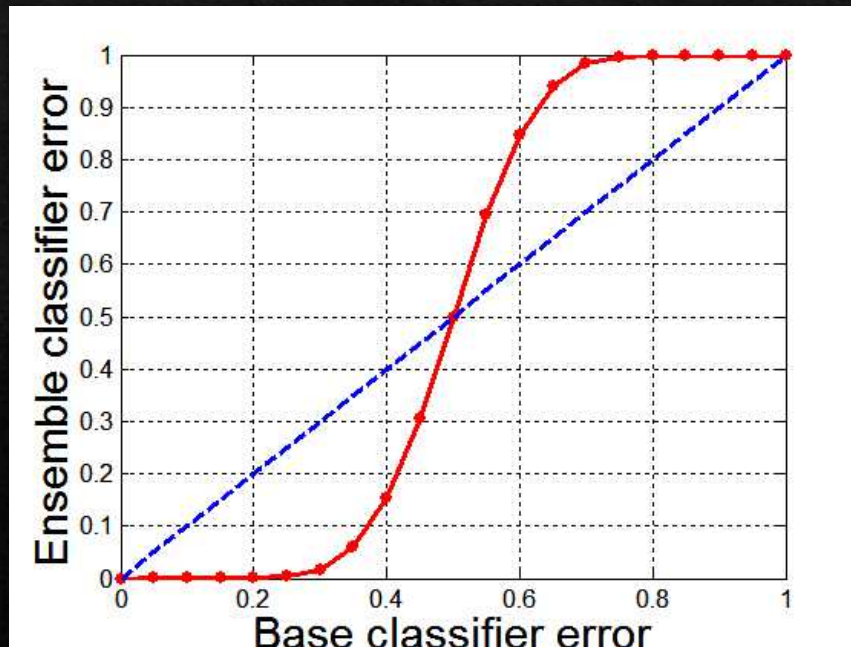
Example: Why Do Ensemble Methods Work?

- Suppose there are 25 base classifiers
 - Each classifier has error rate, $\epsilon = 0.35$
 - Majority vote of classifiers used for classification
 - If all classifiers are identical:
 - ◆ Error rate of ensemble = ϵ (0.35)
 - If all classifiers are independent (errors are uncorrelated):
 - ◆ Error rate of ensemble = probability of having more than half of base classifiers being wrong

$$e_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1 - \epsilon)^{25-i} = 0.06$$

Necessary Conditions for Ensemble Methods

- ◆ Ensemble Methods work better than a single base classifier if:
 1. All base classifiers are independent of each other
 2. All base classifiers perform better than random guessing (error rate < 0.5 for binary classification)



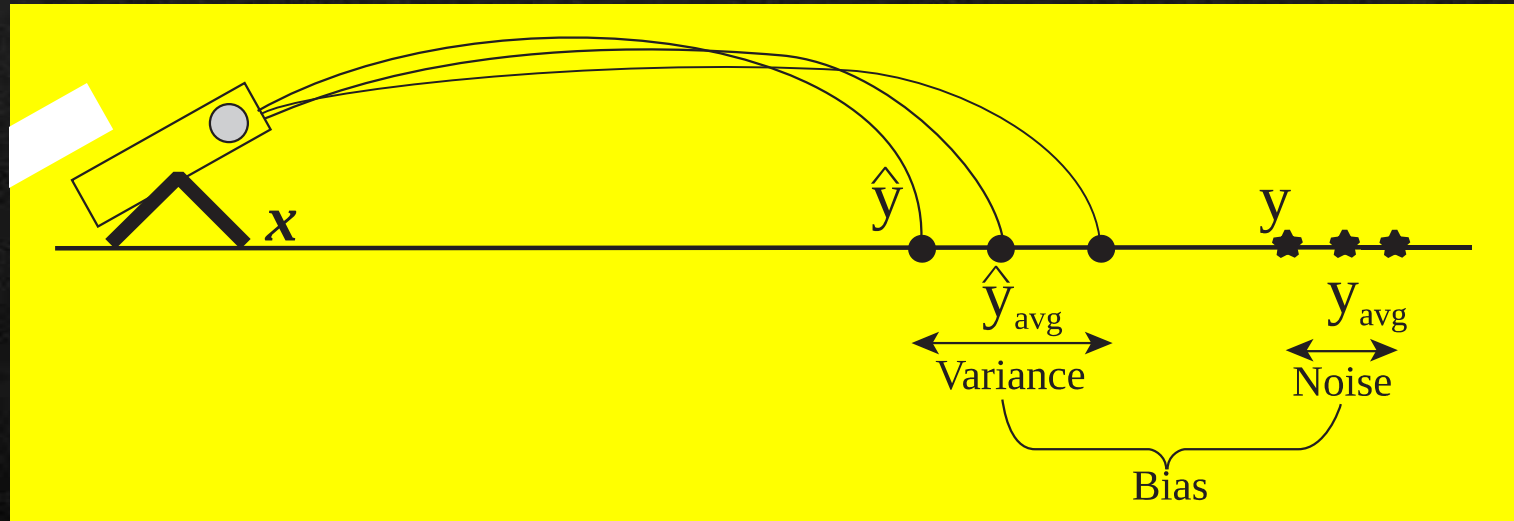
Classification error for an ensemble of 25 base classifiers, assuming their errors are uncorrelated.

Rationale for Ensemble Learning

- ◆ Ensemble Methods work best with **unstable base classifiers**
 - Classifiers that are sensitive to minor perturbations in the training set, due to *high model complexity*
 - Examples: Unpruned decision trees, ANNs, ...

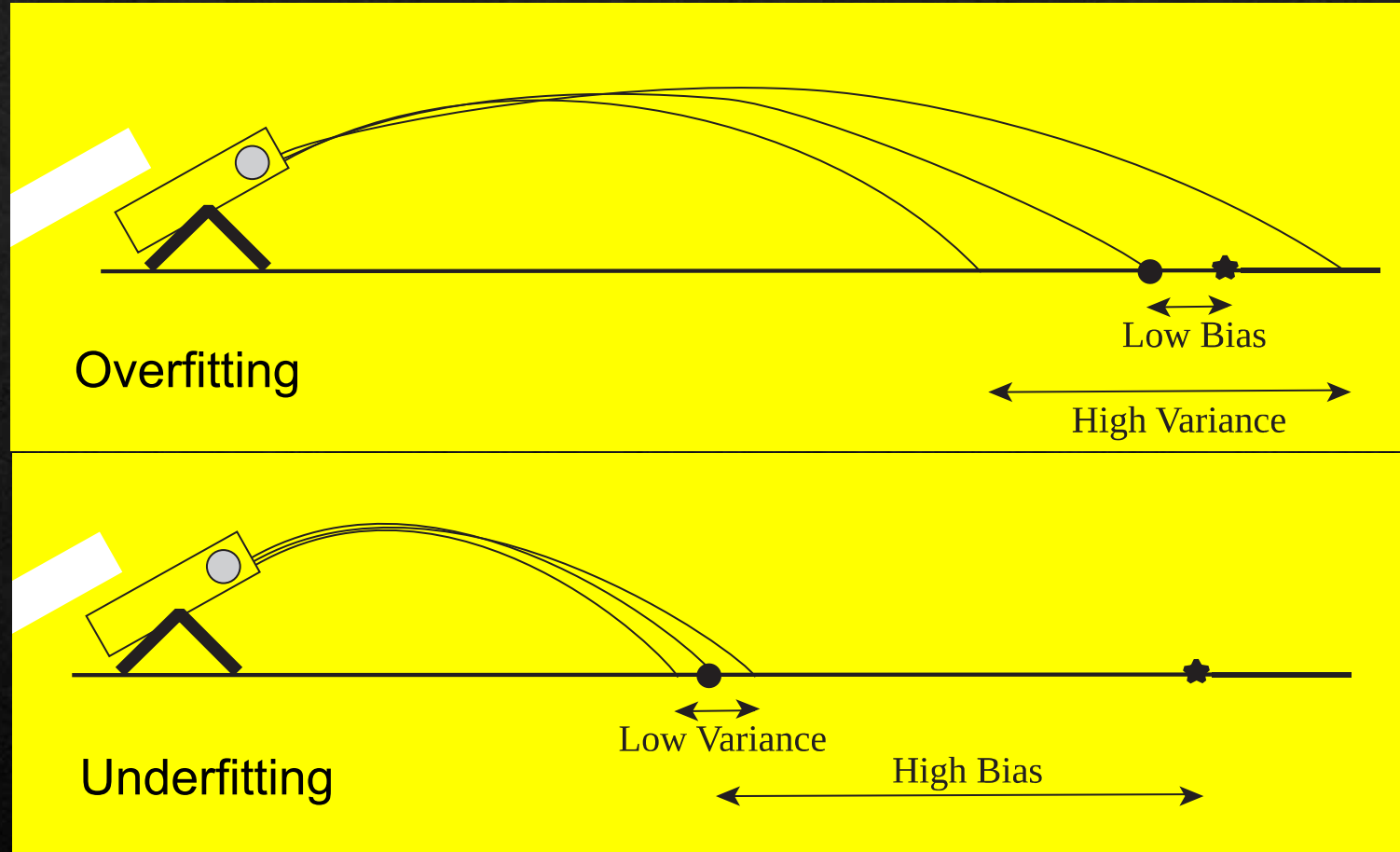
Bias-Variance Decomposition

- ◆ Analogous problem of reaching a target y by firing projectiles from x (regression problem)



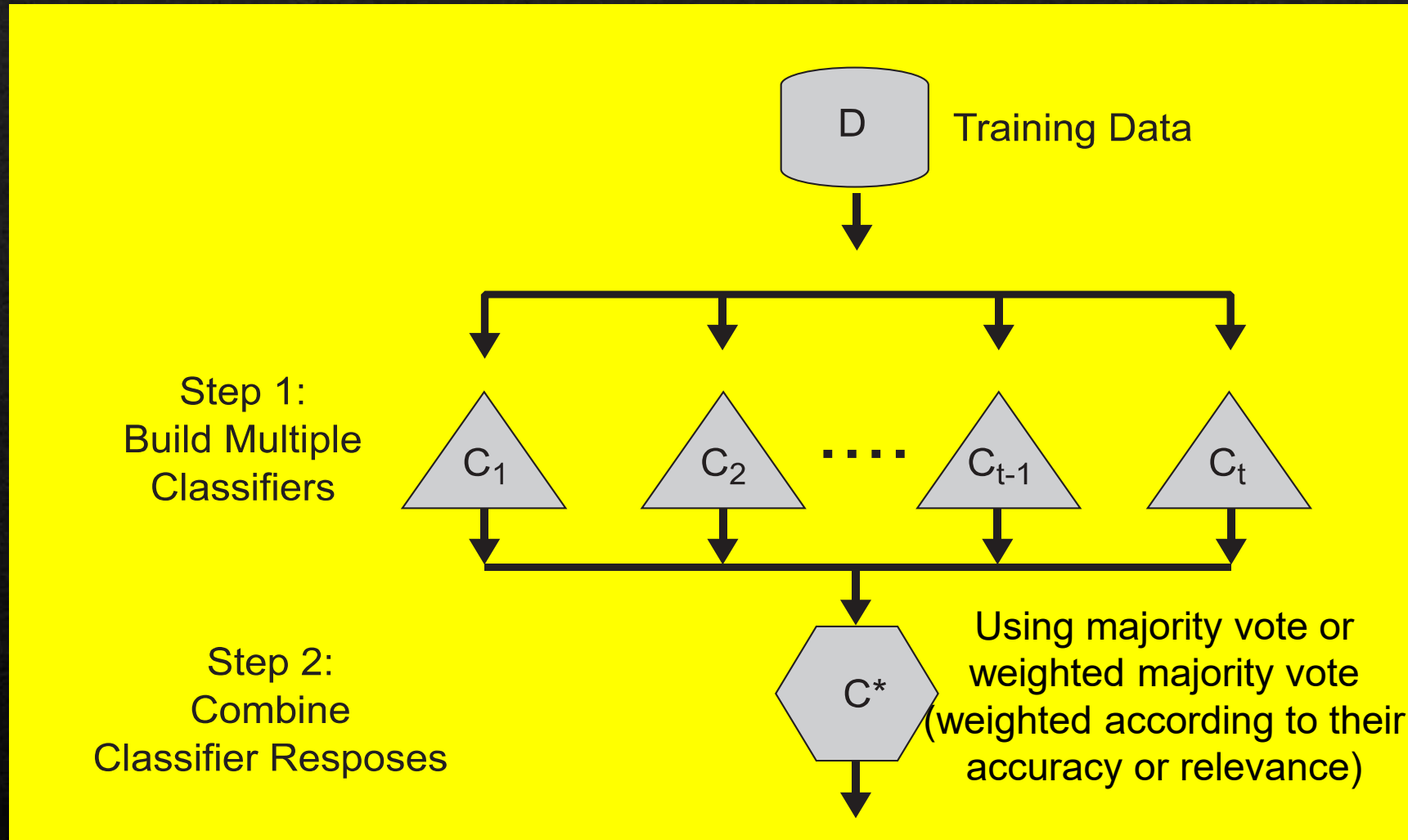
- ◆ For classification, the generalization error of model m can be given by:
$$gen.error(m) = c_1 + bias(m) + c_2 \times variance(m)$$

Bias-Variance Trade-off and Overfitting



- ◆ Ensemble methods try to reduce the variance of complex models (with low bias) by *aggregating* responses of multiple base classifiers

General Approach of Ensemble Learning



Constructing Ensemble Classifiers

- ◆ By manipulating the training set
 - Example: bagging, boosting, random forests
- ◆ By manipulating input features
 - Example: random forests
- ◆ By manipulating class labels
 - Example: error-correcting output coding
- ◆ By manipulating learning algorithm
 - Example: injecting randomness in the initial weights of ANN

	Bagging	Boosting	Stacking
Purpose	Reduce Variance	Reduce Bias	Improve Accuracy
Base Learner Types	Homogeneous	Homogeneous	Heterogeneous
Base Learner Training	Parallel	Sequential	Meta Model
Aggregation	Max Voting, Averaging	Weighted Averaging	Weighted Averaging



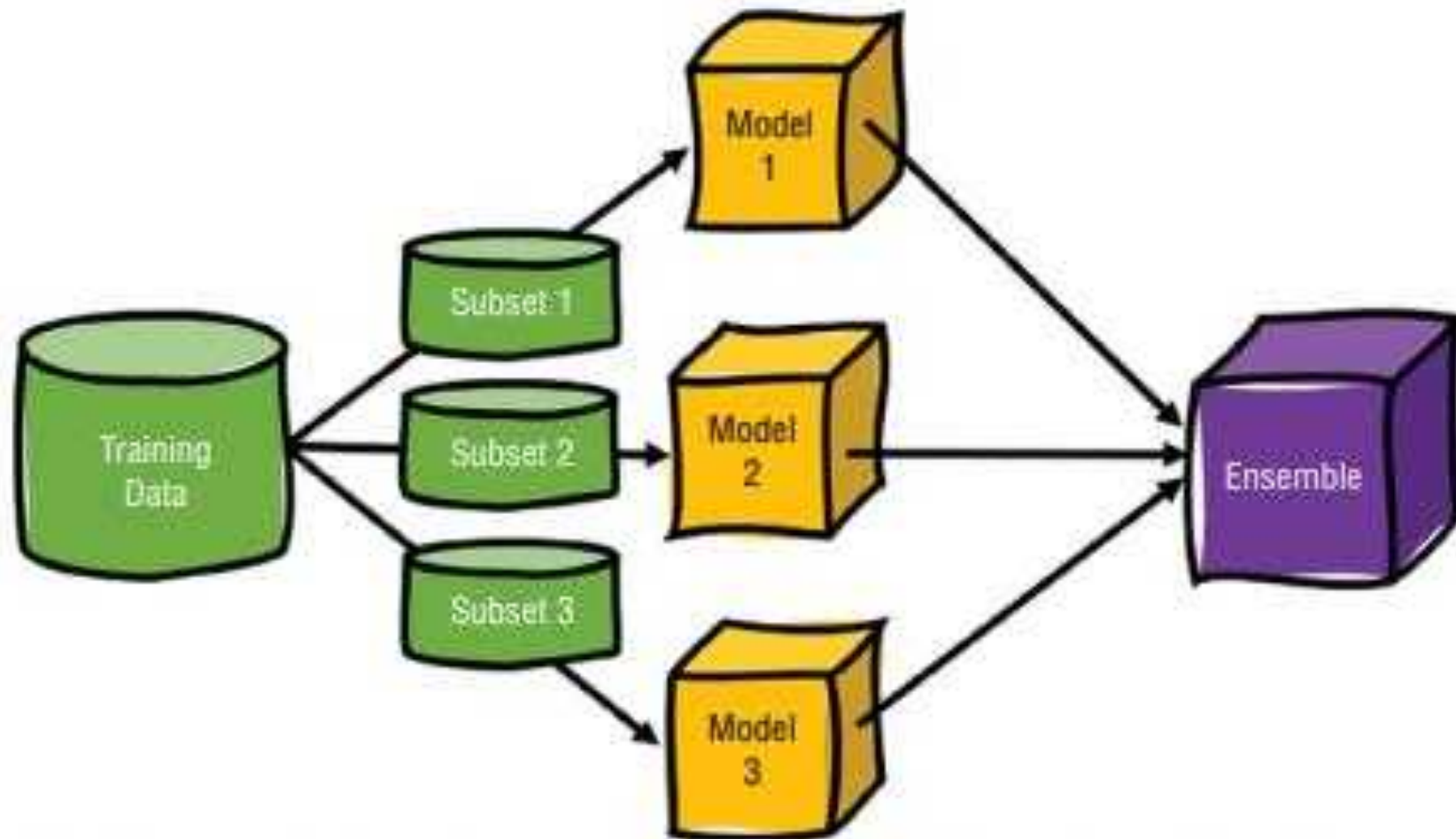


Figure 10.3 The bagging ensemble features independently trained homogenous models in parallel.

Bagging (Bootstrap AGGREGatING)

- ◆ Bootstrap sampling: sampling with replacement

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

- ◆ Build a classifier on each bootstrap sample
- ◆ Probability of a training instance being selected in a bootstrap sample is:
 - $1 - (1 - 1/n)^n$ (n: number of training instances)
 - ~ 0.632 when n is large

Bagging Algorithm

Algorithm 4.5 Bagging algorithm.

- 1: Let k be the number of bootstrap samples.
 - 2: **for** $i = 1$ to k **do**
 - 3: Create a bootstrap sample of size N , D_i .
 - 4: Train a base classifier C_i on the bootstrap sample D_i .
 - 5: **end for**
 - 6: $C^*(x) = \operatorname{argmax}_y \sum_i \delta(C_i(x) = y)$.
 $\{\delta(\cdot) = 1$ if its argument is true and 0 otherwise. $\}$
-

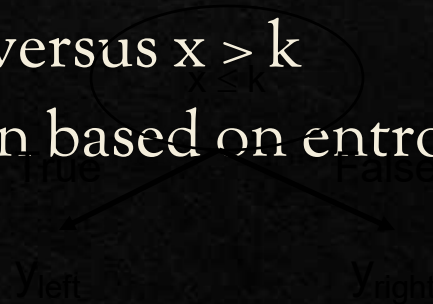
Bagging Example

- Consider a 1-dimensional data set:

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- Classifier is a decision stump (decision tree of size 1)
 - Decision rule: $x \leq k$ versus $x > k$
 - Split point k is chosen based on entropy



Bagging Example

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$
 $x > 0.35 \rightarrow y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

Bagging Example

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$
 $x > 0.35 \rightarrow y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

$x \leq 0.7 \rightarrow y = 1$
 $x > 0.7 \rightarrow y = -1$

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$
 $x > 0.35 \rightarrow y = -1$

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.3 \rightarrow y = 1$
 $x > 0.3 \rightarrow y = -1$

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

$x \leq 0.35 \rightarrow y = 1$
 $x > 0.35 \rightarrow y = -1$

Bagging Example

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$
 $x > 0.75 \rightarrow y = 1$

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

$x \leq 0.75 \rightarrow y = -1$
 $x > 0.75 \rightarrow y = 1$

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$
 $x > 0.75 \rightarrow y = 1$

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$
 $x > 0.75 \rightarrow y = 1$

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

$x \leq 0.05 \rightarrow y = 1$
 $x > 0.05 \rightarrow y = 1$

Bagging Example

- ◆ Summary of Trained Decision Stumps:

Round	Split Point	Left Class	Right Class
1	0.35	1	-1
2	0.7	1	1
3	0.35	1	-1
4	0.3	1	-1
5	0.35	1	-1
6	0.75	-1	1
7	0.75	-1	1
8	0.75	-1	1
9	0.75	-1	1
10	0.05	1	1

Bagging Example

- Use majority vote (the sign of the sum of predictions) to determine the class of ensemble classifier

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Sign	1	1	1	-1	-1	-1	-1	1	1	1

Predicted
Class

- Bagging can also increase the complexity (representation capacity) of simple classifiers such as decision stumps

	Bagging	Boosting	Stacking
Purpose	Reduce Variance	Reduce Bias	Improve Accuracy
Base Learner Types	Homogeneous	Homogeneous	Heterogeneous
Base Learner Training	Parallel	Sequential	Meta Model
Aggregation	Max Voting, Averaging	Weighted Averaging	Weighted Averaging



Boosting

- ◆ An iterative procedure to adaptively change the distribution of training data by focusing more on previously misclassified records
 - Initially, all N records are assigned equal weights (for being selected for training)
 - Unlike bagging, weights may change at the end of each boosting round

Boosting

- ◆ Records that are wrongly classified will have their weights increased in the next round
- ◆ Records that are classified correctly will have their weights decreased in the next round

Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

- Example 4 is hard to classify
- Its weight is increased, therefore it is more likely to be chosen again in subsequent rounds

AdaBoost

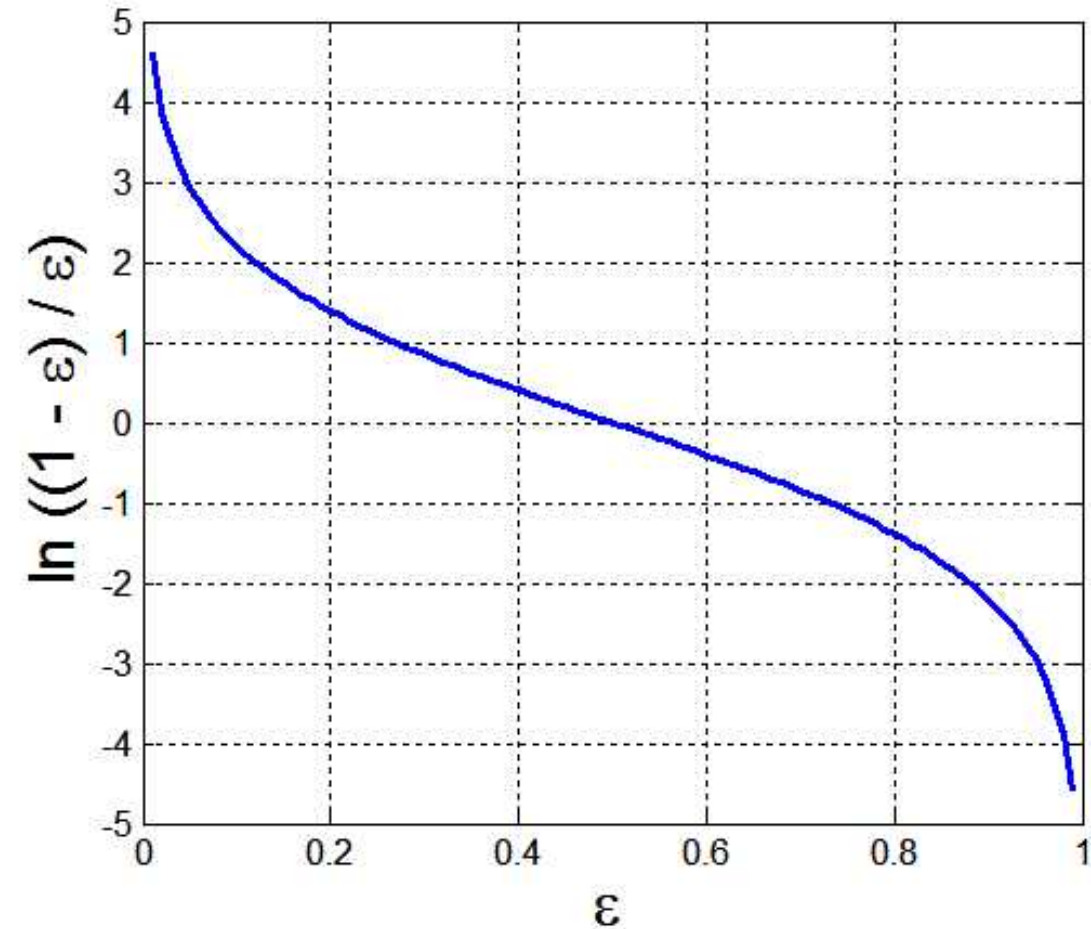
◇ Base classifiers: $C_1, C_2, \dots, C_T$

◇ Error rate of a base classifier:

$$\epsilon_i = \frac{1}{N} \sum_{j=1}^N w_j^{(i)} \delta(C_i(x_j) \neq y_j)$$

◇ Importance of a classifier:

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right)$$



AdaBoost Algorithm

- Weight update:

$$w_j^{(i+1)} = \frac{w_j^{(i)}}{Z_i} \times \begin{cases} e^{-\alpha_i} & \text{if } C_i(x_j) = y_j \\ e^{\alpha_i} & \text{if } C_i(x_j) \neq y_j \end{cases}$$

Where Z_i is the normalization factor

- If any intermediate rounds produce an error rate higher than 50%, the weights are reverted back to $1/n$ and the resampling procedure is repeated

- Classification:

$$C^*(x) = \arg \max_y \sum_{i=1}^T \alpha_i \delta(C_i(x) = y)$$

AdaBoost Algorithm

Algorithm 4.6 AdaBoost algorithm.

- 1: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \dots, N\}$. {Initialize the weights for all N examples.}
 - 2: Let k be the number of boosting rounds.
 - 3: **for** $i = 1$ to k **do**
 - 4: Create training set D_i by sampling (with replacement) from D according to $\mathbf{w}$.
 - 5: Train a base classifier C_i on D_i .
 - 6: Apply C_i to all examples in the original training set, D .
 - 7: $\epsilon_i = \frac{1}{N} \left[\sum_j w_j \delta(C_i(x_j) \neq y_j) \right]$ {Calculate the weighted error.}
 - 8: **if** $\epsilon_i > 0.5$ **then**
 - 9: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \dots, N\}$. {Reset the weights for all N examples.}
 - 10: Go back to Step 4.
 - 11: **end if**
 - 12: $\alpha_i = \frac{1}{2} \ln \frac{1-\epsilon_i}{\epsilon_i}$.
 - 13: Update the weight of each example according to Equation 4.103.
 - 14: **end for**
 - 15: $C^*(\mathbf{x}) = \operatorname{argmax}_y \sum_{j=1}^T \alpha_j \delta(C_j(\mathbf{x}) = y)$.
-

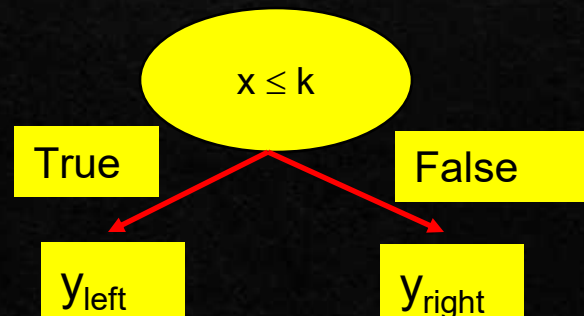
AdaBoost Example

- Consider 1-dimensional data set:

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- Classifier is a decision stump
 - Decision rule: $x \leq k$ versus $x > k$
 - Split point k is chosen based on entropy



AdaBoost Example

- Training sets for the first 3 boosting rounds:

Boosting Round 1:										
x	0.1	0.4	0.5	0.6	0.6	0.7	0.7	0.7	0.8	1
y	1	-1	-1	-1	-1	-1	-1	-1	1	1

Boosting Round 2:										
x	0.1	0.1	0.2	0.2	0.2	0.2	0.3	0.3	0.3	0.3
y	1	1	1	1	1	1	1	1	1	1

Boosting Round 3:										
x	0.2	0.2	0.4	0.4	0.4	0.4	0.5	0.6	0.6	0.7
y	1	1	-1	-1	-1	-1	-1	-1	-1	-1

- Summary:

Round	Split Point	Left Class	Right Class	alpha
1	0.75	-1	1	1.738
2	0.05	1	1	2.7784
3	0.3	1	-1	4.1195

AdaBoost Example

◆ Weights

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
2	0.311	0.311	0.311	0.01	0.01	0.01	0.01	0.01	0.01	0.01
3	0.029	0.029	0.029	0.228	0.228	0.228	0.228	0.009	0.009	0.009

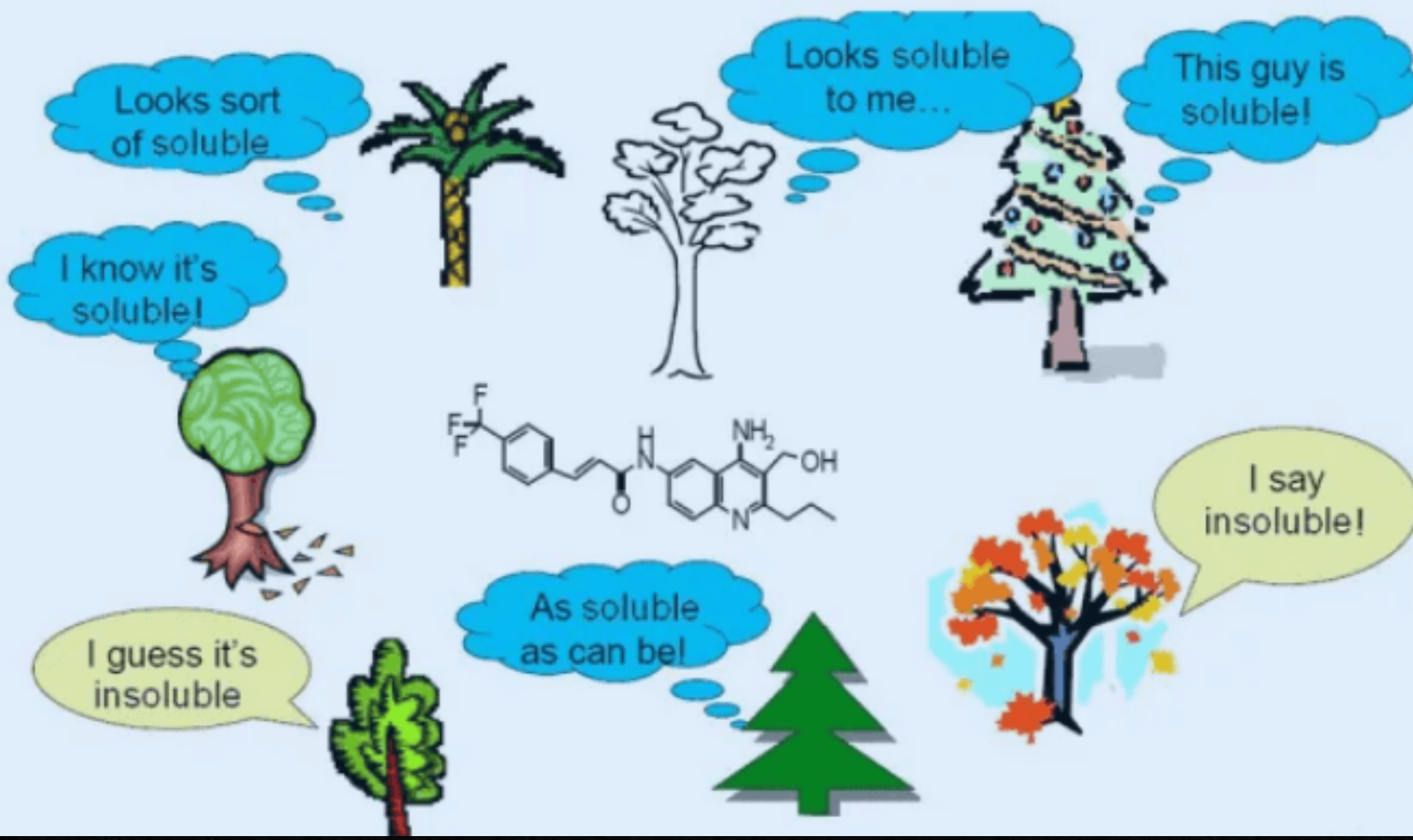
◆ Classification

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	-1	-1	-1	-1	-1	-1	-1	1	1	1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
Sum	5.16	5.16	5.16	-3.08	-3.08	-3.08	-3.08	0.397	0.397	0.397
Sign	1	1	1	-1	-1	-1	-1	1	1	1

Predicted
Class

Random Forest

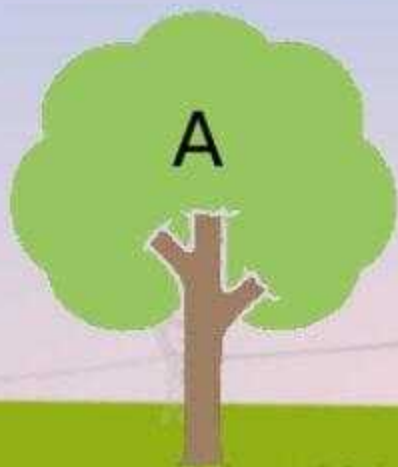
Machine Learning Method



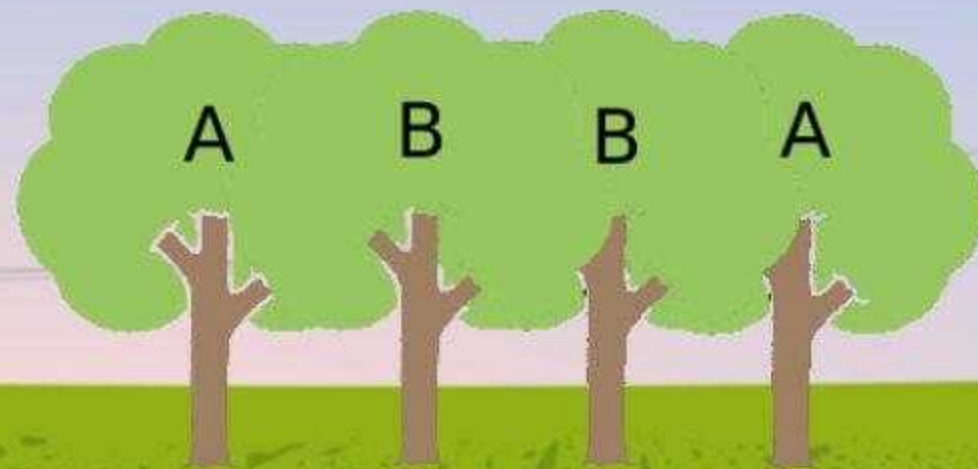
Random Forest Algorithm

- ◆ Construct an ensemble of decision trees by manipulating the training set as well as features
 - Use a bootstrap sample to train every decision tree (similar to Bagging)
 - Use the following tree induction algorithm:
 - ◆ At every internal node of the decision tree, randomly sample p attributes for selecting split criterion
 - ◆ Repeat this procedure until all leaves are pure (unpruned tree)

Decision Tree



Random Forest



Step #1

Create N subsets from original dataset, one for each weak model

Input Dataset

Subset #1

Subset #2

Subset #3

Subset #4

Step #2

Train each weak model with an independent subset, in parallel

Weak Model #1

Weak Model #2

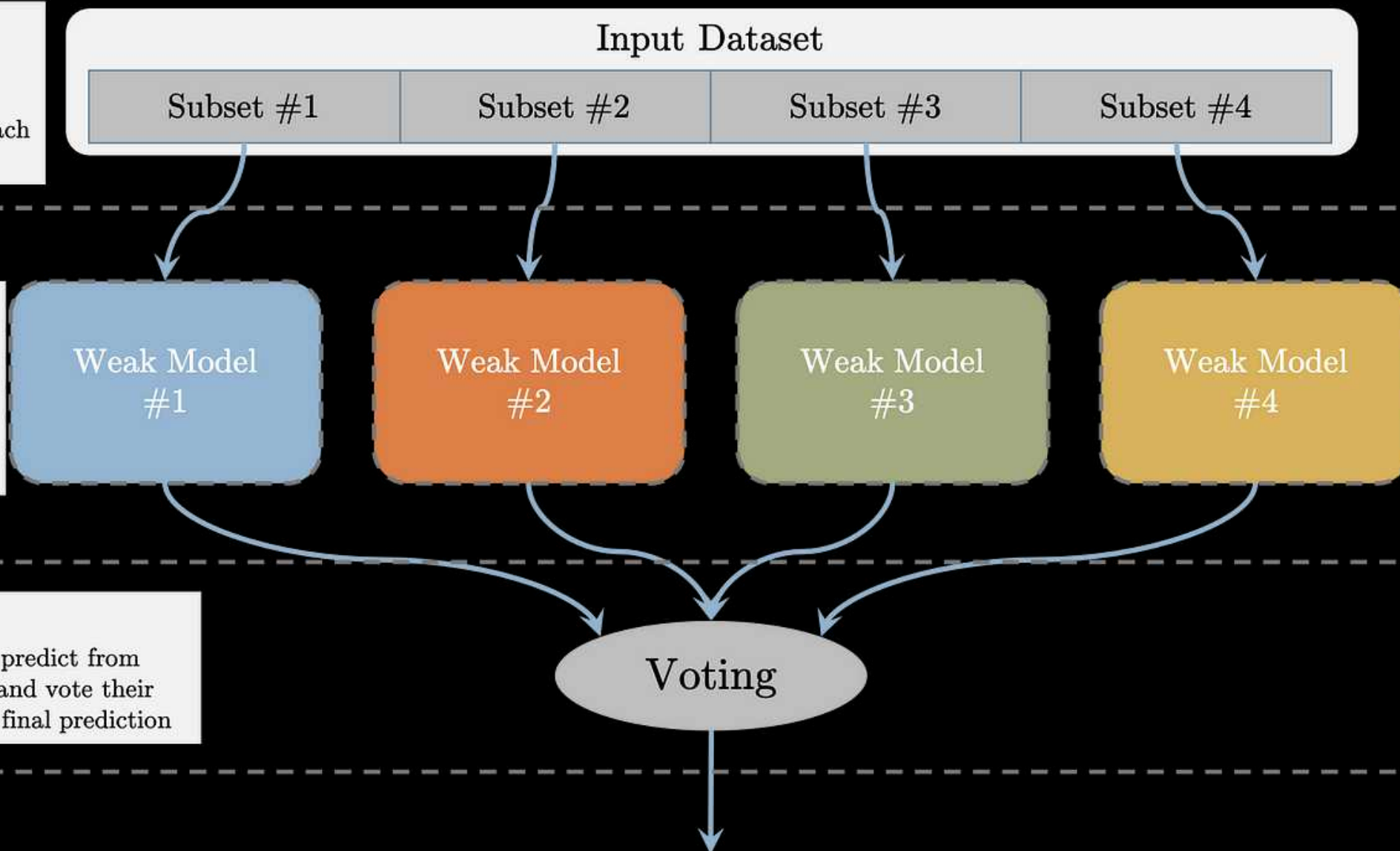
Weak Model #3

Weak Model #4

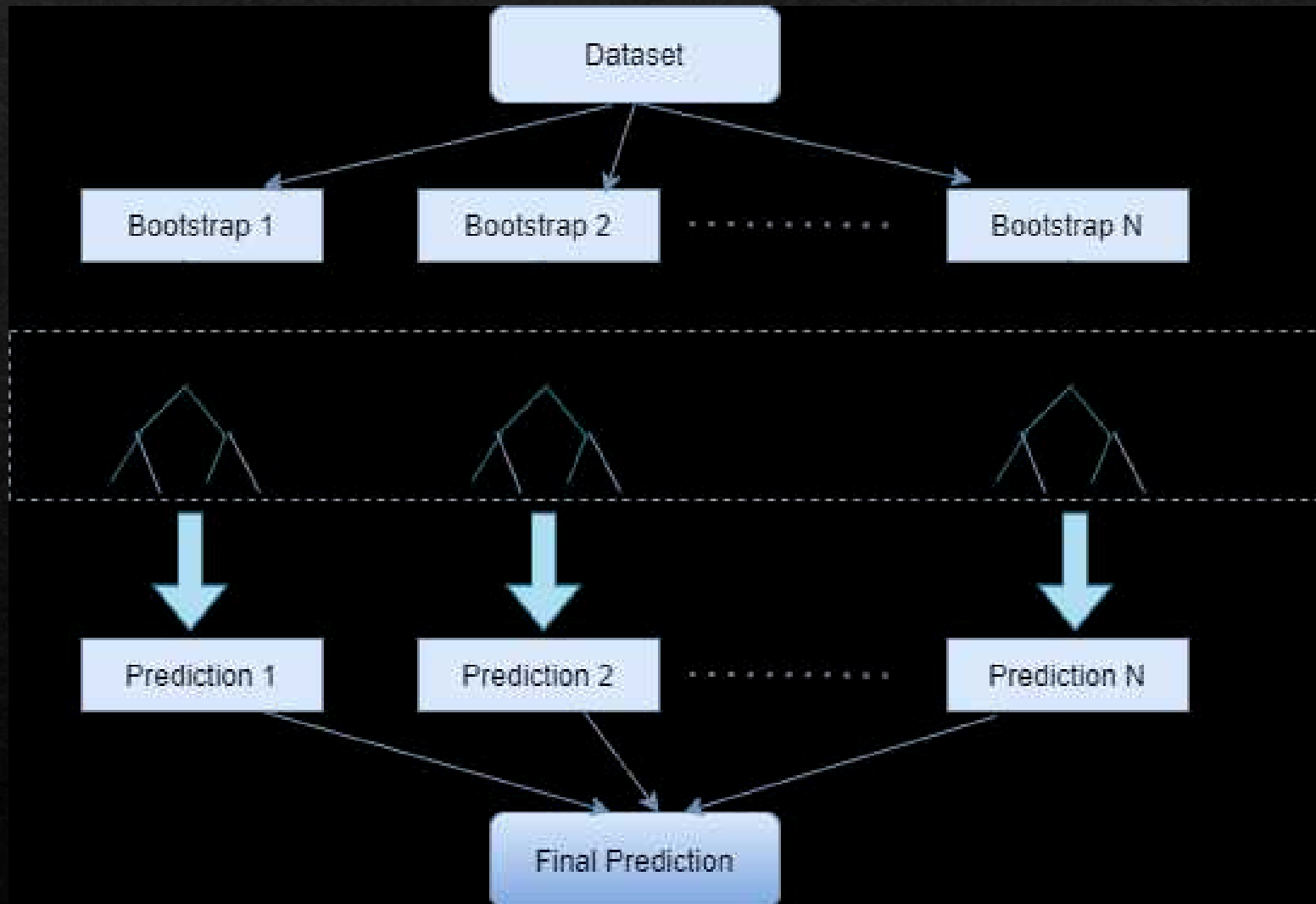
Step #3

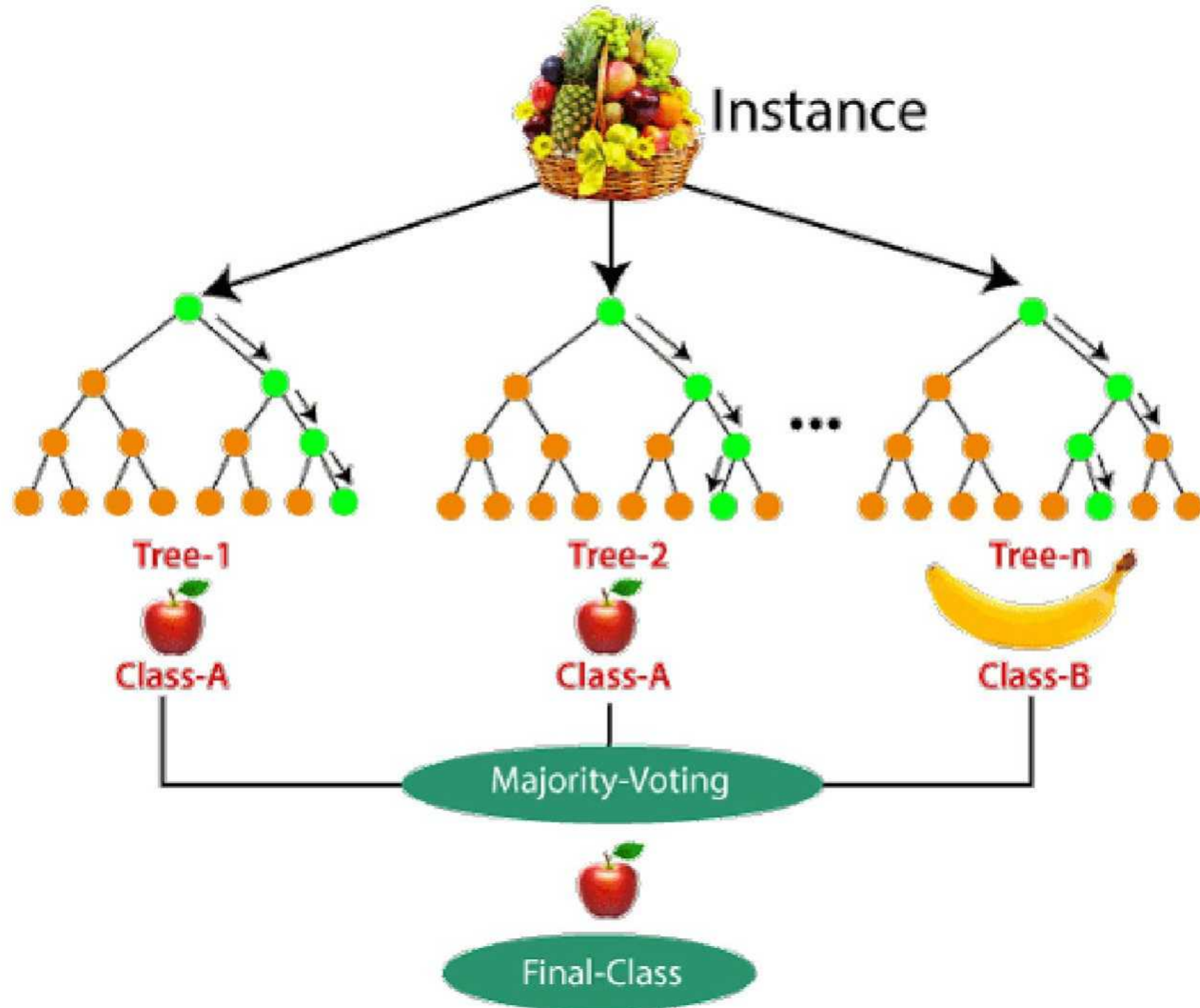
In the test phase, predict from each weak model and vote their predictions to get final prediction

Voting



Random Forest

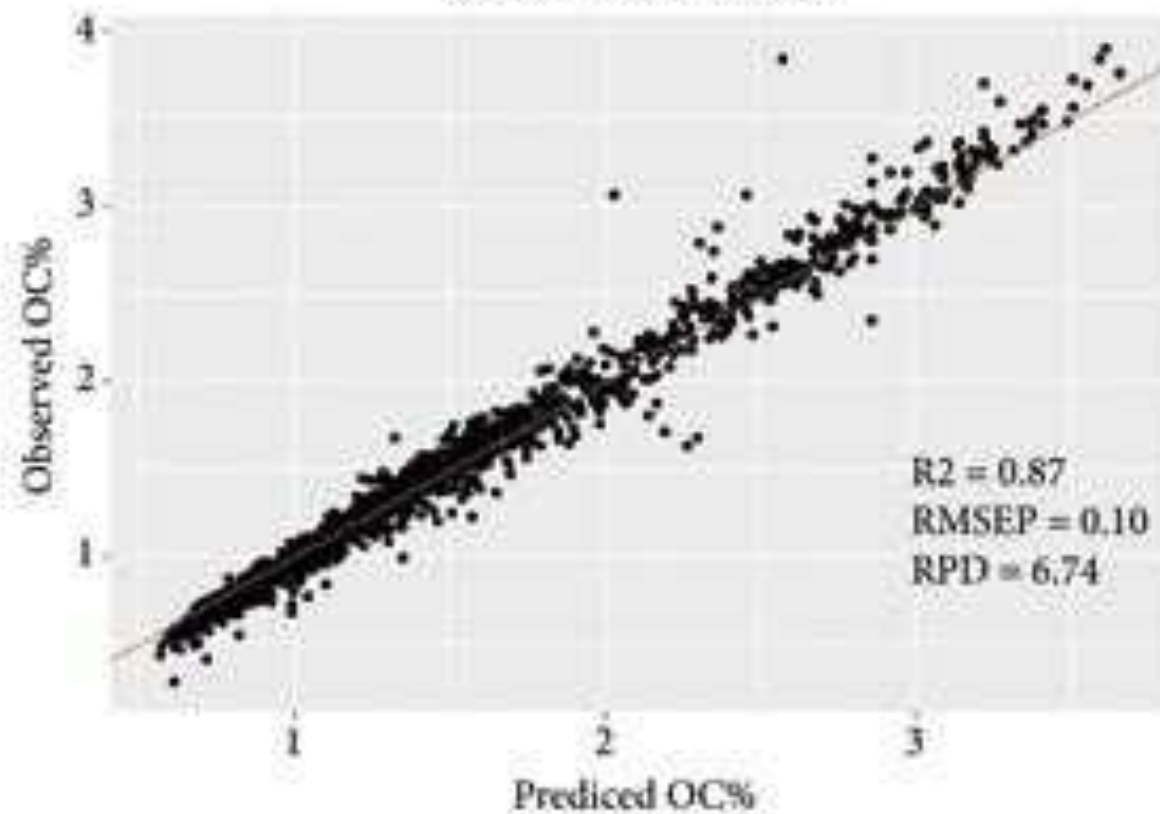




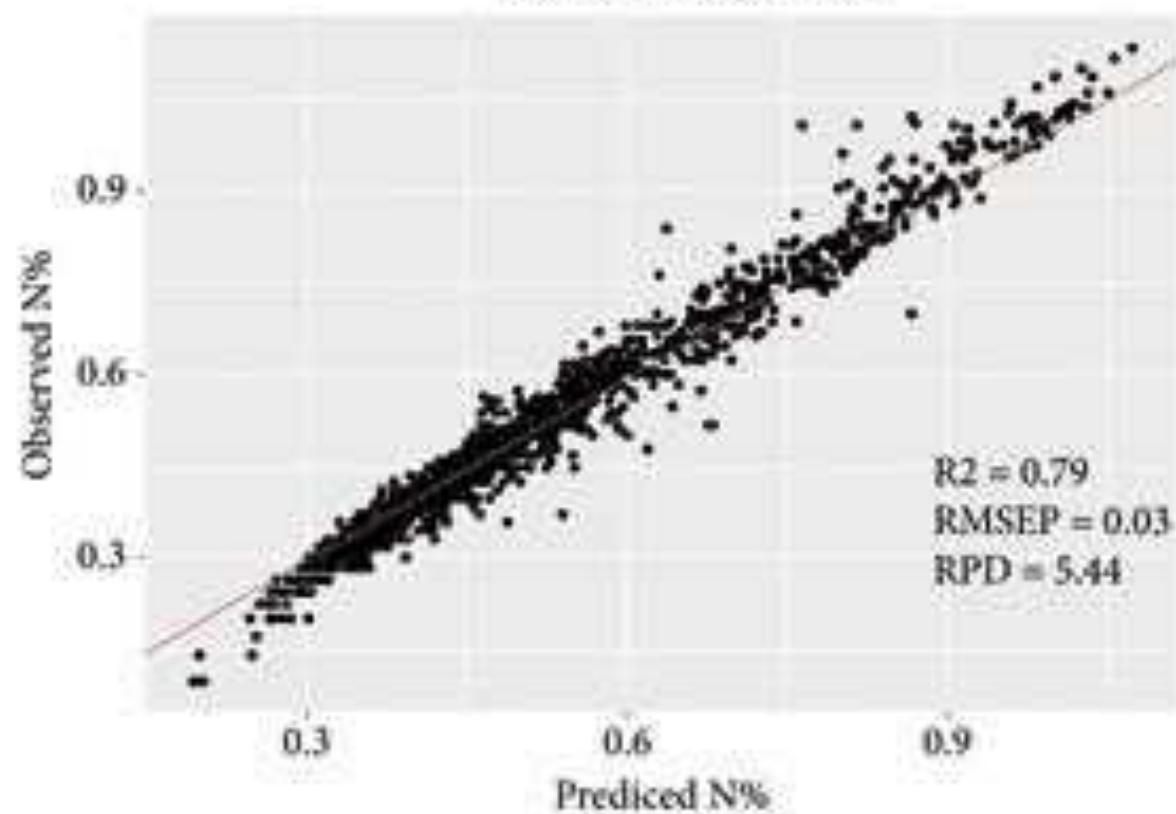
Characteristics of Random Forest

- Base classifiers are unpruned trees and hence are *unstable classifiers*
- Base classifiers are *decorrelated* (due to randomization in training set as well as features)
- Random forests reduce variance of unstable classifiers without negatively impacting the bias
- Selection of hyper-parameter p
 - Small value ensures lack of correlation
 - High value promotes strong base classifiers
 - Common default choices: $\sqrt{d}$, $\log_2(d + 1)$

Random Forest Models



Random Forest Model



A 3D surface plot with a color gradient from dark blue to red. The surface is highly irregular and jagged, with a central horizontal band of dark brown. The text 'XGBOOST' is overlaid in white, italicized font on this band. The background is a light gray grid.

XGBOOST

Gradient Boosting Process Diagram

Residuals



Pseudo residuals are initial guesses of the model to predict value of the target

Make predictions and correcting residual
e.g. Regression trees

Additive Modeling



Weak Learners

Regularizing loss function using gradient descent and adding one tree at a time to the existing model

Process is repeated till the maximum numbers of trees specified is reached or the residual gets very small



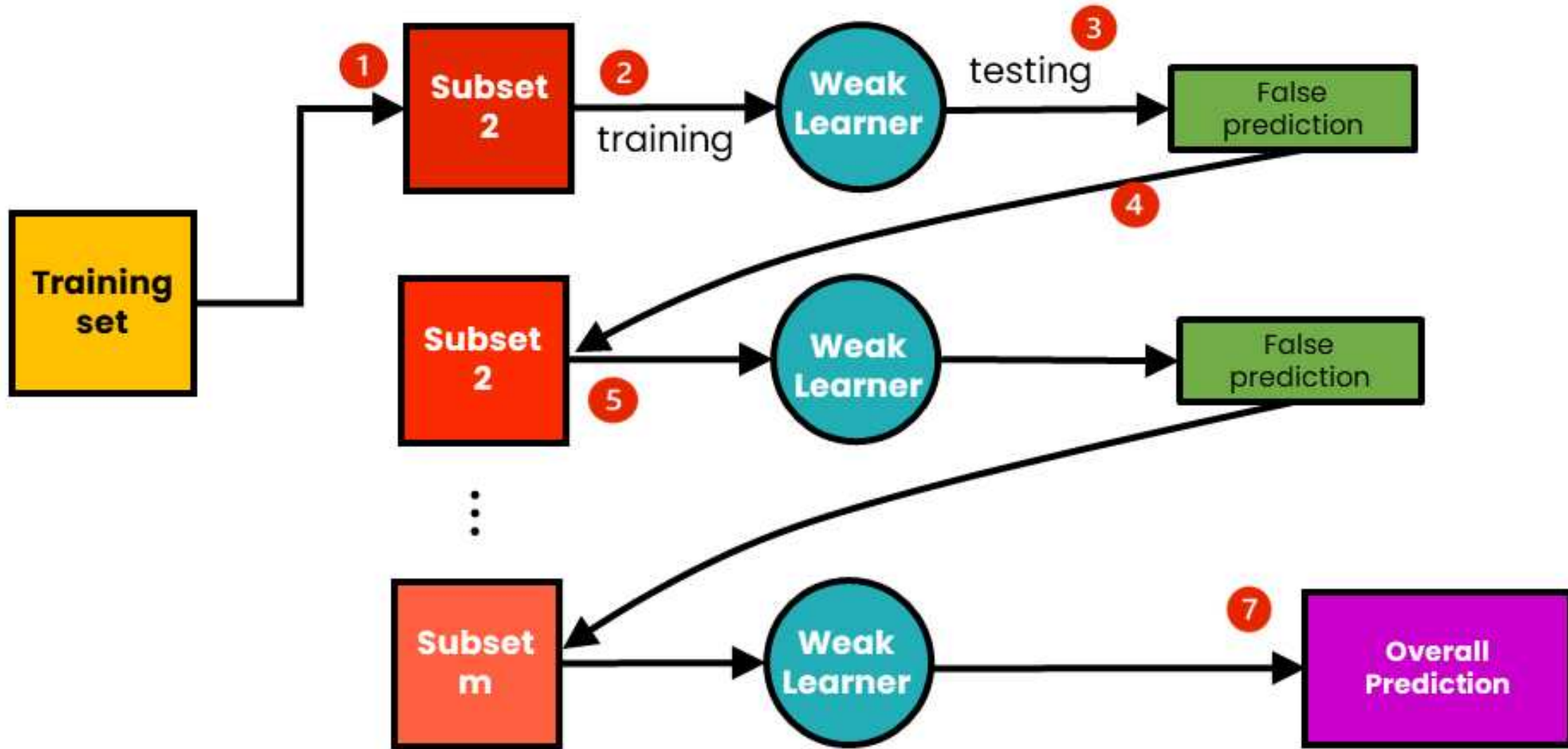
Gradient Boosting



Gradient Boosting

- ◆ Constructs a series of models
 - Models can be any predictive model that has a differentiable loss function
 - Commonly, trees are the chosen model
 - ◆ XGboost (extreme gradient boosting) is a popular package because of its impressive performance
- ◆ Boosting can be viewed as optimizing the loss function by iterative functional gradient descent.
- ◆ Implementations of various boosted algorithms are available in Python, R, Matlab, and more.

The Process of Boosting



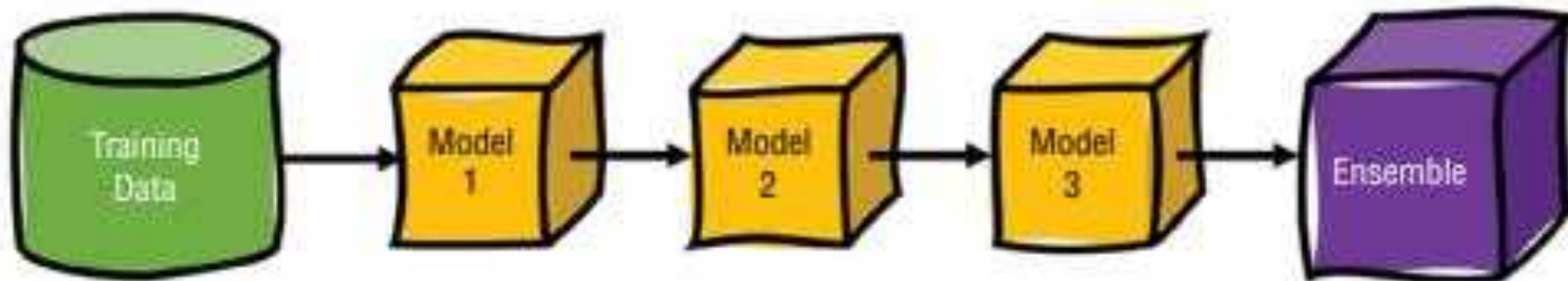
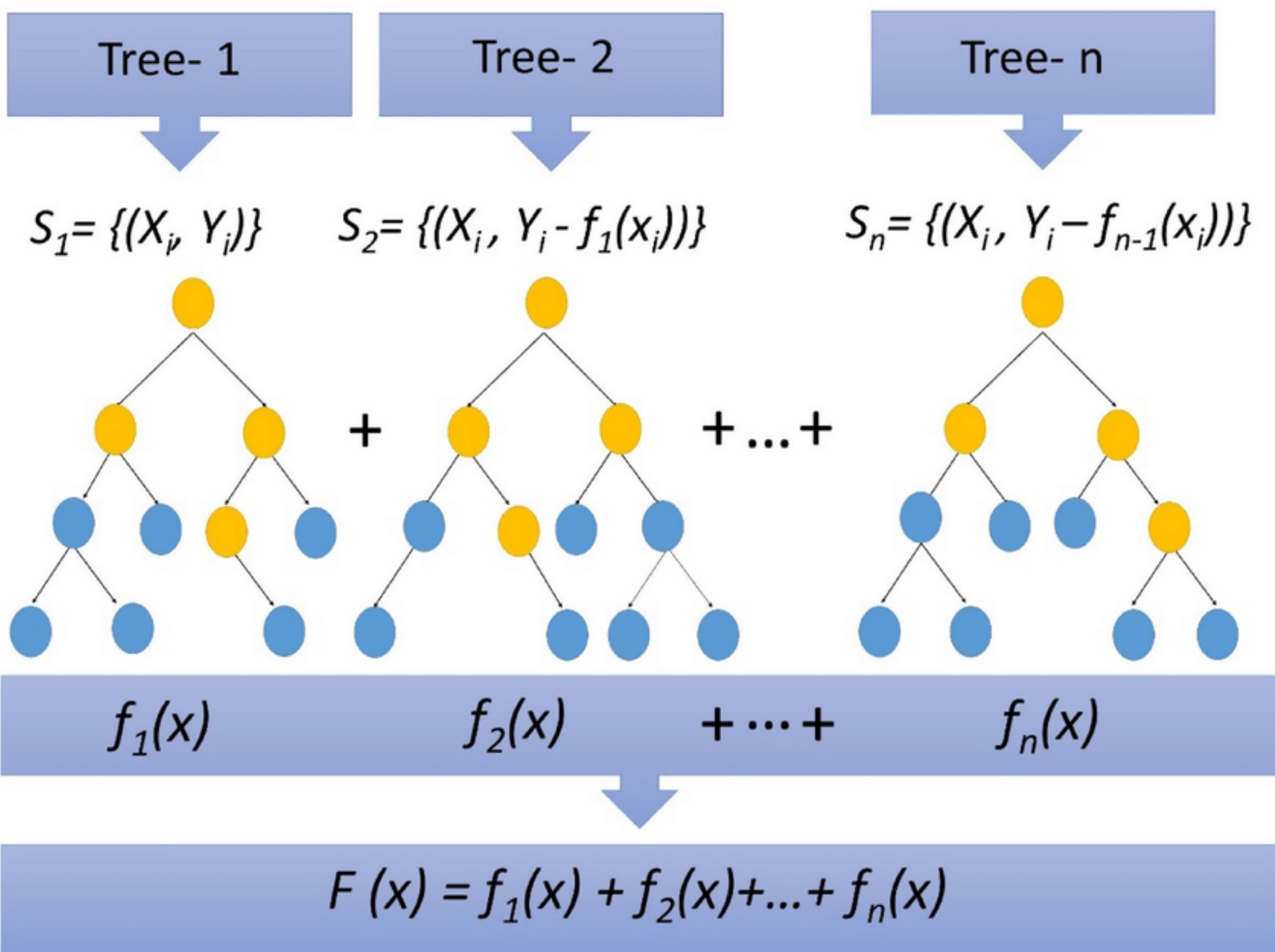
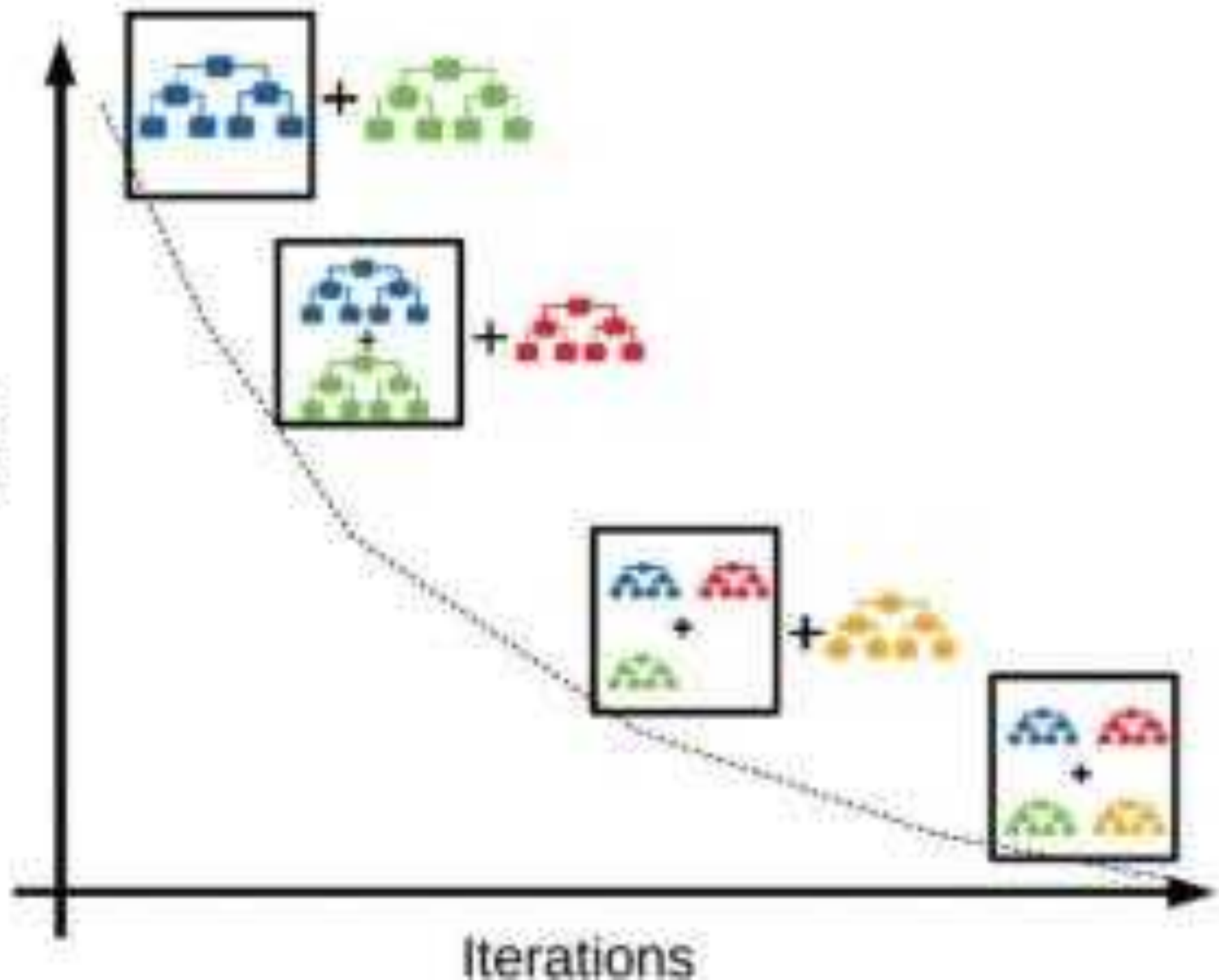


Figure 10.4 The boosting ensemble features a linear sequence of homogeneous models.



Error

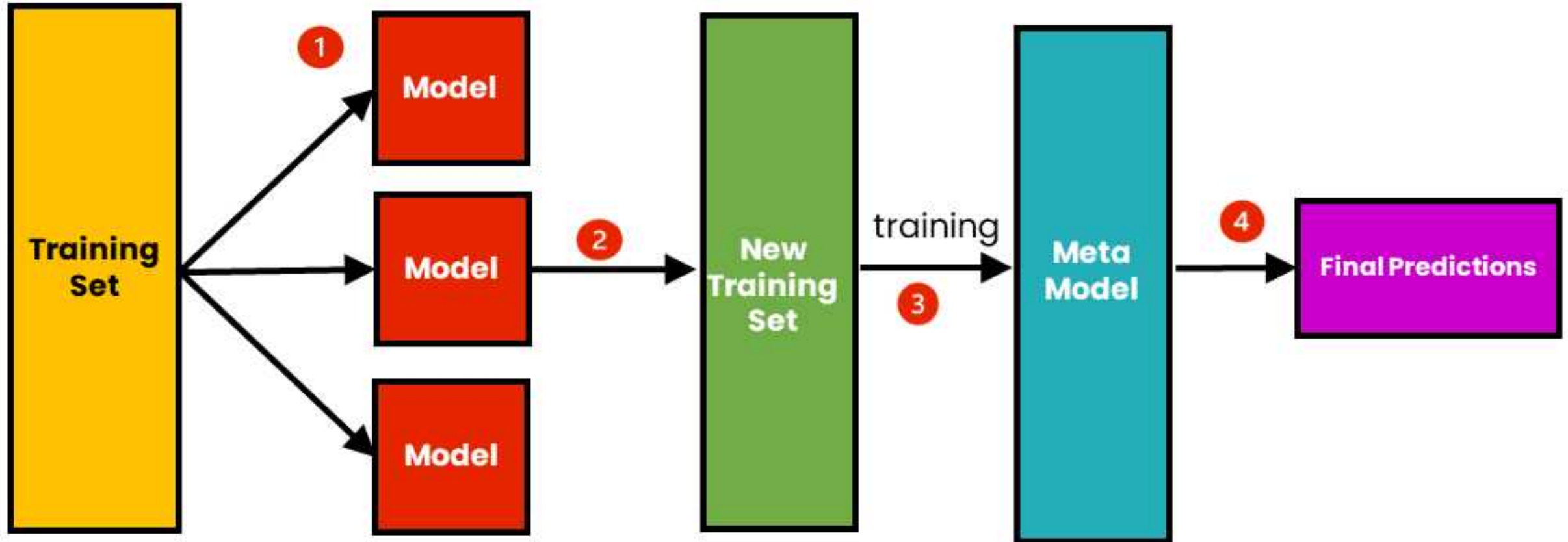


Iterations

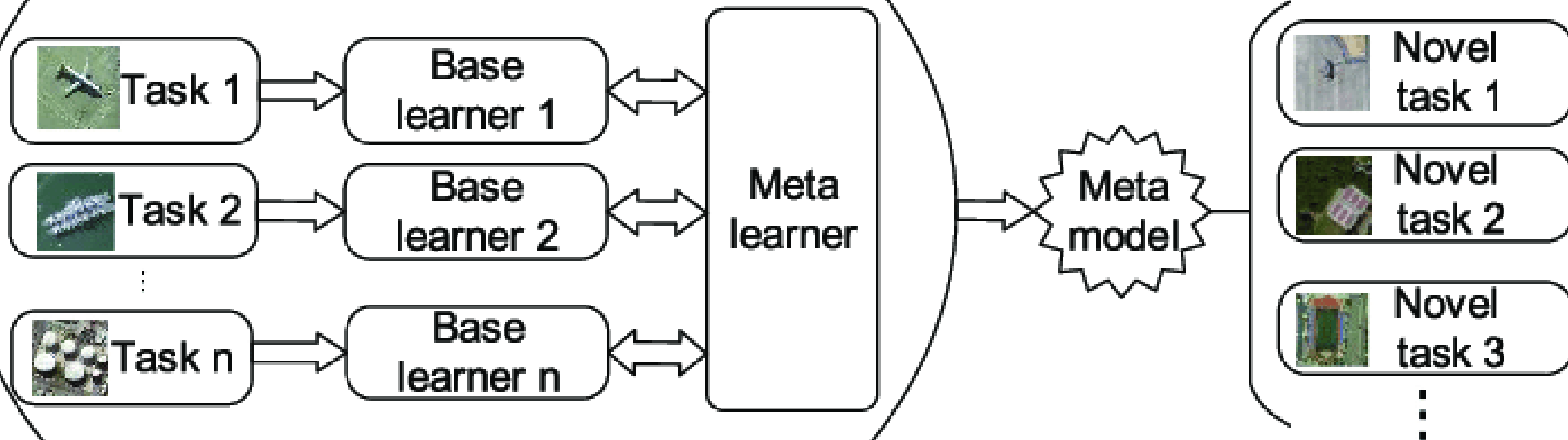
	Bagging	Boosting	Stacking
Purpose	Reduce Variance	Reduce Bias	Improve Accuracy
Base Learner Types	Homogeneous	Homogeneous	Heterogeneous
Base Learner Training	Parallel	Sequential	Meta Model
Aggregation	Max Voting, Averaging	Weighted Averaging	Weighted Averaging

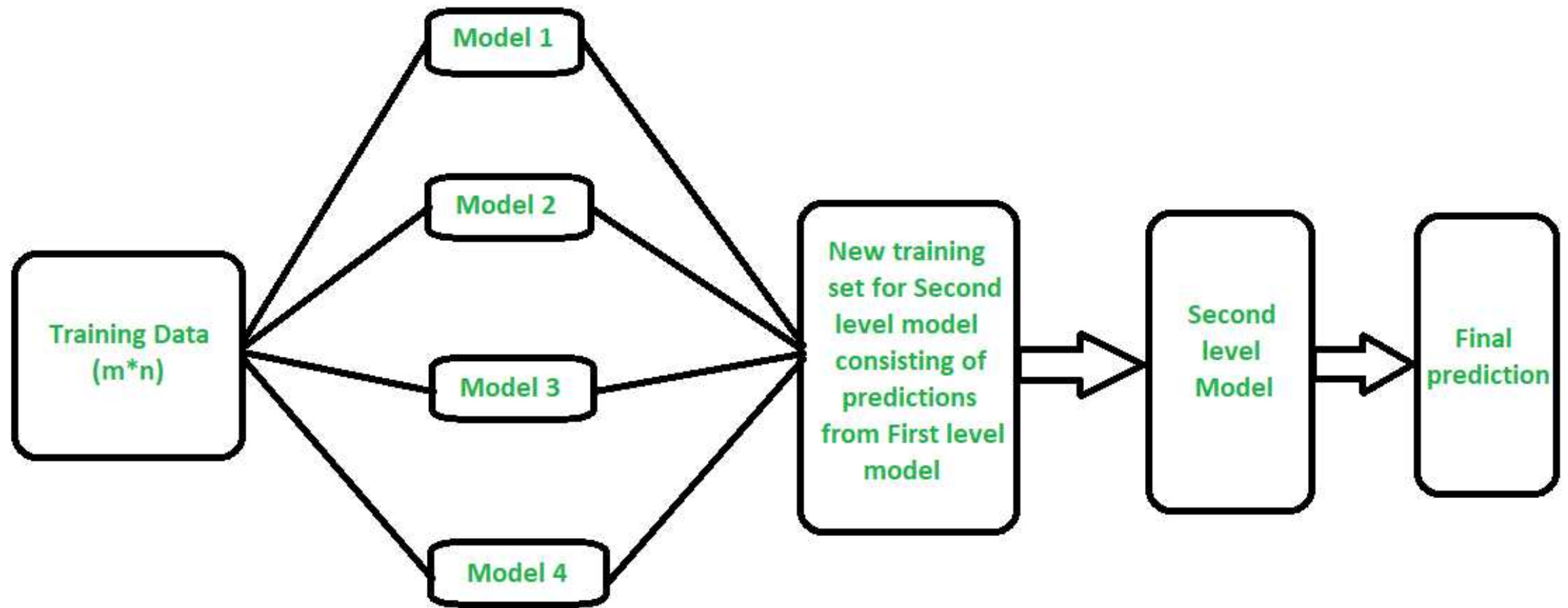


The Process of Stacking



Meta Learning





Original
Training
Data

$\mathbf{X}_{m \times n}$

Level 1 Models

model 1

model 2

⋮

model M

Level 2 training data:
M predictions are now
features

$\begin{bmatrix} \hat{y}^{\mu 1} & \dots & \hat{y}^{\mu M} \\ \vdots & & \vdots \end{bmatrix}_{m \times M}$

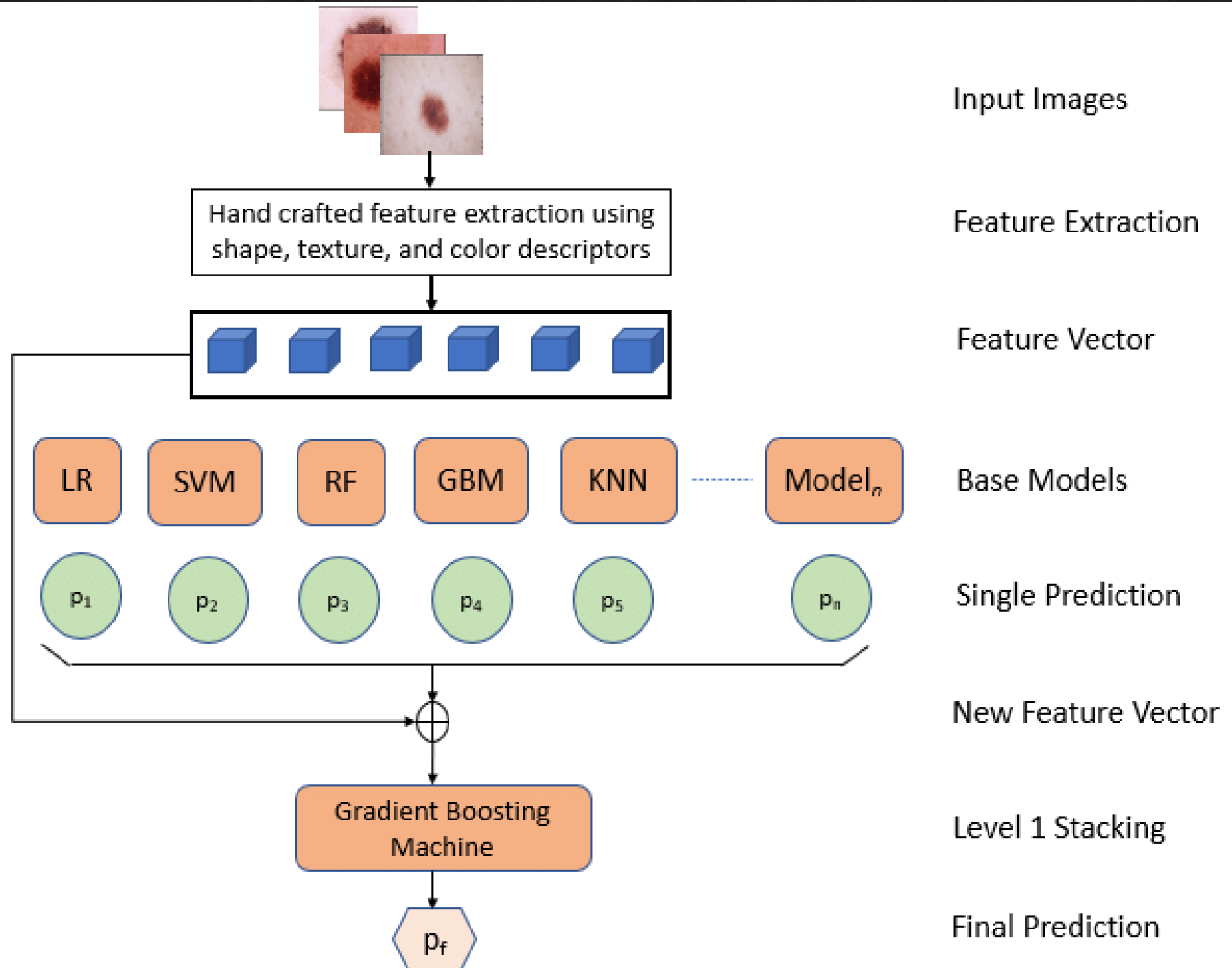
$\mathbf{X}^{(l2)}$

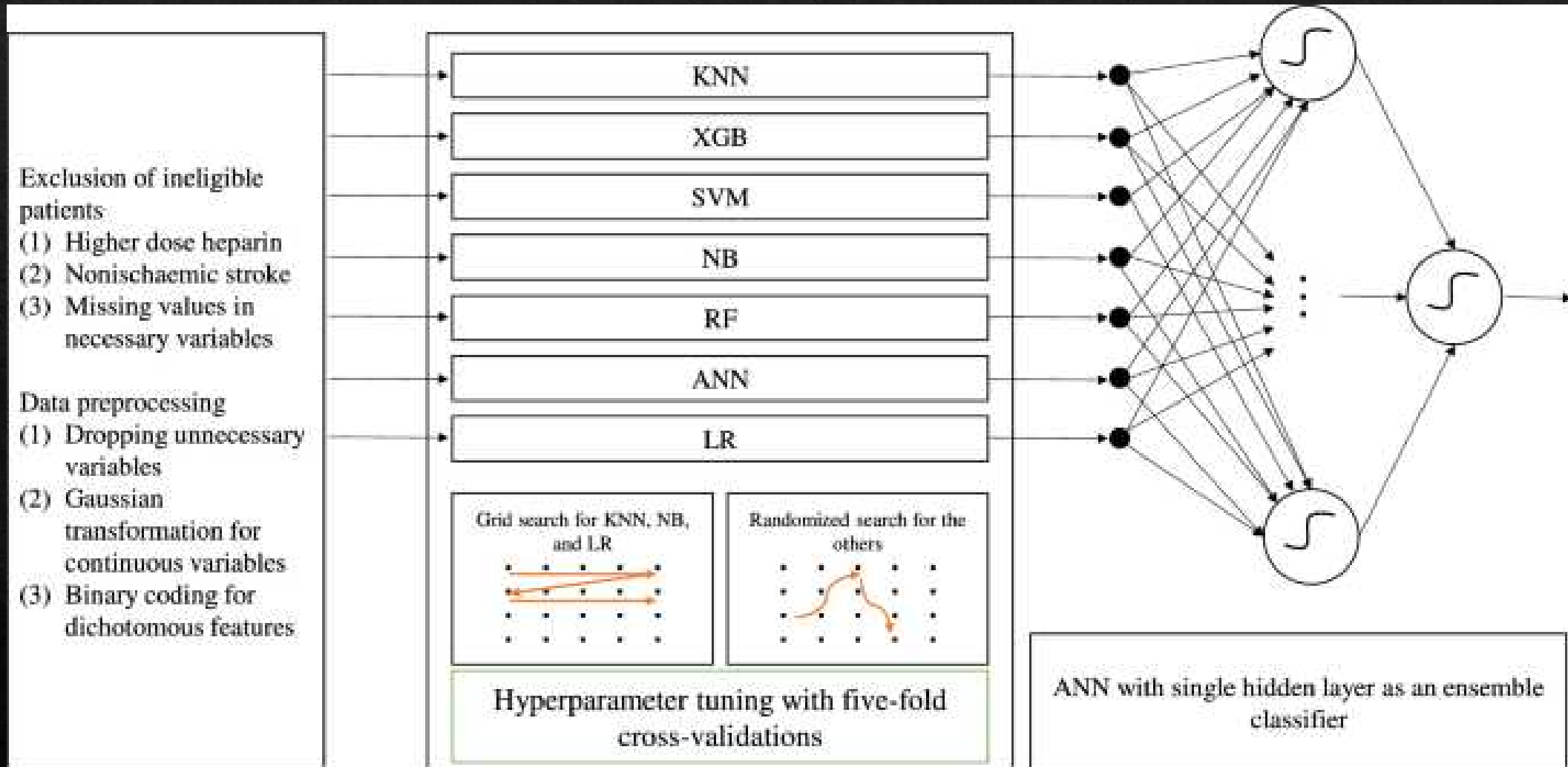
Level 2 Model(s)

Level 2 model

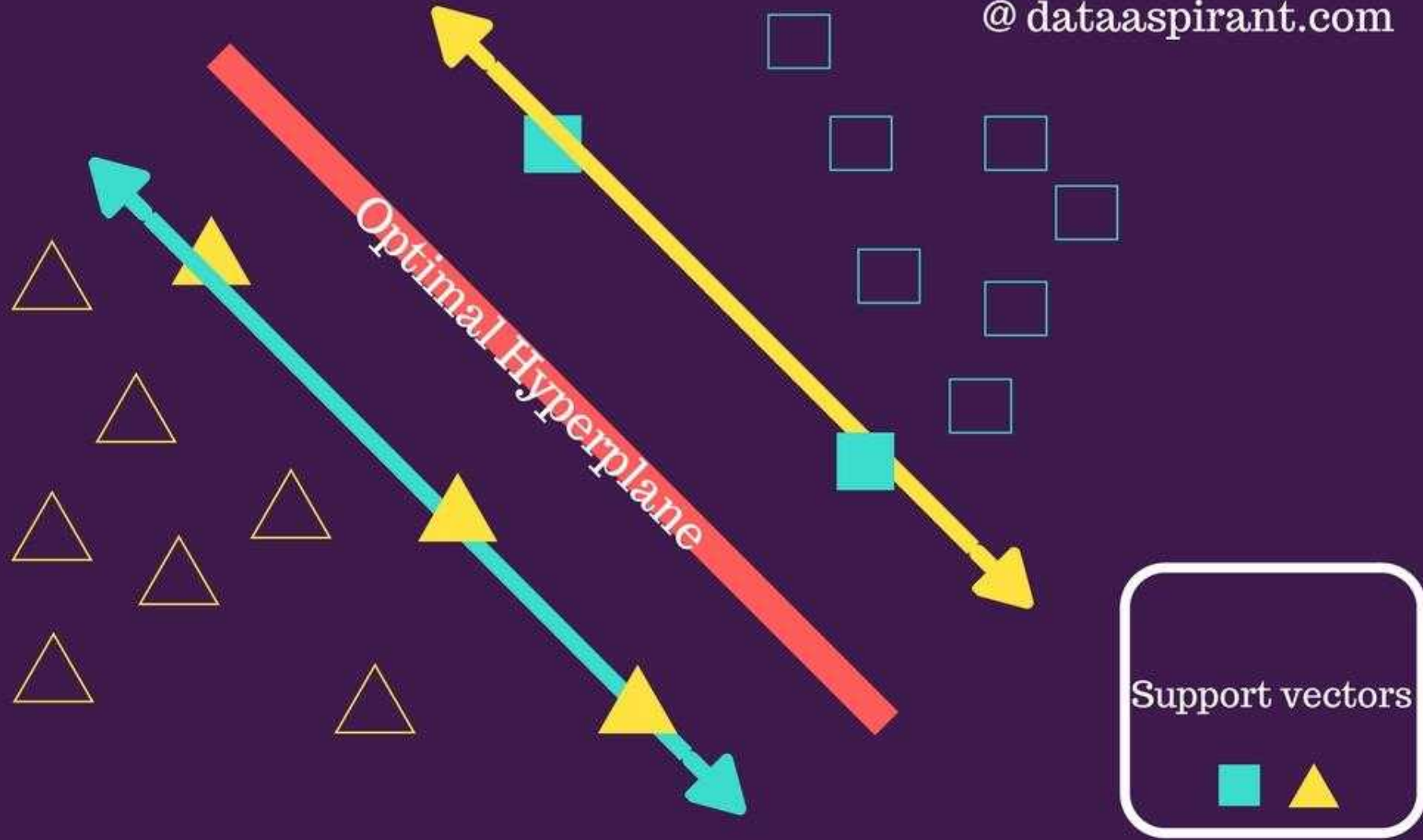
Final prediction

$\hat{y}^{\text{fin}}$



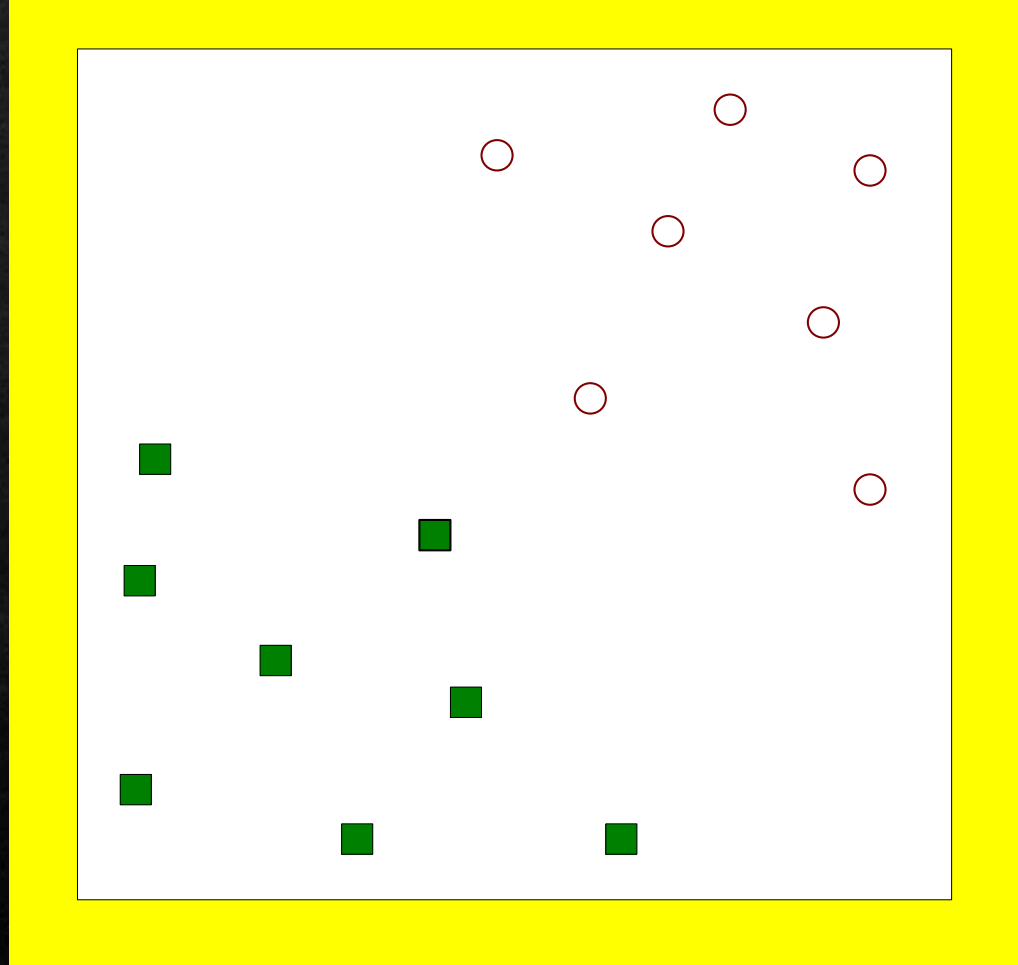


Hwangbo et al. Stacking ensemble learning model to predict 6-month mortality in ischemic stroke patients. *Scientific Reports* 12, 17389, 2022.



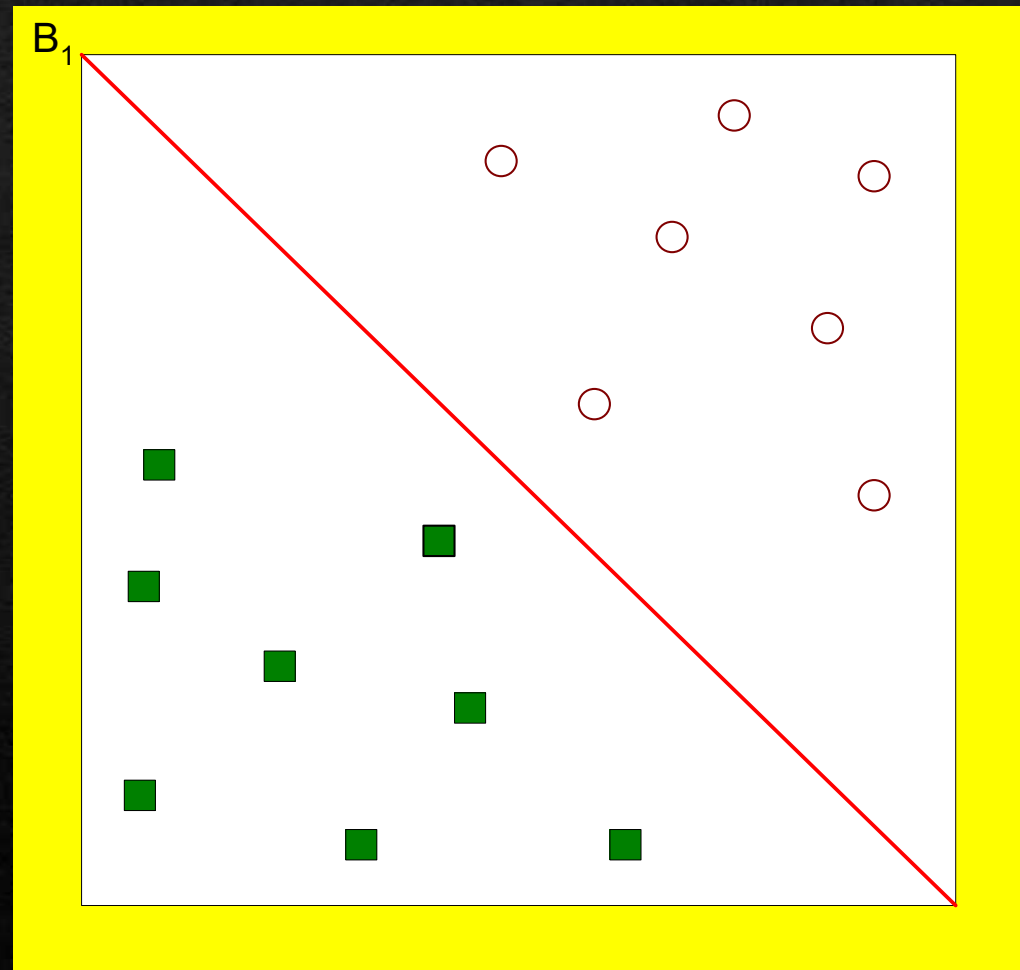
SUPPORT VECTOR MACHINE (SVM)

Support Vector Machines, SVM



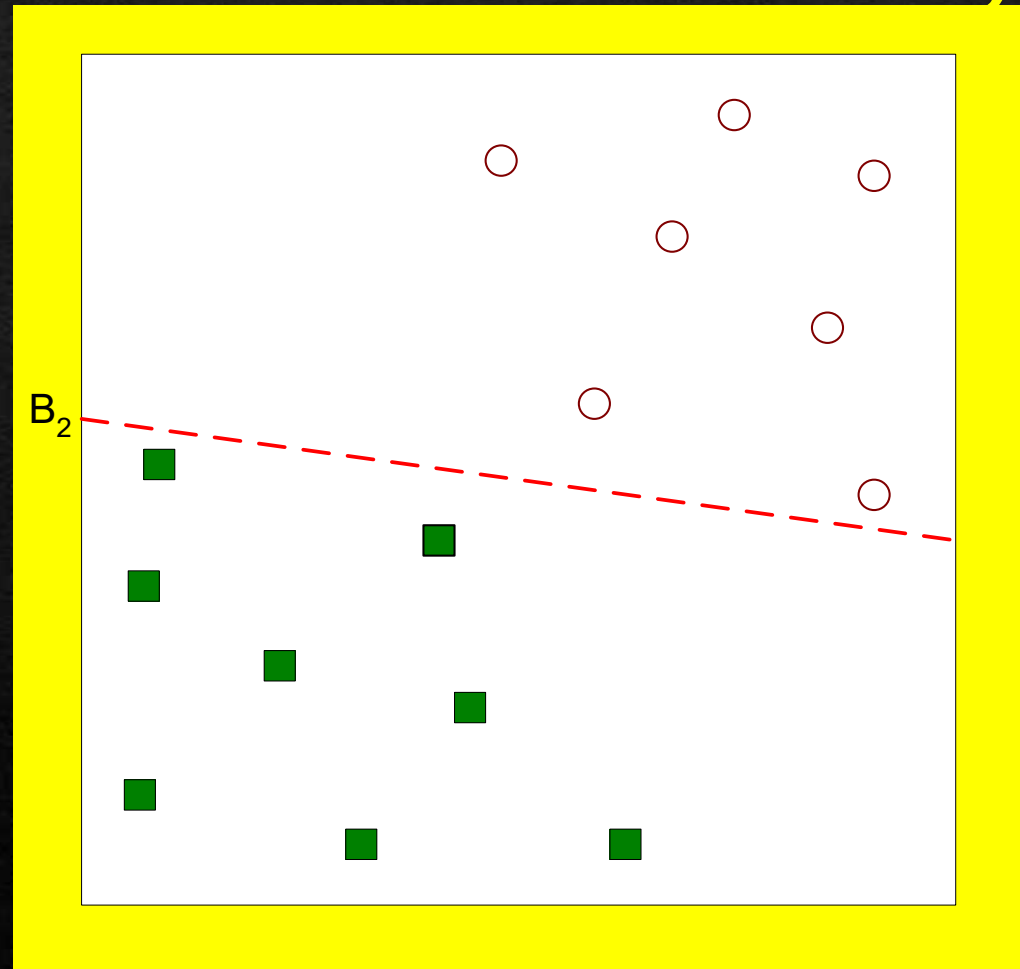
- ◆ Find a linear hyperplane (decision boundary) that will separate the data

Support Vector Machines, SVM



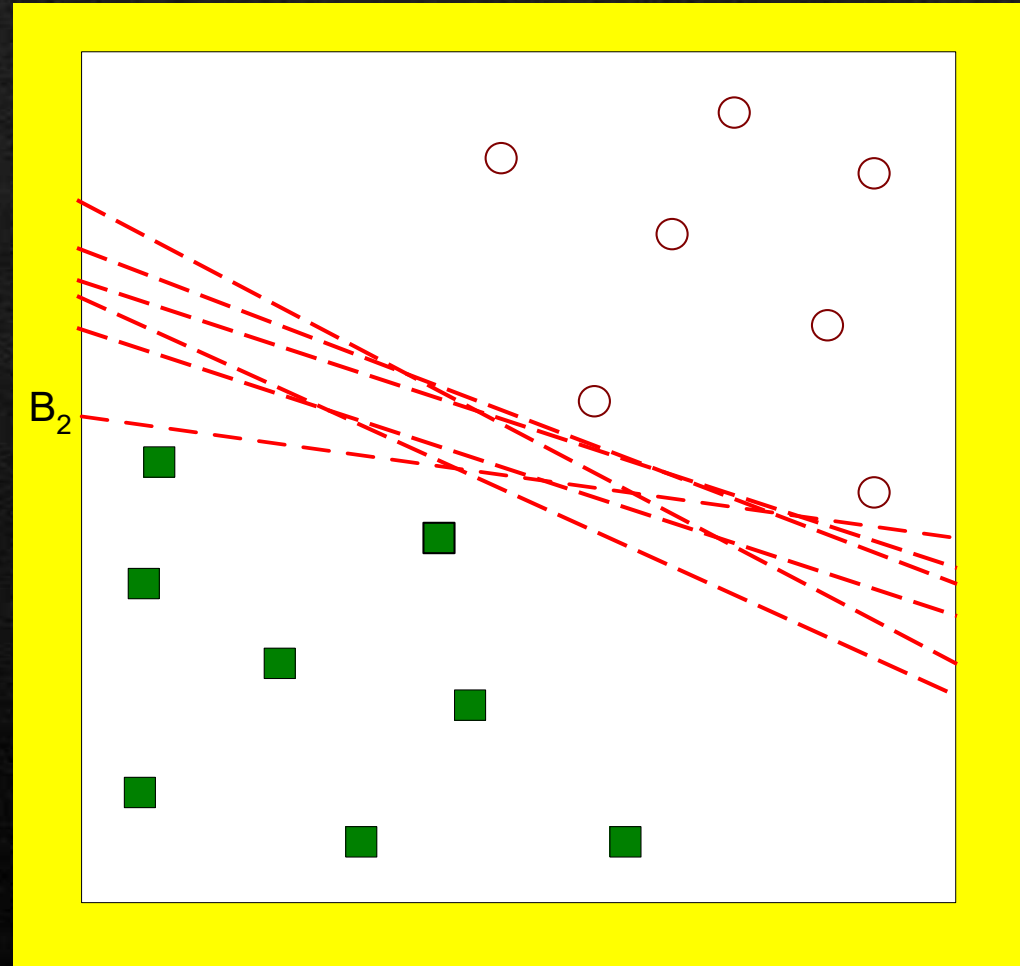
◇ One Possible Solution

Support Vector Machines, SVM



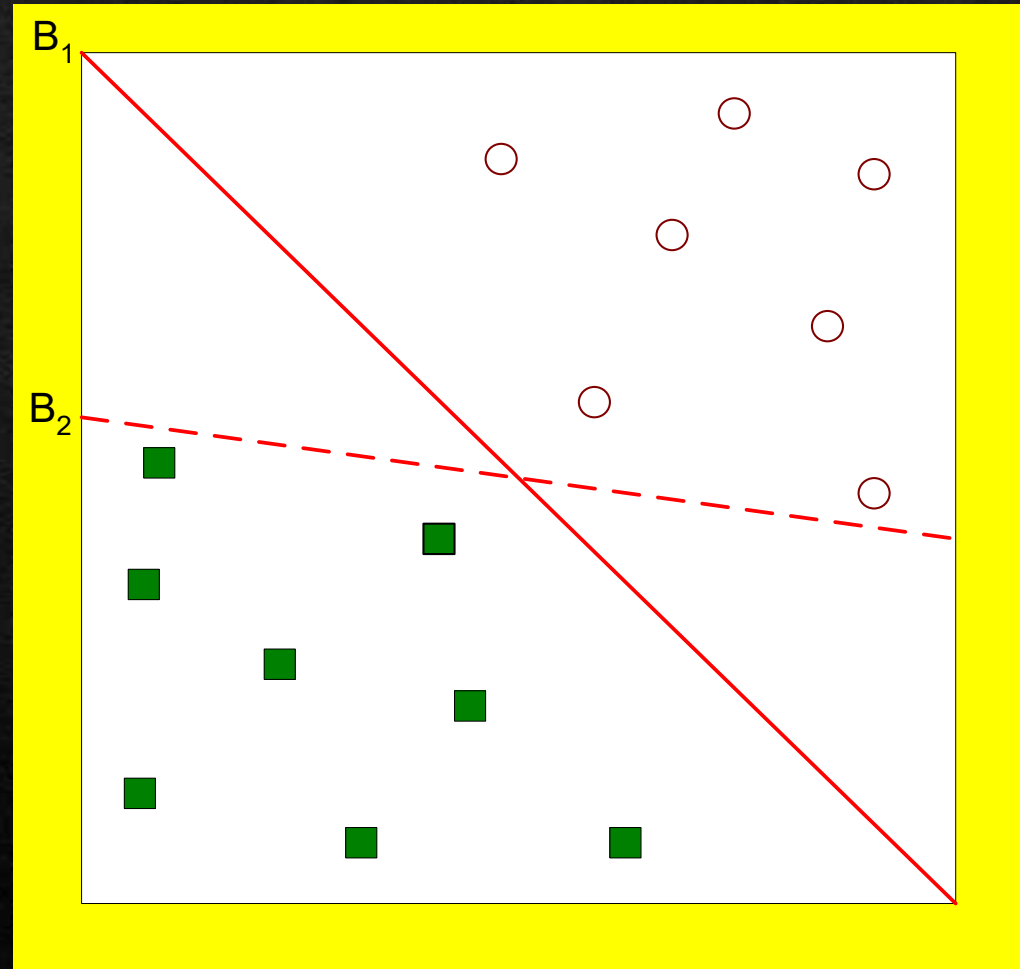
◇ Another possible solution

Support Vector Machines, SVM



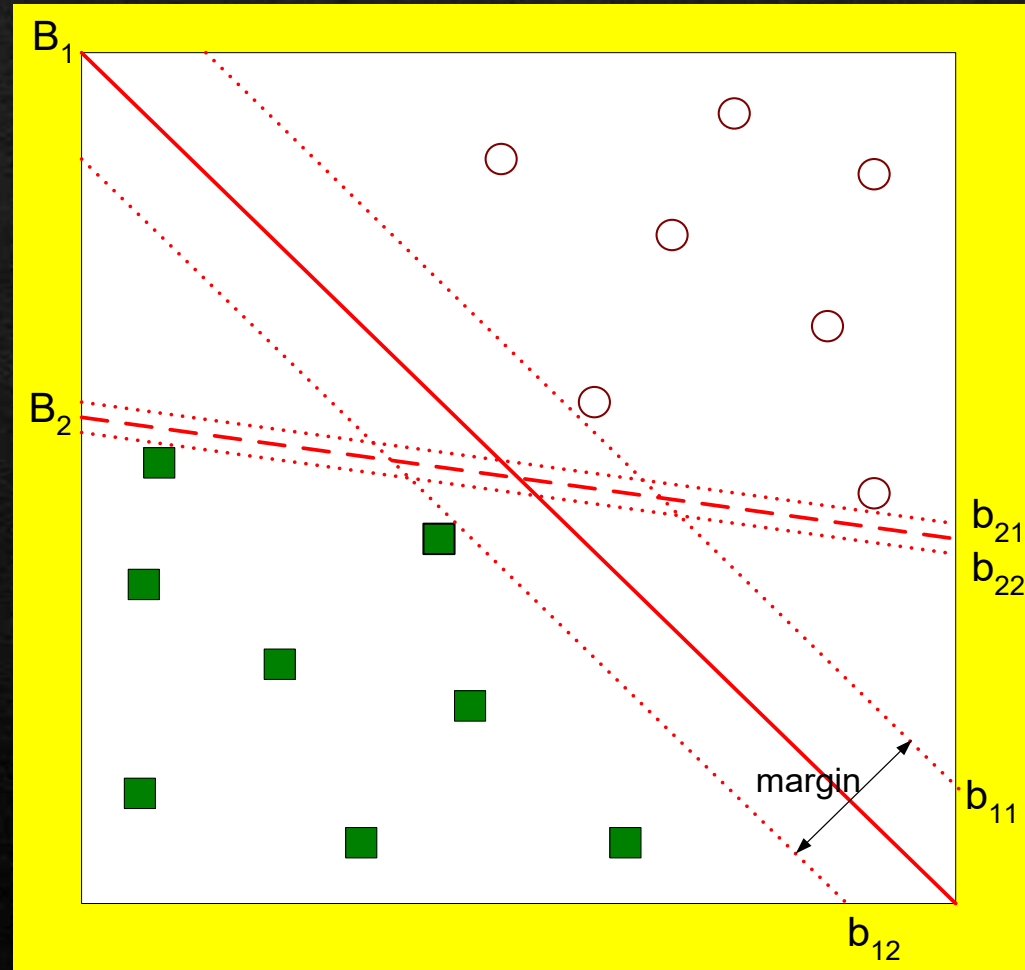
◇ Other possible solutions

Support Vector Machines, SVM



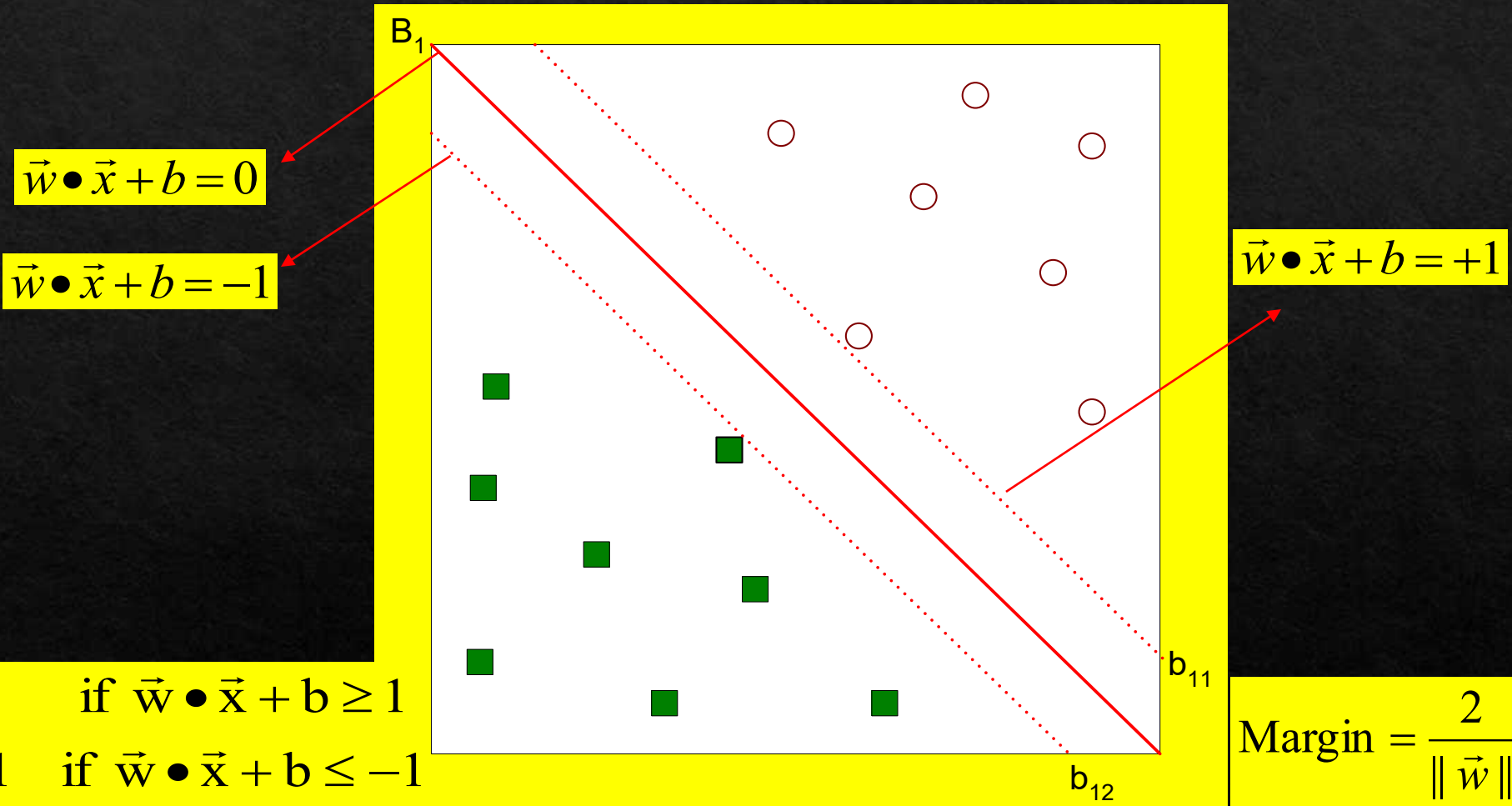
- ◆ Which one is better? B_1 or B_2 ?
- ◆ How do you define better?

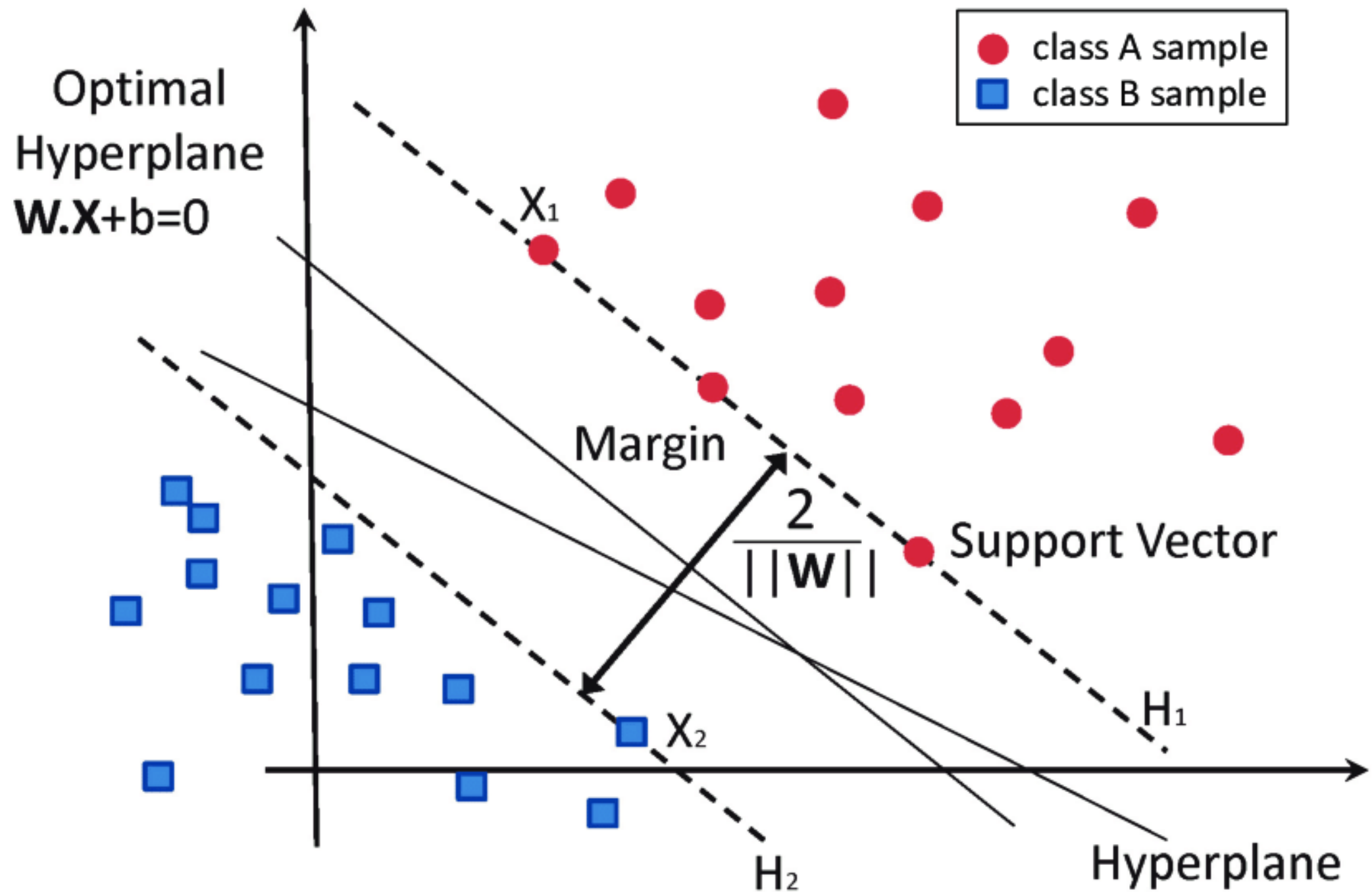
Support Vector Machines, SVM



- ◇ Find hyperplane **maximizes** the margin $\Rightarrow B_1$ is better than B_2

Support Vector Machines, SVM





Linear SVM

◆ Linear model: $f(\vec{x}) = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x} + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x} + b \leq -1 \end{cases}$

- ◆ Learning the model is equivalent to determining the values of $\vec{w}$ and b
- How to find $\vec{w}$ and b from training data?

Learning Linear SVM

♦ Objective is to maximize: $\text{Margin} = \frac{2}{\|\vec{w}\|}$

- Which is equivalent to minimizing:

$$L(\vec{w}) = \frac{\|\vec{w}\|^2}{2}$$

- Subject to the following constraints:

$$y_i = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 \end{cases}$$

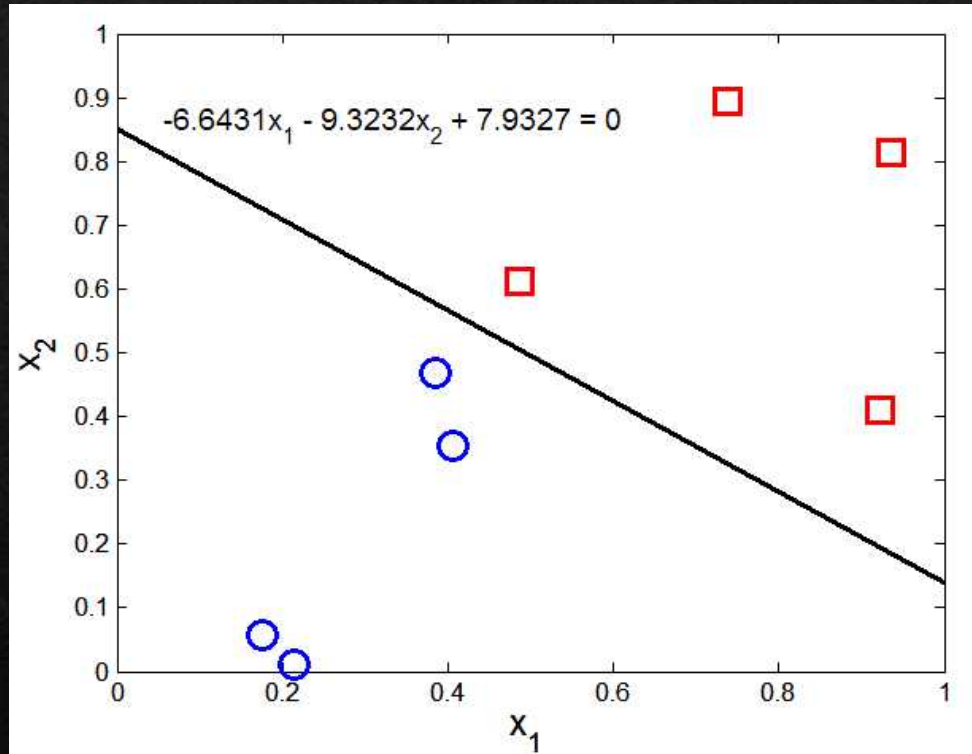
or

$$y_i(w \bullet x_i + b) \geq 1, \quad i = 1, 2, \dots, N$$

♦ This is a constrained optimization problem

▢ Solve it using the Lagrange multiplier method

Example of Linear SVM



Support vectors

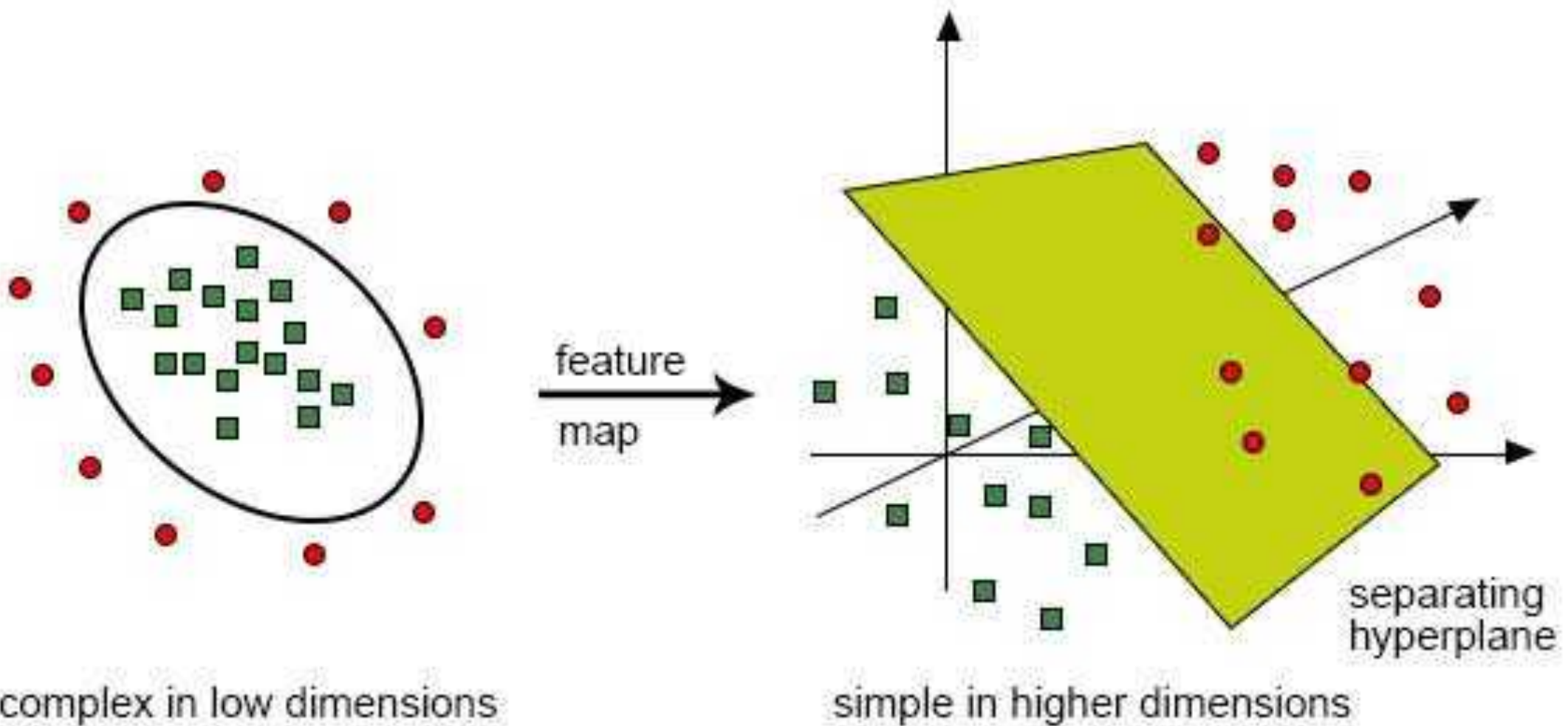
x1	x2	y	λ
0.3858	0.4687	1	65.5261
0.4871	0.611	-1	65.5261
0.9218	0.4103	-1	0
0.7382	0.8936	-1	0
0.1763	0.0579	1	0
0.4057	0.3529	1	0
0.9355	0.8132	-1	0
0.2146	0.0099	1	0

Learning Linear SVM

- ◆ Decision boundary depends only on support vectors
 - If you have a dataset with the same support vectors, the decision boundary will not change
 - How to classify using SVM once $\mathbf{w}$ and b are found? Given a test record, $\mathbf{x}_i$

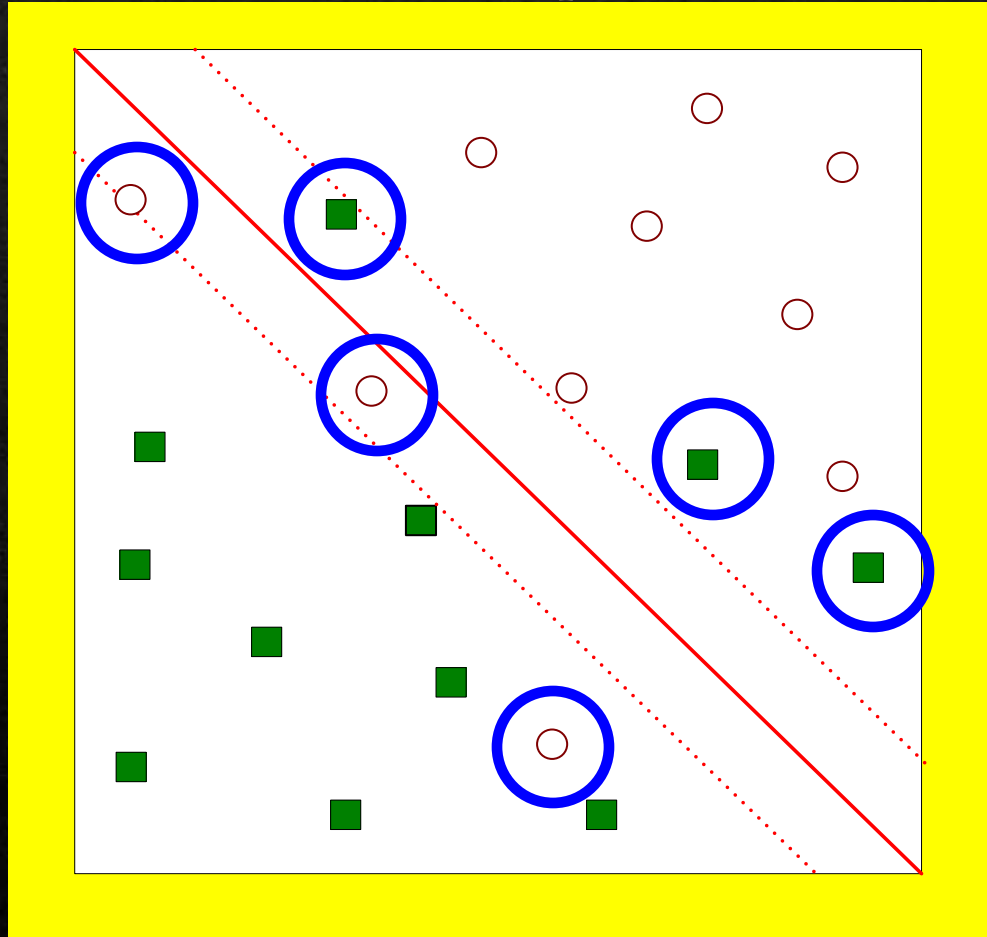
$$f(\vec{x}_i) = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 \end{cases}$$

Separation may be easier in higher dimensions



Support Vector Machines

- What if the problem is not linearly separable?



Support Vector Machines

- What if the problem is not linearly separable?
 - Introduce slack variables
 - Need to minimize:

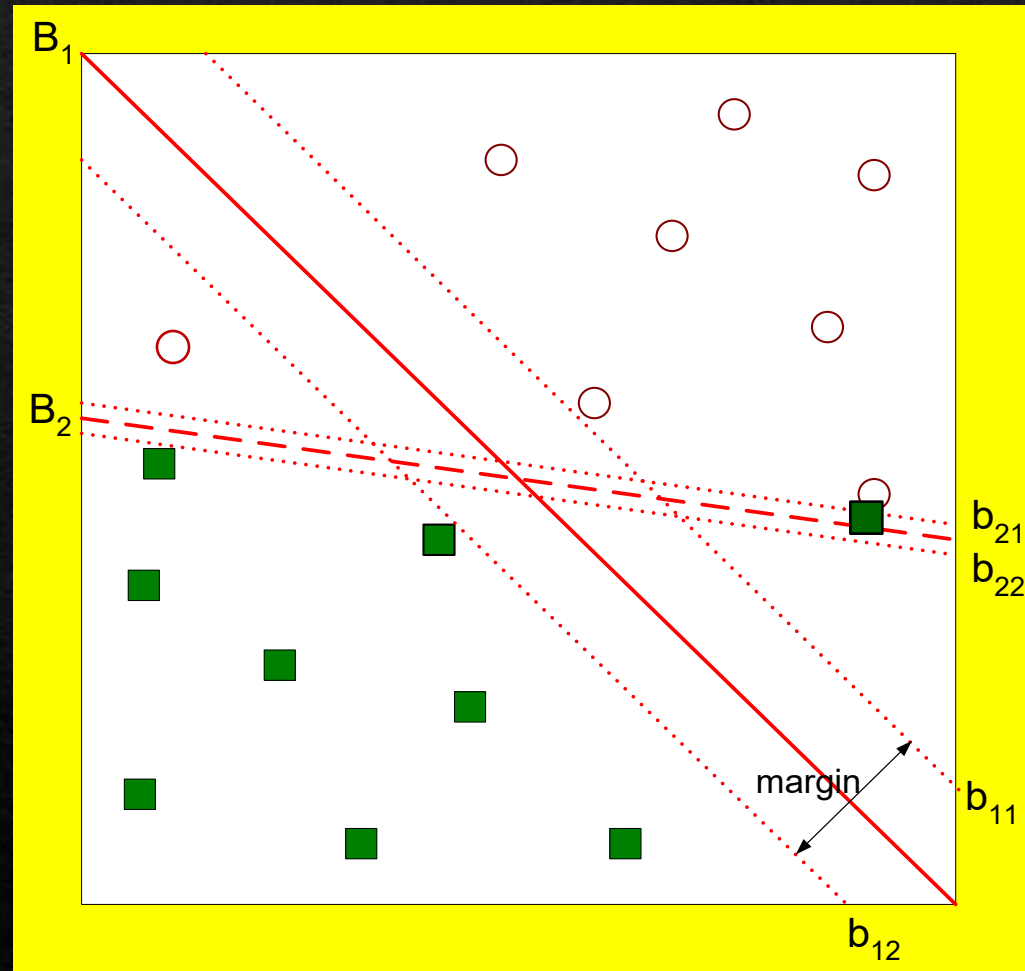
$$L(w) = \frac{\|\vec{w}\|^2}{2} + C \left(\sum_{i=1}^N \xi_i \right)$$

- Subject to:

$$y_i = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 - \xi_i \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 + \xi_i \end{cases}$$

- If k is 1 or 2, this leads to a similar objective function as linear SVM but with different constraints

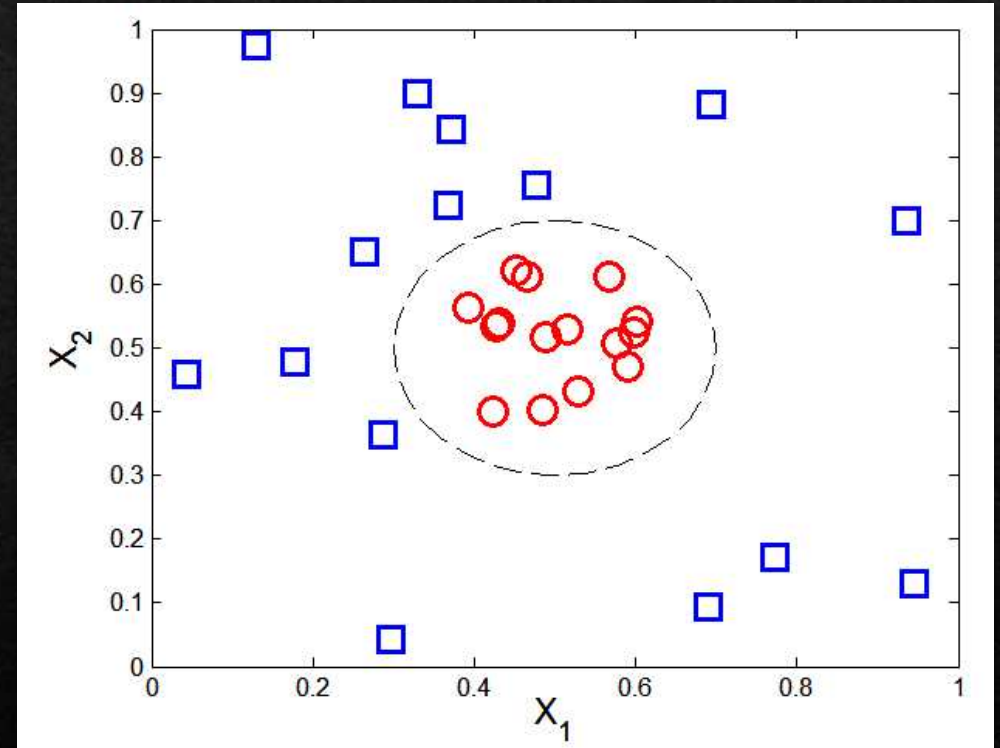
Support Vector Machines



- ◇ Find the hyperplane that optimizes both factors

Nonlinear Support Vector Machines

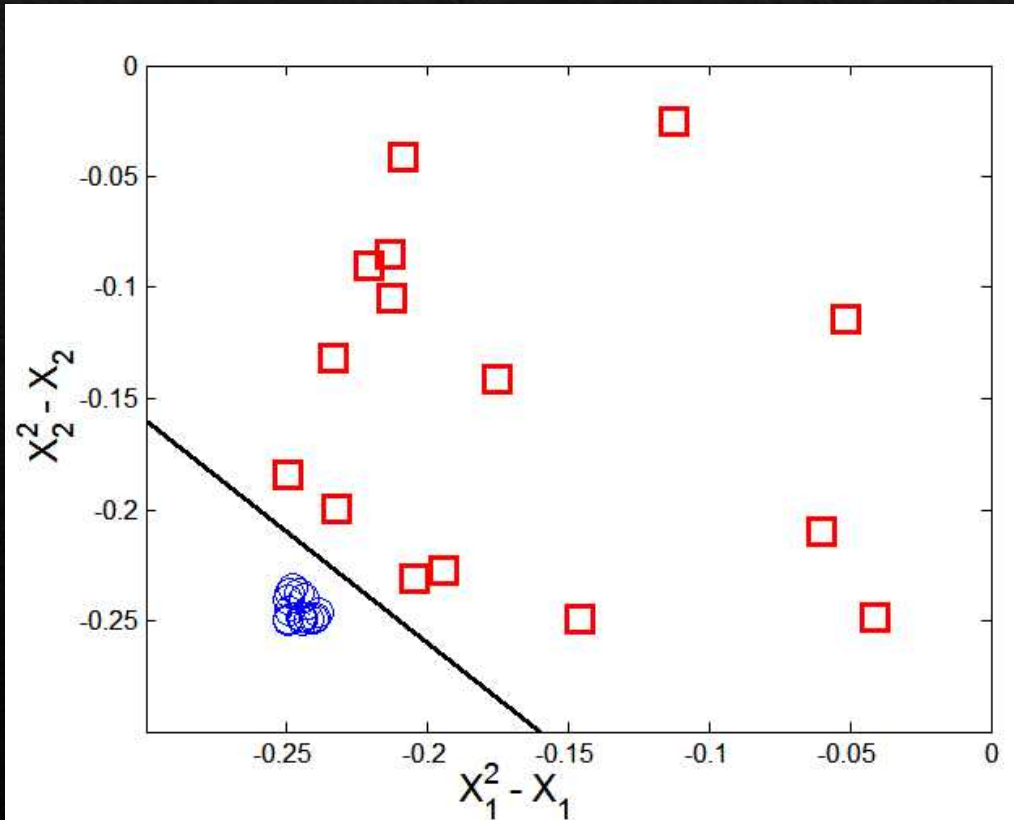
- What if decision boundary is not linear?



$$y(x_1, x_2) = \begin{cases} 1 & \text{if } \sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2} > 0.2 \\ -1 & \text{otherwise} \end{cases}$$

Nonlinear Support Vector Machines

- ◆ Transform data into higher dimensional space



$$x_1^2 - x_1 + x_2^2 - x_2 = -0.46.$$

$$\Phi : (x_1, x_2) \longrightarrow (x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, 1).$$

$$w_4x_1^2 + w_3x_2^2 + w_2\sqrt{2}x_1 + w_1\sqrt{2}x_2 + w_0 = 0.$$

Decision boundary:

$$\vec{w} \bullet \Phi(\vec{x}) + b = 0$$

Learning Nonlinear SVM

- ♦ Optimization problem:

$$\begin{aligned} & \min_{\mathbf{w}} \frac{\|\mathbf{w}\|^2}{2} \\ & \text{subject to} \quad y_i(\mathbf{w} \cdot \Phi(\mathbf{x}_i) + b) \geq 1, \quad \forall \{(\mathbf{x}_i, y_i)\} \end{aligned}$$

- ♦ Which leads to the same set of equations (but involve $\Phi(\mathbf{x})$ instead of $\mathbf{x}$)

$$L_D = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}_j)$$

$$\begin{aligned} \mathbf{w} &= \sum_i \lambda_i y_i \Phi(\mathbf{x}_i) \\ \lambda_i \{ y_i (\sum_j \lambda_j y_j \Phi(\mathbf{x}_j) \cdot \Phi(\mathbf{x}_i) + b) - 1 \} &= 0, \end{aligned}$$

$$f(\mathbf{z}) = \text{sign}(\mathbf{w} \cdot \Phi(\mathbf{z}) + b) = \text{sign}(\sum_{i=1}^n \lambda_i y_i \Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{z}) + b).$$

Learning NonLinear SVM

- ◆ Issues:
 - What type of mapping function Φ should be used?
 - How to do the computation in high dimensional space?
 - ◆ Most computations involve dot product $\Phi(\mathbf{x}_i) \bullet \Phi(\mathbf{x}_j)$
 - ◆ Curse of dimensionality?

Learning Nonlinear SVM

- ◆ Kernel Trick:

- $\Phi(\mathbf{x}_i) \bullet \Phi(\mathbf{x}_j) = K(\mathbf{x}_i, \mathbf{x}_j)$

- $K(\mathbf{x}_i, \mathbf{x}_j)$ is a kernel function (expressed in terms of the coordinates in the original space)

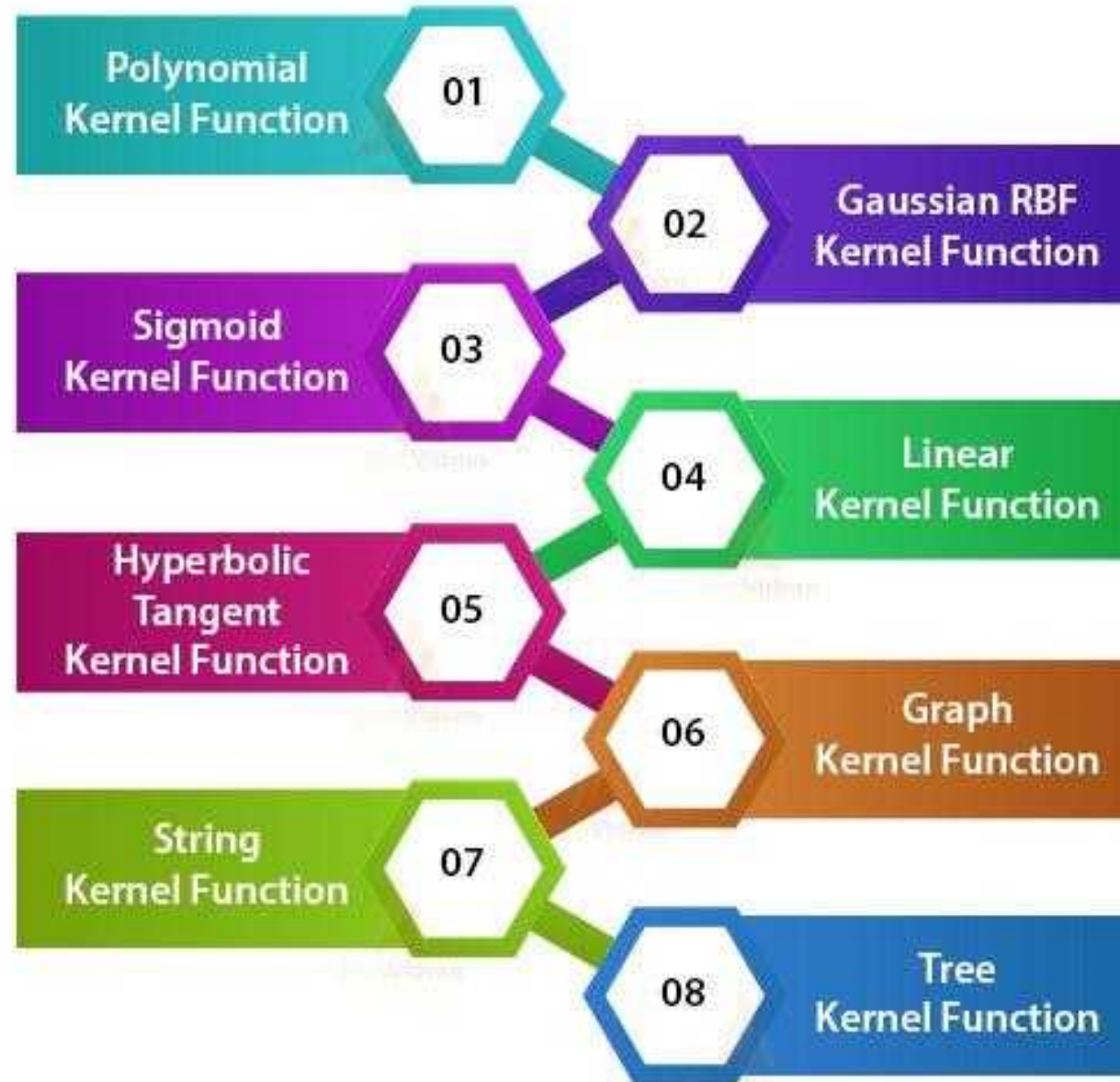
- ◆ Examples:

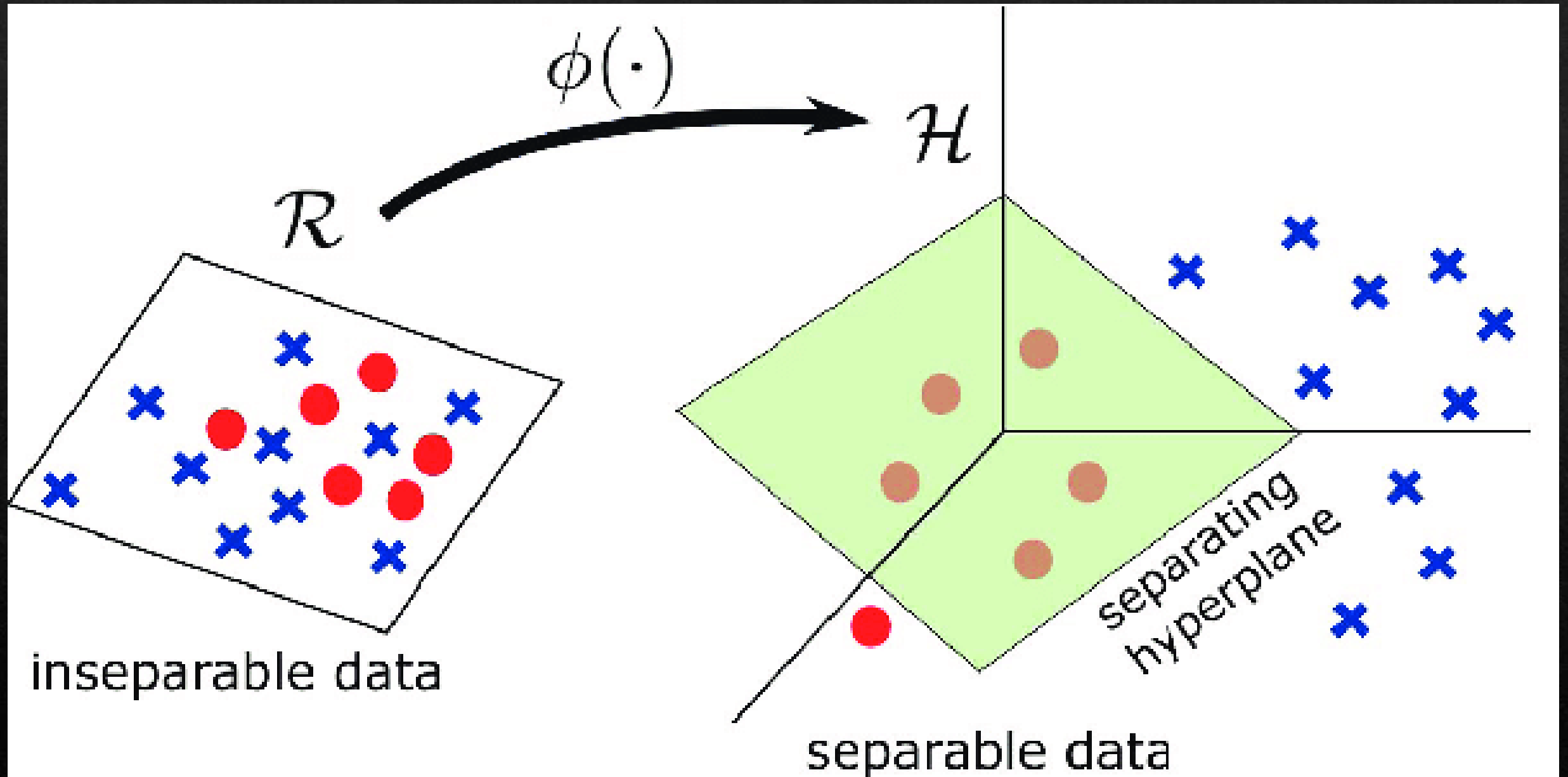
$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + 1)^p$$

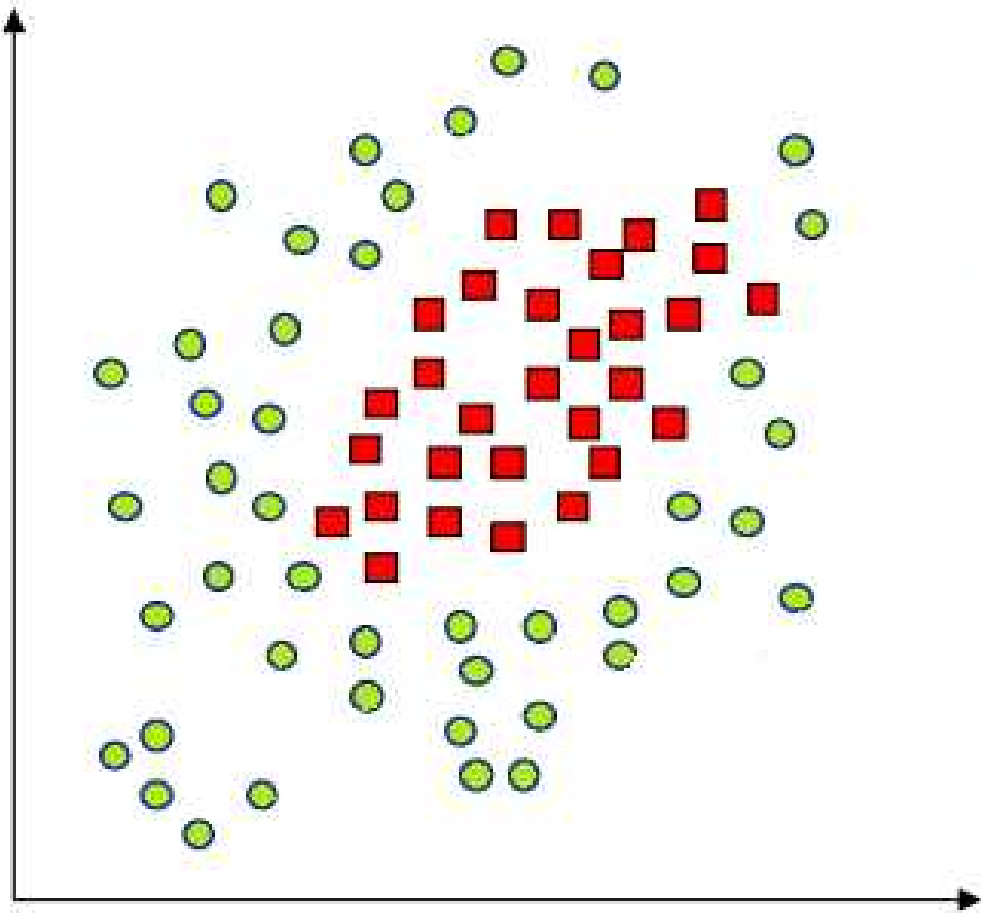
$$K(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x} - \mathbf{y}\|^2 / (2\sigma^2)}$$

$$K(\mathbf{x}, \mathbf{y}) = \tanh(k\mathbf{x} \cdot \mathbf{y} - \delta)$$

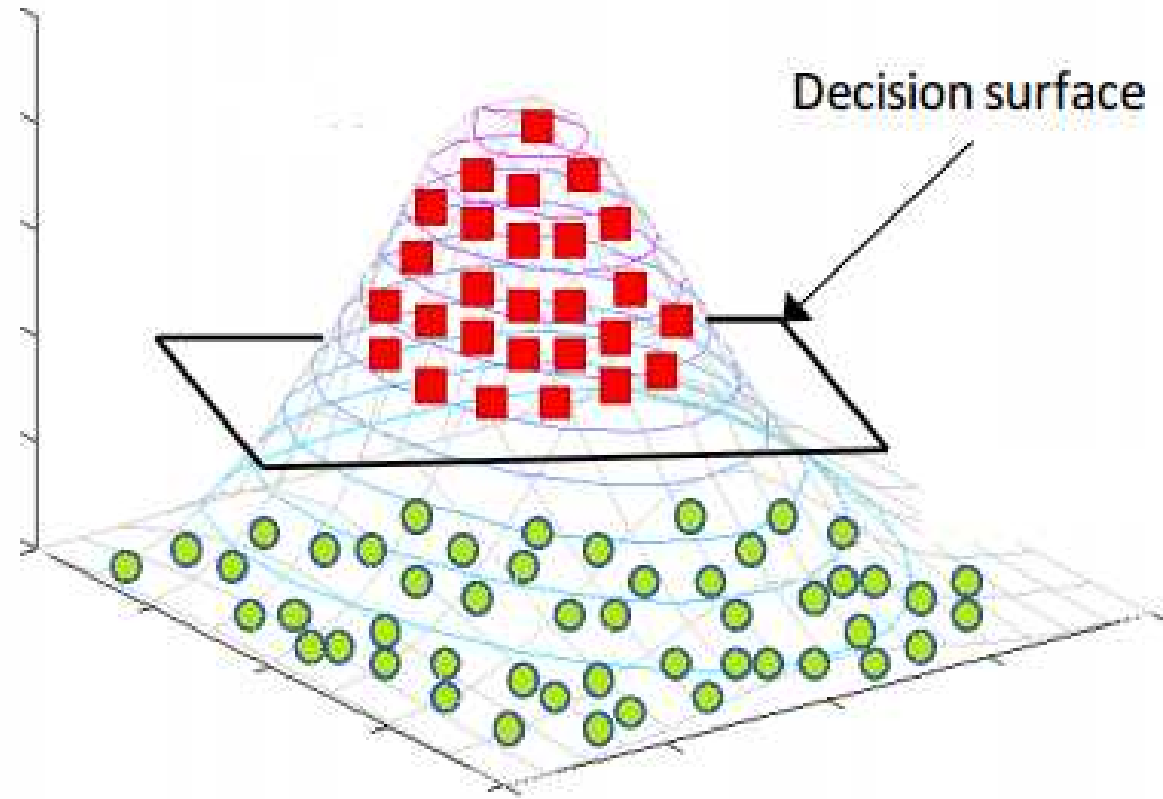
Types of Kernel Functions



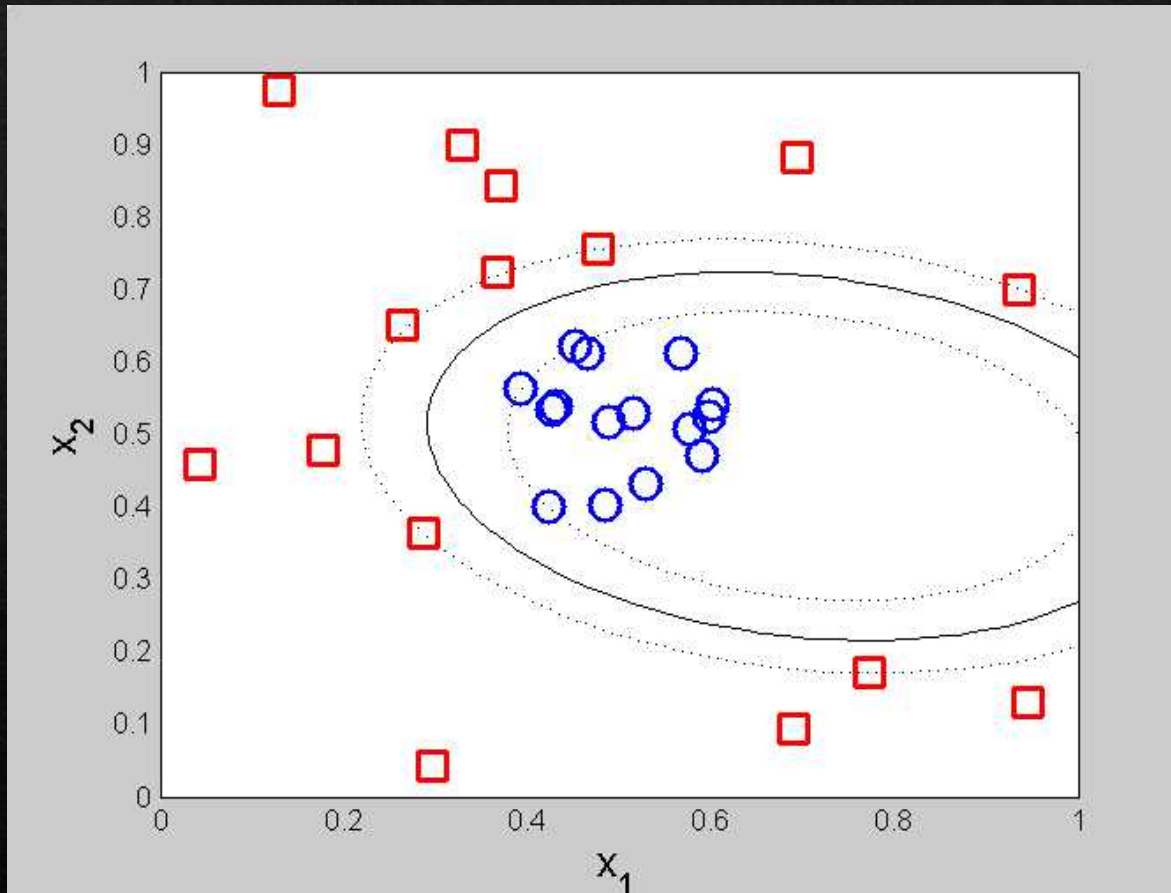




kernel
→

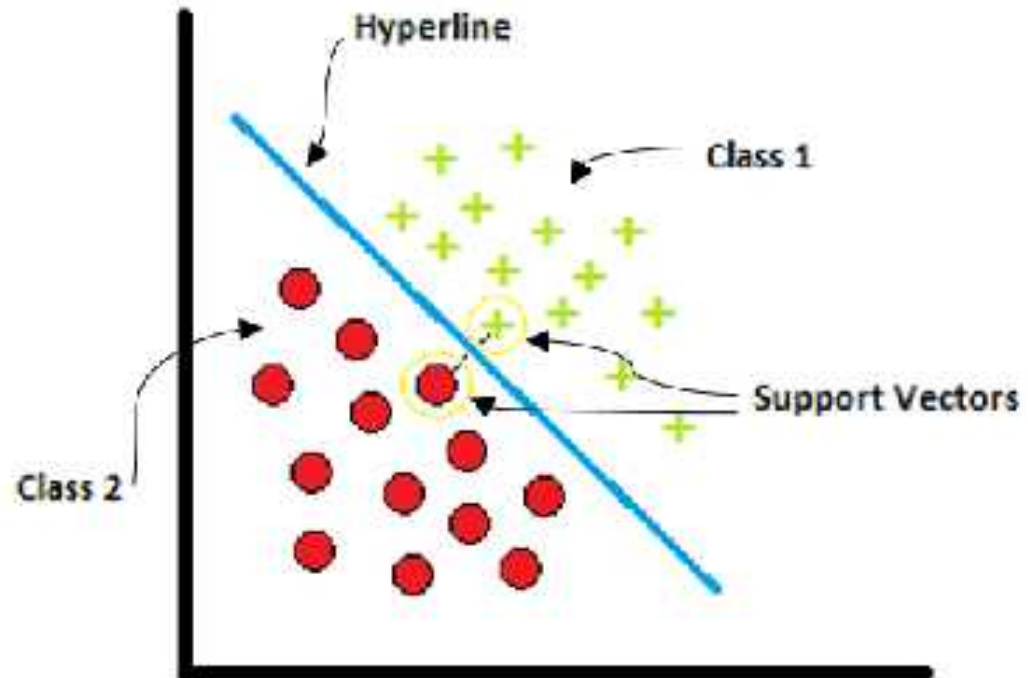


Example of Nonlinear SVM

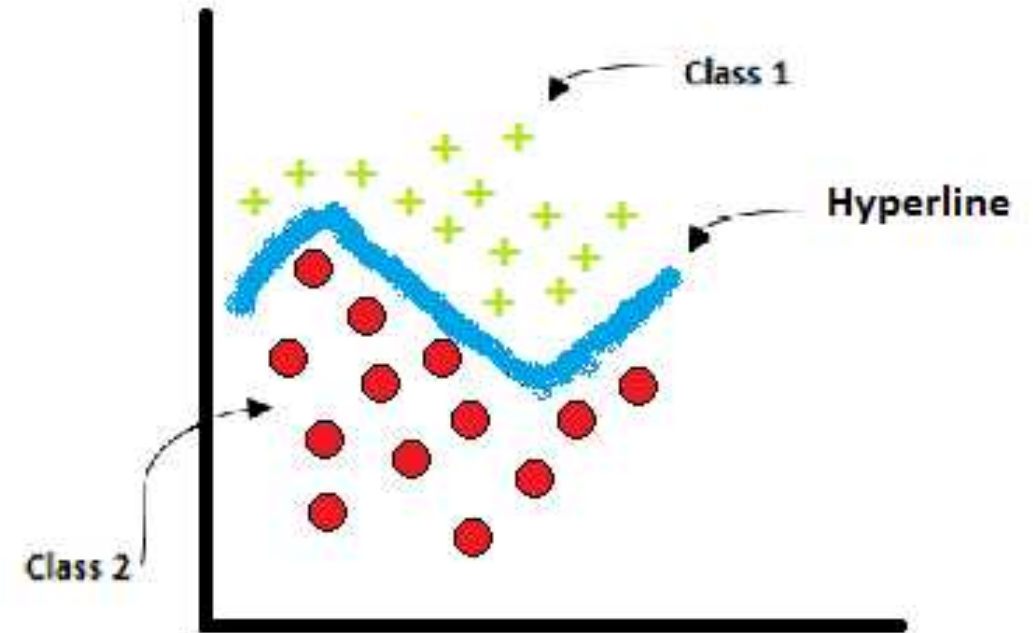


SVM with polynomial
degree 2 kernel

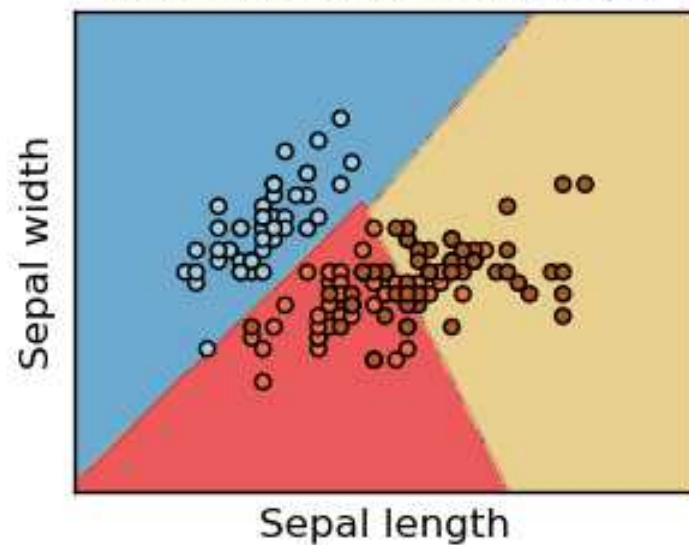
Linear Classification



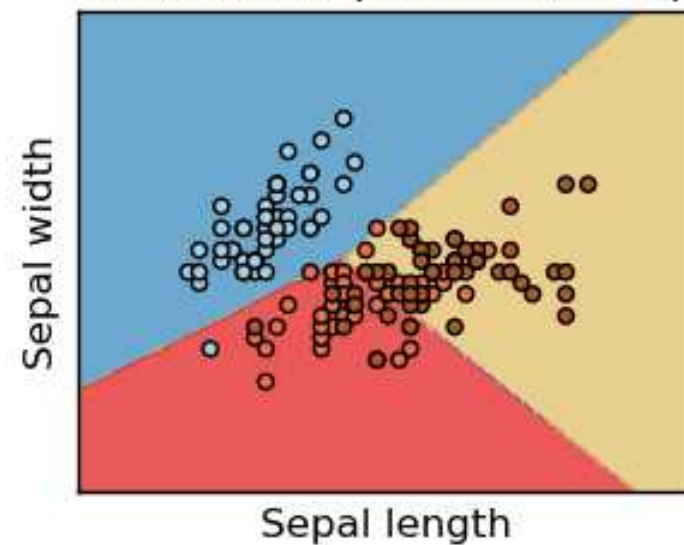
Polynomial Classification



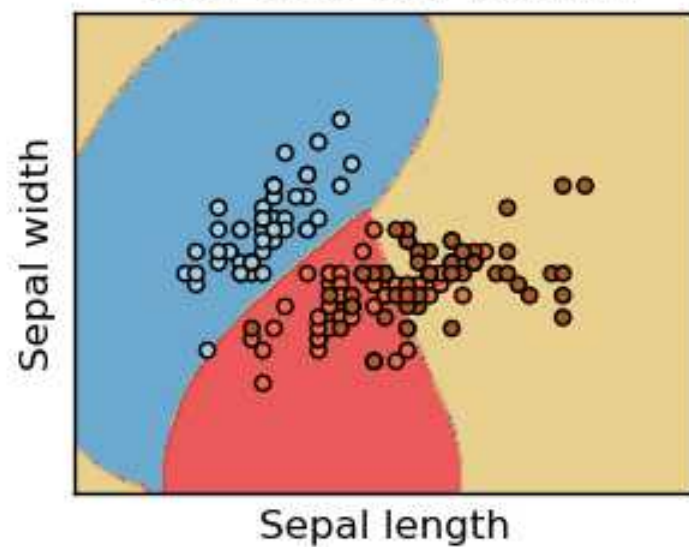
SVC with linear kernel



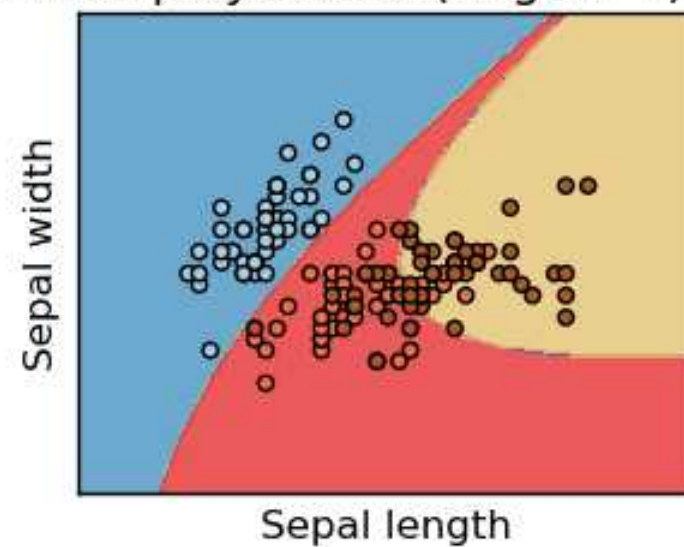
LinearSVC (linear kernel)



SVC with RBF kernel



SVC with polynomial (degree 3) kernel



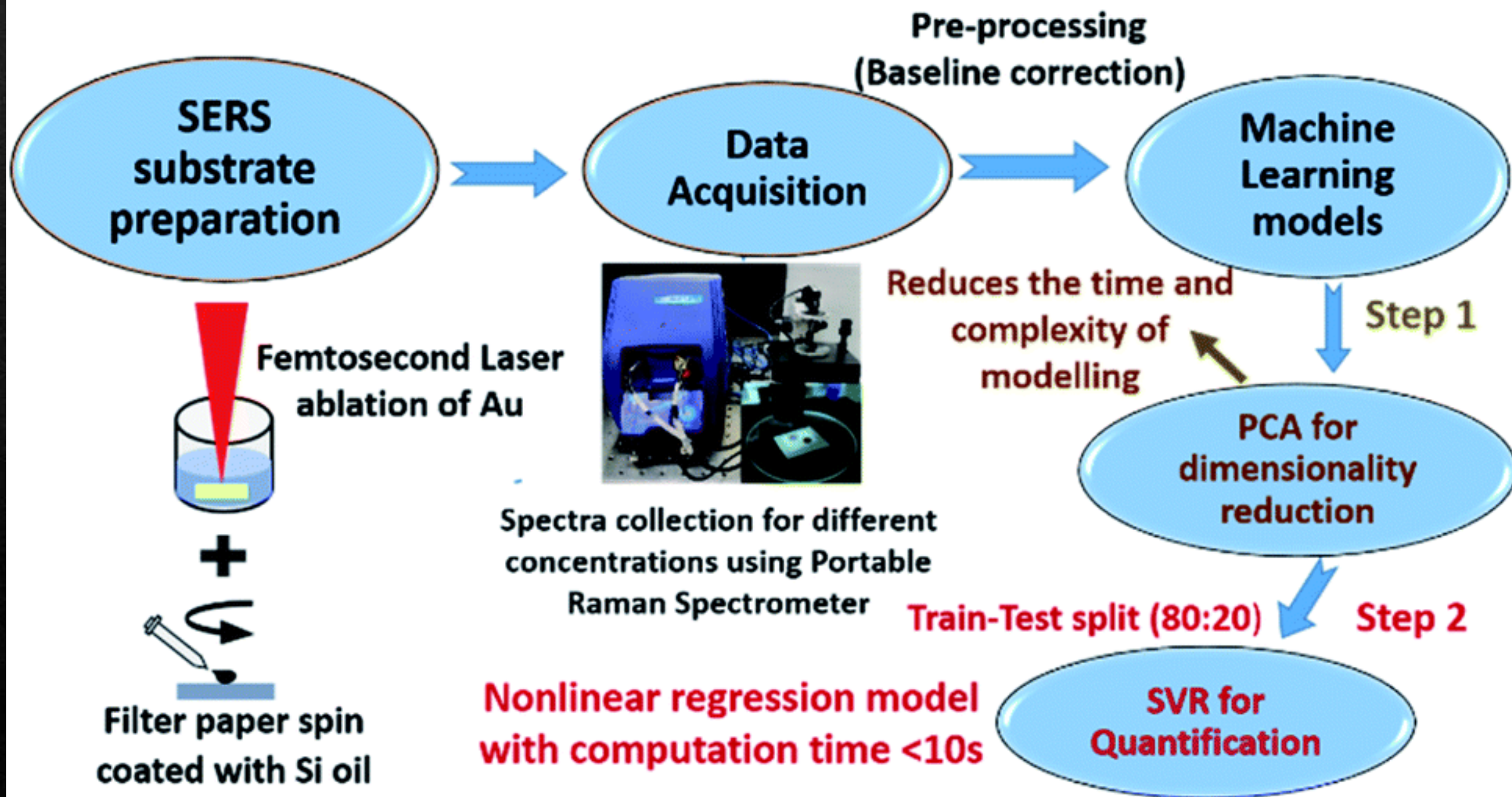
Learning Nonlinear SVM

- ◆ Advantages of using kernel:
 - Do not have to know the mapping function Φ
 - Computing dot product $\Phi(x_i) \bullet \Phi(x_j)$ in the original space avoids the curse of dimensionality
- ◆ Not all functions can be kernels
 - Must make sure there is a corresponding Φ in some high-dimensional space
 - Mercer's theorem

Characteristics of SVM

- ◆ The learning problem is formulated as a convex optimization problem
 - Efficient algorithms are available to find the global minima
 - Many of the other methods use greedy approaches and find locally optimal solutions
 - High computational complexity for building the model
- ◆ Robust to noise
- ◆ Overfitting is handled by maximizing the margin of the decision boundary,
- ◆ SVM can handle irrelevant and redundant attributes better than many other techniques
- ◆ The user needs to provide the type of kernel function and cost function
- ◆ Difficult to handle missing values
- ◆ What about categorical variables?

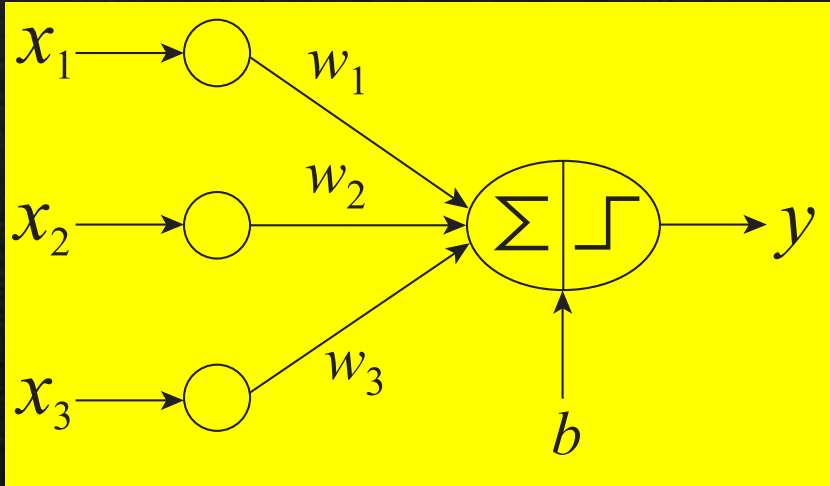
Machine Learning for Quantification in SERS using flexible substrate



Artificial Neural Networks (ANN)

- ◆ **Basic Idea:** A complex non-linear function can be learned as a composition of simple processing units
- ◆ ANN is a collection of simple processing units (nodes) that are connected by directed links (edges)
 - Every node receives signals from incoming edges, performs computations, and transmits signals to outgoing edges
 - Analogous to the *human brain*, where nodes are neurons and signals are electrical impulses
 - The weight of an edge determines the strength of the connection between the nodes
- ◆ Simplest ANN: **Perceptron** (single neuron)

Basic Architecture of Perceptron



$$y = \begin{cases} 1, & \text{if } \mathbf{w}^T \mathbf{x} + b > 0. \\ -1, & \text{otherwise.} \end{cases}$$

$$\tilde{\mathbf{w}} = (\mathbf{w}^T \ b)^T$$

$$\tilde{\mathbf{x}} = (\mathbf{x}^T \ 1)^T$$

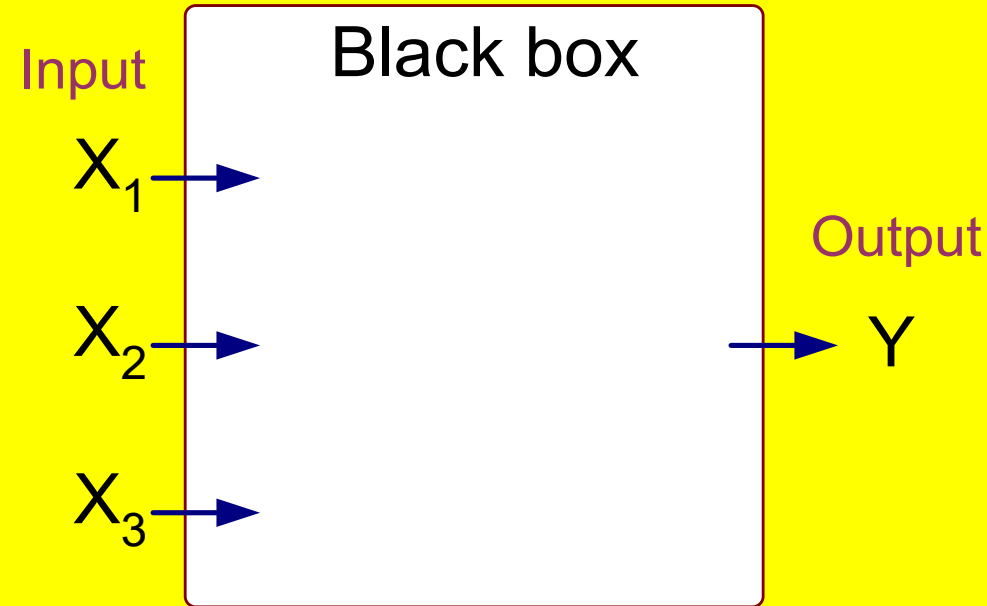
$$\hat{y} = \text{sign}(\tilde{\mathbf{w}}^T \tilde{\mathbf{x}})$$

Activation Function

- Learns linear decision boundaries
- Related to logistic regression (activation function is sign instead of sigmoid)

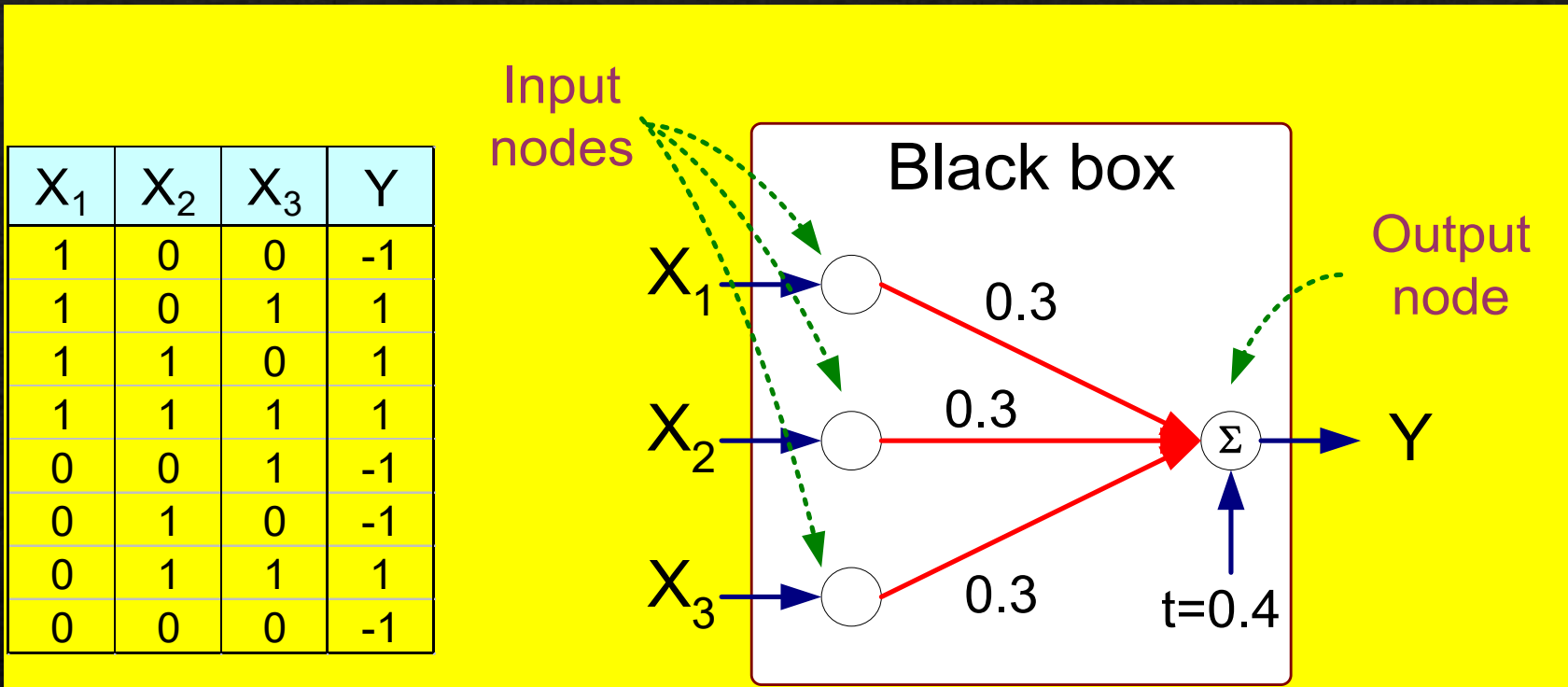
Perceptron Example

X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



Output Y is 1 if at least two of the three inputs are equal to 1.

Perceptron Example



$$Y = \text{sign}(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$$

$$\text{where } \text{sign}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Perceptron Learning Rule

- ◆ Initialize the weights ($w_o, w_1, \dots, w_d$)
- ◆ Repeat
 - ▢ For each training example (x_i, y_i)
 - ◆ Compute $\hat{y}_i$
 - ◆ Update the weights:

$$w_j^{(k+1)} = w_j^{(k)} + \lambda (y_i - \hat{y}_i^{(k)}) x_{ij}$$

- ◆ Until stopping condition is met
- ◆ k : iteration number; λ : learning rate

Perceptron Learning Rule

- ◆ Weight update formula:

$$w_j^{(k+1)} = w_j^{(k)} + \lambda (y_i - \hat{y}_i^{(k)}) x_{ij}$$

- ◆ Intuition:

- Update weight based on error: $e = (y_i - \hat{y}_i)$

- ◆ If $y = \hat{y}$, $e=0$: no update needed
- ◆ If $y > \hat{y}$, $e=2$: weight must be increased (assuming X_{ij} is positive) so that $\hat{y}$ will increase
- ◆ If $y < \hat{y}$, $e=-2$: weight must be decreased (assuming X_{ij} is positive) so that $\hat{y}$ will decrease

Example of Perceptron Learning

$$\lambda = 0.1$$

X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1

	w_0	w_1	w_2	w_3
0	0	0	0	0
1	-0.2	-0.2	0	0
2	0	0	0	0.2
3	0	0	0	0.2
4	0	0	0	0.2
5	-0.2	0	0	0
6	-0.2	0	0	0
7	0	0	0.2	0.2
8	-0.2	0	0.2	0.2

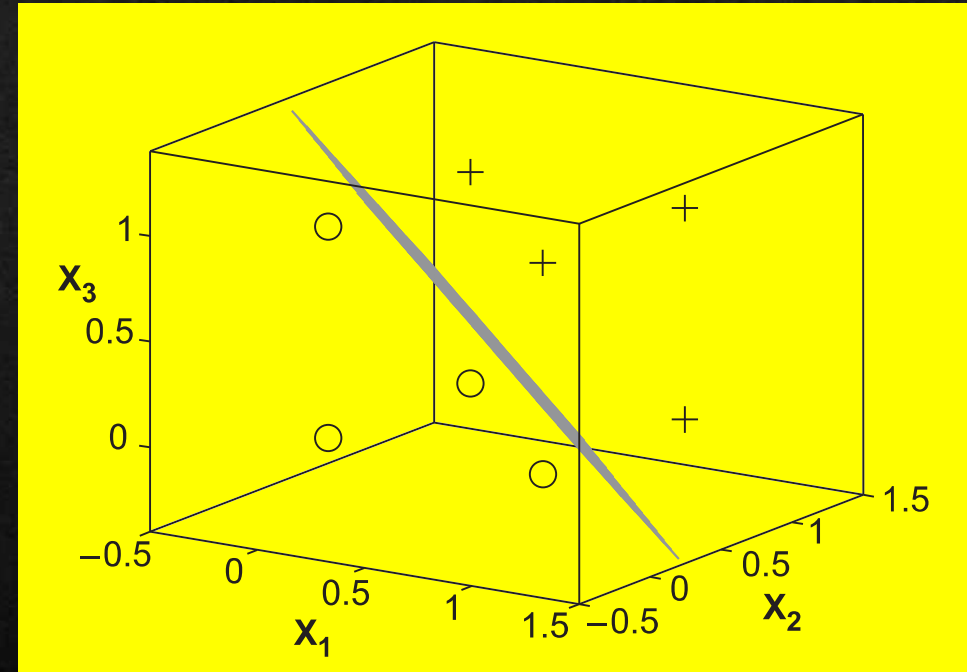
Epoch	w_0	w_1	w_2	w_3
0	0	0	0	0
1	-0.2	0	0.2	0.2
2	-0.2	0	0.4	0.2
3	-0.4	0	0.4	0.2
4	-0.4	0.2	0.4	0.4
5	-0.6	0.2	0.4	0.2
6	-0.6	0.4	0.4	0.2

Weight updates over first epoch

Weight updates over all epochs

Perceptron Learning

- ◆ Since y is a linear combination of input variables, decision boundary is linear



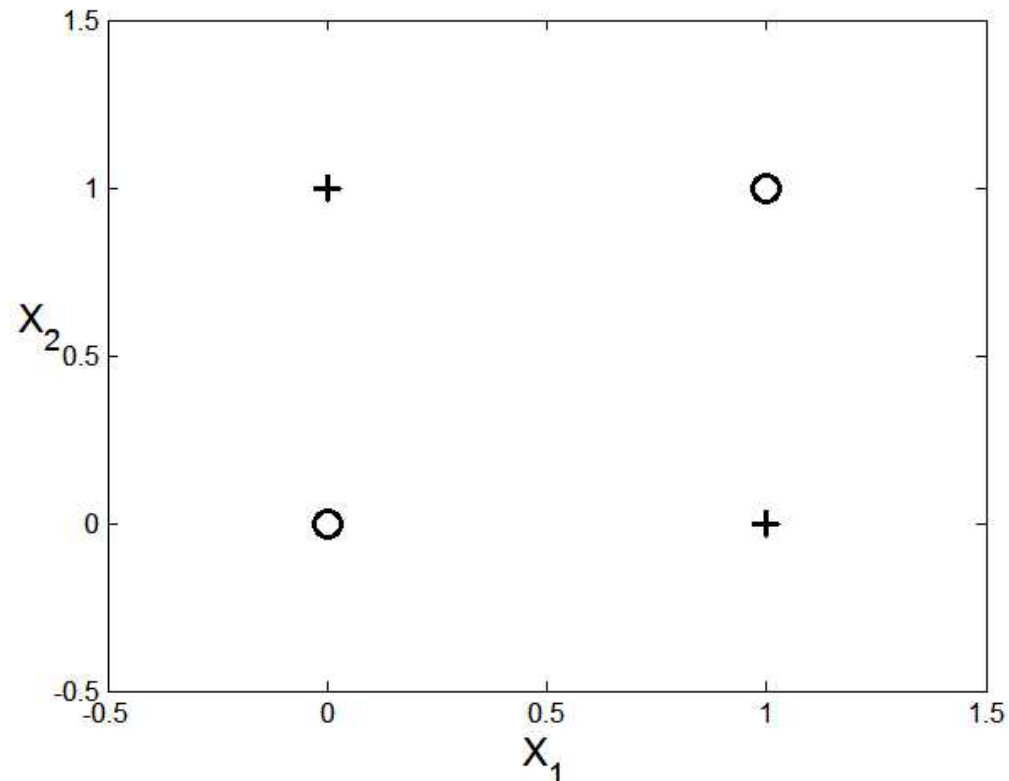
Nonlinearly Separable Data

For nonlinearly separable problems, the perceptron learning algorithm will fail because no linear hyperplane can separate the data perfectly

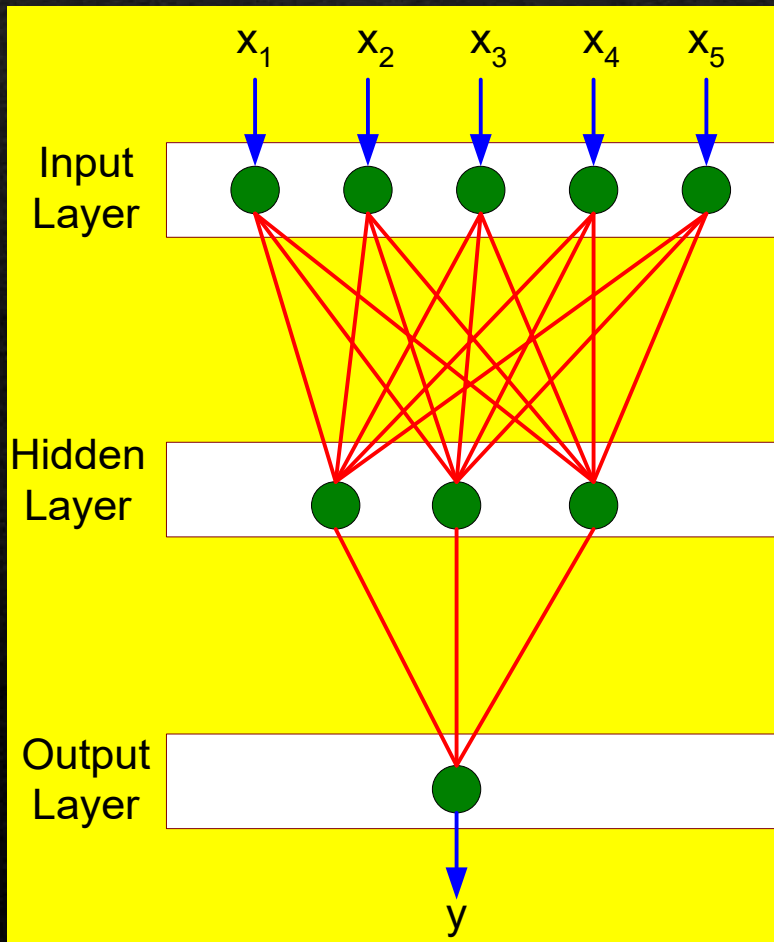
XOR Data

$$y = x_1 \oplus x_2$$

x_1	x_2	y
0	0	-1
1	0	1
0	1	1
1	1	-1



Multi-layer Neural Network

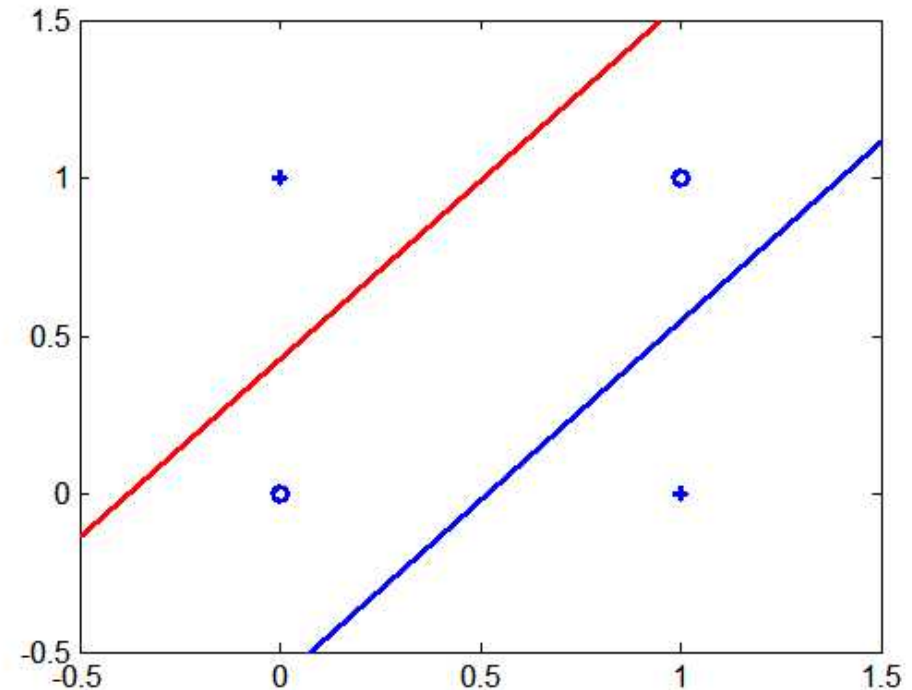
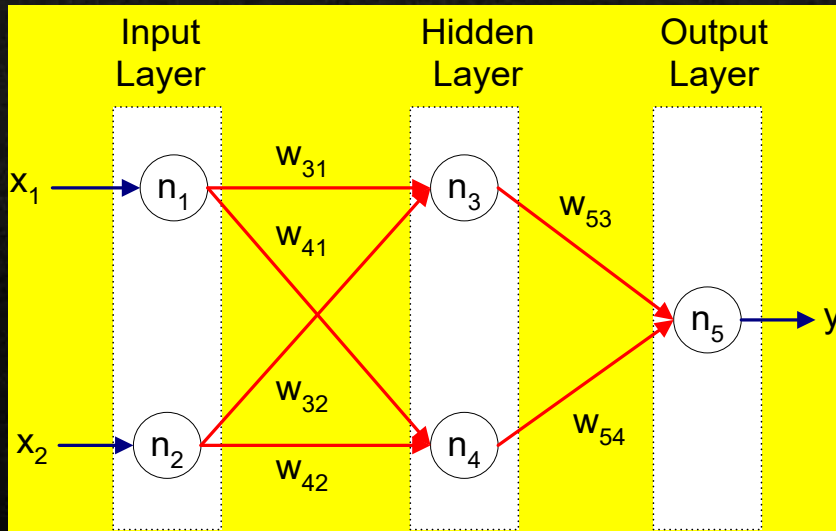


- ◆ More than one *hidden layer* of computing nodes
- ◆ Every node in a hidden layer operates on activations from the preceding layer and transmits activations forward to nodes of the next layer
- ◆ Also referred to as “feedforward neural networks”

Multi-layer Neural Network

- Multi-layer neural networks with at least one hidden layer can solve any type of classification task involving nonlinear decision surfaces

XOR Data



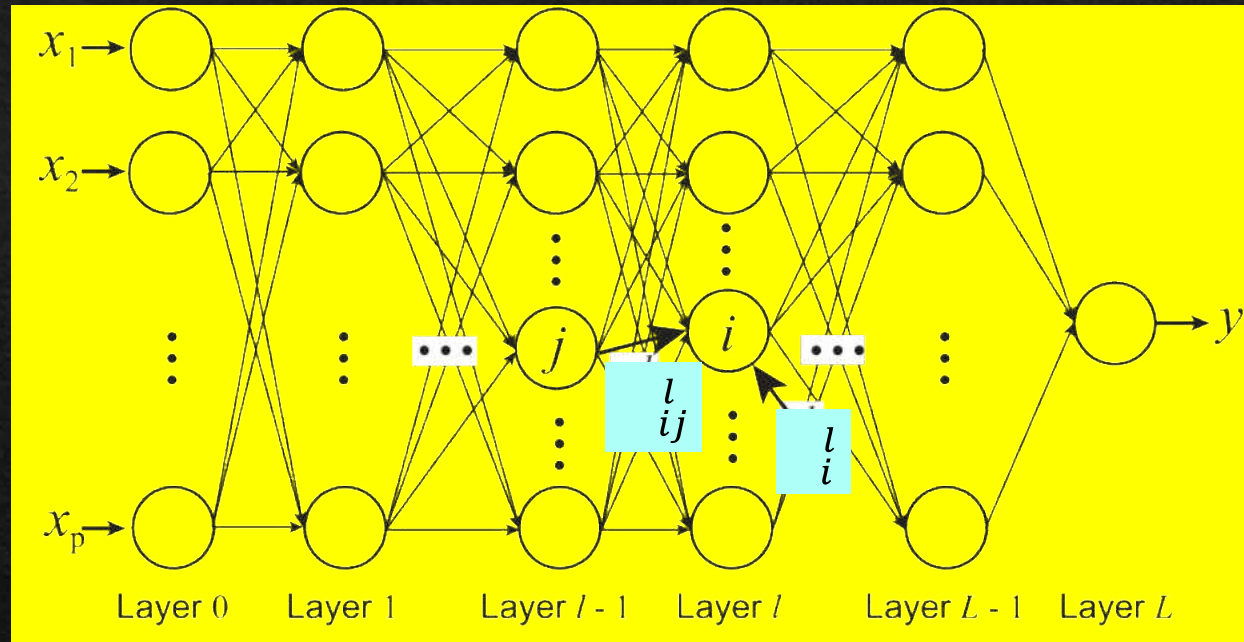
Why Multiple Hidden Layers?

- ◆ Activations at hidden layers can be viewed as features extracted as functions of inputs
- ◆ Every hidden layer represents a level of abstraction
 - *Complex features are compositions of simpler features*



- ◆ The number of layers is known as the **depth** of the ANN
 - *Deeper networks express a complex hierarchy of features*

Multi-Layer Network Architecture



$$a_i^l = f(z_i^l) = f\left(\sum_j w_{ij}^l a_j^{l-1} + b_i^l\right)$$

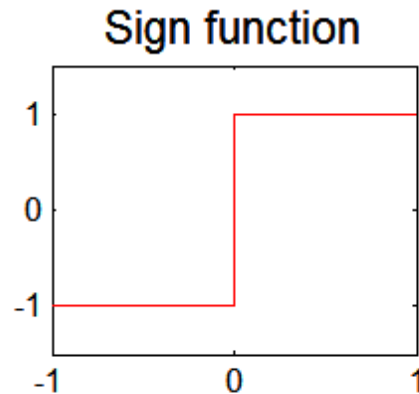
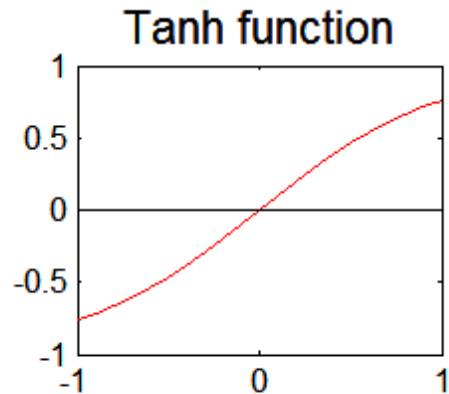
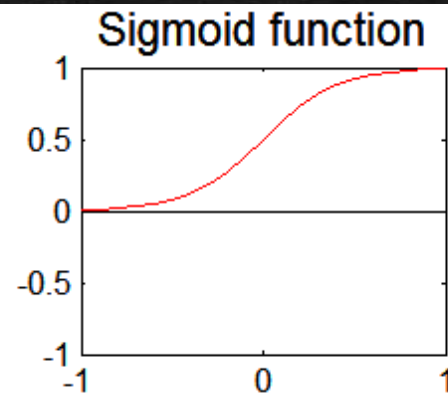
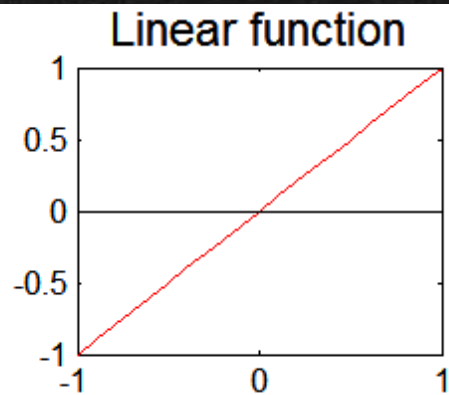
Activation value at
node i at layer l

Activation
Function

Linear Predictor

Activation Functions

$$a_i^l = f(z_i^l) = f\left(\sum_j w_{ij}^l a_j^{l-1} + b_i^l\right)$$



$$a_i^l = \sigma(z_i^l) = \frac{1}{1 + e^{-z_i^l}}$$
$$\frac{\partial a_i^l}{\partial z_i^l} = \frac{\partial \sigma(z_i^l)}{\partial z_i^l} = a_i^l(1 - a_i^l)$$

Learning Multi-layer Neural Network

- ◆ Can we apply the perceptron learning rule to each node, including hidden nodes?
 - Perceptron learning rule computes error term $e = y - \hat{y}$ and updates weights accordingly
 - ◆ Problem: how to determine the true value of y for hidden nodes?
 - Approximate error in hidden nodes by error in the output nodes
 - ◆ Problem:
 - Not clear how adjustment in the hidden nodes affects overall error
 - No guarantee of convergence to optimal solution

Gradient Descent

- ◆ Loss Function to measure errors across all training points

$$E(\mathbf{w}, \mathbf{b}) = \sum_{k=1}^n \text{Loss}(y_k, \hat{y}_k)$$

Squared Loss:

$$\text{Loss}(y_k, \hat{y}_k) = (y_k - \hat{y}_k)^2$$

- ◆ Gradient descent: Update parameters in the direction of “maximum descent” in the loss function across all points

$$\begin{aligned} w_{ij}^l &\leftarrow w_{ij}^l - \lambda \frac{\partial E}{\partial w_{ij}^l}, \\ b_i^l &\leftarrow b_i^l - \lambda \frac{\partial E}{\partial b_i^l}, \end{aligned}$$

λ : learning rate

- ◆ Stochastic gradient descent (SGD): update the weight for every instance (minibatch SGD: update over min-batches of instances)

Computing Gradients

$$\frac{\partial L_{ij}}{\partial w_j^l} = \sum_{k=1}^n \frac{\partial \text{Loss}(\hat{y}_k, y_k)}{\partial w_j^l}.$$

$$\hat{y} = a^L$$

$$a_i^l = f(z_i^l) = f\left(\sum_j w_{ij}^l a_j^{l-1} + b_i^l\right)$$

◆ Using chain rule of differentiation (on a single instance):

$$\frac{\partial \text{Loss}}{\partial w_{ij}^l} = \frac{\partial \text{Loss}}{\partial a_i^l} \times \frac{\partial a_i^l}{\partial z_i^l} \times \frac{\partial z_i^l}{\partial w_{ij}^l}.$$

◆ For sigmoid activation function:

$$\frac{\partial \text{Loss}}{\partial w_{ij}^l} = \delta_i^l \times a_i^l (1 - a_i^l) \times a_j^{l-1},$$

where $\delta_i^l = \frac{\partial \text{Loss}}{\partial a_i^l}.$

◆ How can we compute δ_i^l for every layer?

Backpropagation Algorithm

- ◆ At output layer L:

$$\delta^L = \frac{\partial \text{Loss}}{\partial a^L} = \frac{\partial (y - a^L)^2}{\partial a^L} = 2(a^L - y).$$

- ◆ At a hidden layer l (using chain rule):

$$\delta_j^l = \sum_i (\delta_i^{l+1} \times a_i^{l+1} (1 - a_i^{l+1}) \times w_{ij}^{l+1}).$$

- ▢ Gradients at layer l can be computed using gradients at layer $l + 1$
- ▢ Start from layer L and “backpropagate” gradients to all previous layers
- ◆ Use gradient descent to update weights at every epoch
- ◆ For the next epoch, use updated weights to compute loss fn. and its gradient
- ◆ Iterate until convergence (loss does not change)

Design Issues in ANN

- ◆ Number of nodes in the input layer
 - One input node per **binary/continuous** attribute
 - K or $\log_2 k$ nodes for each **categorical** attribute with k values
- ◆ Number of nodes in the output layer
 - One output for the binary class problem
 - k or $\log_2 k$ nodes for k -class problem
- ◆ Number of hidden layers and nodes per layer
- ◆ Initial weights and biases
- ◆ Learning rate, max. number of epochs, mini-batch size for mini-batch SGD, ...

Characteristics of ANN

- ◆ Multilayer ANN are universal approximators but could suffer from overfitting if the network is too large
 - Naturally represents a hierarchy of features at multiple levels of abstractions
- ◆ Gradient descent may converge to a local minimum
- ◆ Model building is compute-intensive, but testing is fast
- ◆ Can handle redundant and irrelevant attributes because weights are automatically learned for all attributes
- ◆ Sensitive to noise in training data
 - This issue can be addressed by incorporating model complexity in the loss function
- ◆ Difficult to handle missing attributes

Deep Learning Trends

- ◆ Training **deep** neural networks (more than 5-10 layers) could only be possible in recent times with:
 - Faster computing resources (GPU)
 - Larger labeled training sets
- ◆ Algorithmic Improvements in Deep Learning
 - Responsive activation functions (e.g., RELU)
 - Regularization (e.g., Dropout)
 - Supervised pre-training
 - Unsupervised pre-training (auto-encoders)
- ◆ Specialized ANN Architectures:
 - Convolutional Neural Networks (for image data)
 - Recurrent Neural Networks (for sequence data)
 - Residual Networks (with skip connections)
- ◆ Generative Models: Generative Adversarial Networks